

Similitud de firmas manuscritas, caso práctico para validación automática de documentos electorales.

Tesina de Grado para la obtención del grado de Licenciado en Ciencias de la Computación

Autor: Mariano Crosetti C-5994/3

Director: Dante Zanarini
Co-director: Pablo Granitto

31 de diciembre de 2024

Agradecimientos

poner agradecimientos acá. antes va el resumen.

Índice general

1. Resumen **5**

2. Introducción **6**
2.1. Motivación . . . 6
2.2. Organización . . . 6

3. Marco Teórico **8**
3.1. Conceptos básicos . . . 8
3.1.1. Inteligencia artificial . . . 8
3.1.2. Aprendizaje automático . . . 9
3.1.3. Terminología del Aprendizaje Automático . . . 9
3.1.4. Dilema sesgo-varianza y sobreajuste . . . 11
3.1.5. Conjunto de validación . . . 14
3.1.5.1. Validación cruzada . . . 14
3.1.6. Elección de hiperparámetros . . . 15
3.1.7. Conjunto de test . . . 16
3.1.8. Métricas . . . 17
3.1.8.1. Métricas en problemas de regresión . . . 17
3.1.8.2. Métricas en problemas de clasificación . . . 17
3.1.8.3. Métricas en problemas de clasificación binaria . . . 19
3.1.8.4. Métricas en el problema de Verificación Automática de Firmas Manuscritas 20
3.1.8.5. Análisis costo-beneficio para las métricas en clasificación binaria . . . 21
3.1.9. Importancia de la Metodología de Evaluación . . . 24
3.1.10. Reducción de la dimensionalidad . . . 24
3.2. Clasificadores . . . 25
3.2.1. Random Forest . . . 25
3.2.1.1. Árboles de Decisión . . . 25
3.2.1.2. *Classification And Regression Tree (CART)* . . . 26
3.2.1.3. *Random Forest* . . . 27
3.2.2. Support Vector Machine . . . 28
3.3. Redes Neuronales Artificiales . . . 30
3.3.1. Un poco de historia . . . 31
3.3.2. Formulación actual . . . 33
3.3.3. Descenso por gradiente . . . 35
3.3.3.1. Sobre funciones de costo "*realistas*" . . . 35
3.3.3.2. Descenso por gradiente estocástico . . . 37
3.3.3.3. Importancia de la elección adecuada del *learning rate* . . . 38
3.3.4. Función de costo . . . 38
3.3.4.1. Linearidad de gradientes respecto a última capa de activaciones . . . 39
3.3.5. Funciones de activación . . . 39
3.3.6. Teorema Aproximación Universal . . . 41
3.3.7. Algoritmo de *backpropagation* . . . 42

3.3.8. Extensiones al algoritmo de descenso por gradiente . . . 43
3.3.8.1. Momentum . . . 44
3.3.8.2. RMSProp . . . 44
3.3.8.3. Adam . . . 45
3.3.9. Inicialización de los parámetros . . . 45
3.3.10. Mejorando la generalización - técnicas de regularización . . . 45
3.3.10.1. *Data augmentation* . . . 46
3.3.10.2. *Batch normalization* . . . 47
3.3.10.3. *Weigth decay* . . . 48
3.3.10.4. *Dropout* . . . 49
3.3.11. Configurando el *Learning Rate* . . . 50
3.3.11.1. Seteando el *Learning Rate* inicial . . . 50
3.3.11.2. *Cycle Learning Rates* . . . 50
3.3.11.3. *One Cycle Policy* . . . 51
3.3.12. Aprendiendo las representaciones . . . 52
3.3.13. Redes Neuronales Convolucionales . . . 53
3.3.13.1. Convoluciones . . . 54
3.3.13.2. Aritmética de las convoluciones . . . 54
3.3.13.2.1. *Padding* y *Stride* . . . 55
3.3.13.2.2. *Pooling* . . . 56
3.3.13.3. *Receptive field* . . . 57
3.3.13.4. Visualizando las características aprendidas . . . 58
3.3.13.5. Redes completamente convolucionales . . . 58
3.3.13.6. *ResNets* . . . 60
3.3.13.7. *Fine-tuning* . . . 61
3.3.14. Arquitecturas de redes siamesas . . . 63
3.3.14.1. Triplet loss . . . 63

4. Formulación del problema **65**
4.1. Preliminares . . . 65
4.2. Sobre los telegramas de votación en Santa Fe . . . 65
4.3. Definiendo los objetivos . . . 67
4.3.1. Modelado del problema . . . 67
4.3.2. Definición de los experimentos . . . 67
4.3.2.1. Desarrollo: entrenamiento y validación . . . 68
4.3.2.2. Construcción del conjunto de pares . . . 68
4.3.3. Elección de la métrica . . . 68
4.4. Conjuntos de datos utilizados . . . 69
4.4.1. Experimento 1 . . . 69
4.4.2. Experimento 2 . . . 69

5. Trabajos relacionados y estado del arte **70**
5.1. Verificación Automática de Firmas Manuscritas . . . 70
5.1.1. Categorías de firmas . . . 71
5.1.2. Proceso *clásico* de VAFM . . . 71
5.1.3. Extracción de características . . . 73
5.1.3.1. Invarianza rotacional mediante DFT . . . 74
5.1.4. Reutilizando las técnicas en nuestro problema . . . 74
5.1.4.1. Entrenamiento de clasificadores y evaluación . . . 74
5.1.4.2. Métricas y convenciones . . . 75
5.1.5. Resultados de [Parodi 2014] . . . 75
5.1.5.1. Críticas metodológicas . . . 76
5.1.5.1.1. Desbalance en el conjunto de datos de entrenamiento . . . 76
5.1.5.1.2. Error metodológico en la evaluación de firmas ficticias . . . 76

5.1.5.1.3. Desbalance en el conjunto de datos de evaluación 76
5.2. Conjuntos de datos 76
5.2.1. GPDS300 76
5.3. Problemas de similitud 77
5.3.1. Problemas clásicos similares: *Face Recognition* y *Face Verification* 77
5.3.2. *One-Shot Learning* 78
5.3.3. *Redes Siamesas* 78
5.3.4. 79
5.3.5. *Image Similarity* 79
5.3.6. *Omniglot Dataset* 79

6. Experimentos **81**
6.1. Formulación del problema en la práctica 81
6.2. Conjuntos de datos 81
6.2.1. Conjuntos de datos curados 81
6.2.1.0.1. SigComp 2009 Train (NISDICC). 82
6.2.1.0.2. SigComp 2009 Test (NFI). 82
6.2.2. Confección del *dataset* propio (cropeo de firmas) 82
6.3. Resultados 82
6.3.1. Resultados utilizando la tesis de Marianela 82
6.3.2. Resultados utilizando redes siamesas 82
6.3.3. Resultados con diferentes arquitecturas de redes siamesas 83
6.3.4. Online triplet loss y offline triplet loss 83

7. Conclusiones **84**

A. Construcción del conjunto de datos **85**

B. Trabajo futuro **86**

C. Bibliografía **88**

Capítulo 1

Resumen

En esta tesina se aborda un problema práctico vinculado al proceso de escrutinio provisorio con asistencia digital de la Provincia de Santa Fe: detectar automáticamente errores de confección en telegramas electorales verificando que las firmas manuscritas de las autoridades de mesa estén presentes y sean consistentes a lo largo de todas las páginas del documento. A diferencia del problema clásico de *Verificación Automática de Firmas Manuscritas* (VAFM), aquí no se busca detectar falsificaciones sino inconsistencias de autoría entre firmas que deberían corresponder a la misma persona, con el objetivo de reducir retransmisiones y trabajo humano innecesario en el centro de cómputo [Informe 2019 UNR GSF; Parodi 2014].

El problema se formula como una tarea de clasificación binaria donde, dados dos recortes de firmas, se debe decidir si fueron realizadas por el mismo autor o no [Sammut et al. 2012]. Para ello se estudian conceptos fundamentales de Aprendizaje Automático y Aprendizaje Profundo, prestando especial atención al diseño correcto de los conjuntos de entrenamiento, validación y test, así como a la elección de métricas robustas al desbalance y acordes al uso práctico del sistema (*precision*, *recall*, curvas ROC y Precision–Recall, etc.). La tesina contrasta enfoques basados en extracción manual de características y clasificadores clásicos (como el esquema SVM + grilla circular de [Parodi 2014]) con métodos modernos de redes neuronales convolucionales y arquitecturas siamesas para aprendizaje de representaciones.

En la fase experimental se parte de la confección de un conjunto de datos propio a partir de telegramas reales: se recortan automáticamente las zonas de firmas aprovechando que la ubicación es fija, y se entrena primero un clasificador para distinguir entre "hay firma" y "no hay firma", que sirve como herramienta para limpiar y estructurar el *dataset* definitivo de pares de firmas. Luego se entrenan distintos modelos de similitud: un *baseline* inspirado en la tesis de Parodi (features manuales + Random Forest/SVM) y diversos modelos de redes siamesas convolucionales, variando arquitectura, función de costo y estrategia de generación de pares y tripletas (incluyendo variantes *online* y *offline* de *triplet loss*).

Los resultados muestran que las arquitecturas siamesas convolucionales, entrenadas con funciones de costo orientadas a aprendizaje de métricas (como *triplet loss*), superan de manera clara a los métodos basados en extracción manual de características tanto en conjuntos de datos estándar de firmas como en el dominio específico de telegramas electorales. Se alcanza una región de operación con *recall* cercano a 0.9 y alta *precision*, consistente con el objetivo práctico de reducir a una fracción las retransmisiones debidas a errores de firmas, manteniendo a la vez un número manejable de falsas alarmas. Finalmente se discuten las limitaciones del enfoque (dependencia del conjunto de datos, supuestos sobre el formato de los telegramas, ausencia de falsificaciones intencionales) y se señalan líneas de trabajo futuro, entre ellas el uso de modelos generativos para enriquecer las representaciones y la aplicación del método a otros problemas de similitud de imágenes.

Capítulo 2

Introducción

2.1. Motivación

La motivación es resolver una problemática real y compleja utilizando técnicas del estado del arte de Visión computarizada. Se presenta la ruta para resolver problemas prácticos a necesidades reales donde todas las etapas requieren de especial cuidado: recolección de datos, la formulación del problema, el planteo de métricas acordes, la confección rápida de una base de referencia (*baseline*), la elección, entrenamiento y evaluación del modelo y finalmente la implementación en el entorno real.

Como caso práctico se analizarán telegramas electorales[1], constatando que las firmas manuscritas de las autoridades de mesas[2] estén presentes y sean coincidentes en cada una de las hojas[3]. De ese modo se pretende detectar fallas en la confección (como que se confundan de casillero[4]) de forma anticipada, o sea, antes del análisis manual por parte del centro de cómputo, ahorrando trabajo humano.

Elegimos este problema puesto que es interesante: no es un clásico problema de clasificación. Sino que nuestro algoritmo deberá ser suficientemente robusto como para medir la similitud entre *tipos de firmas nunca antes vistas*, de modo que pueda evaluar si dos firmas *son del mismo tipo*[5].

Para resolver la problemática planteada, analizamos técnicas del *Estado del Arte* de *Visión Computarizada* que atañen tanto al tratamiento de imágenes de firmas, como métodos que se utilizan para resolver el problema de similitud de imágenes en general. Comparamos además la *performance* de métodos modernos de *Aprendizaje Profundo* contra clasificadores basados en extracción manual de características propuesto en una Tesis doctoral del 2014 [Parodi 2014], mostrando mejoras sustanciales. Y finalmente, desarrollamos un método propio para el entrenamiento de representaciones, que mostramos ampliamente superador a las más prometedoras técnicas del *Estado del Arte* que hemos probado.

2.2. Organización

La tesina descompone el problema en fases: formulación del problema, elección de métricas, se presenta el problema de construir rápidamente un conjunto de datos etiquetados utilizando un *baseline naive*. Luego se presenta una solución anterior a la fase del *deep learning*. Se presentan métodos de

[1] Documentos que se confeccionan para transmitir el resultado de una mesa de votación

[2] Las autoridades de mesa son el/la presidente de mesa y los/las auxiliares. Para más detalle sobre el proceso de escrutinio provisorio con asistencia digital véase [Informe 2019 UNR GSF].

[3] Las autoridades de mesas deben firmar todas las hojas del telegrama

[4] Cada hoja del telegrama cuenta con casilleros específicos para cada autoridad. En el capítulo 4 se muestra un ejemplo de telegrama electoral utilizado durante las elecciones provinciales de Santa Fe 2019, tal como muestra la figura 4.1

[5] Entendiéndose por *mismo tipo* a si son de la misma autoría. Ahondaremos en la clasificación de las firmas manuscritas en la sección 5.1 donde se hablará del problema de *Verificación Automática de Firmas Manuscritas* (VAFM).

vanguardia para manejar los hiper parámetros (como *Cycle Policy Learning Rate*) y se toman decisiones en cuánto a la arquitectura elegida en base a trabajos preexistentes.

Capítulo 3

Marco Teórico

3.1. Conceptos básicos

En la presente sección se pretende introducir los conceptos básicos de Inteligencia Artificial, Aprendizaje Automático (*Machine Learning*), Aprendizaje Profundo (*Deep Learning*) y Visión Computarizada (*Computer Vision*) necesarios para comprender el presente trabajo.

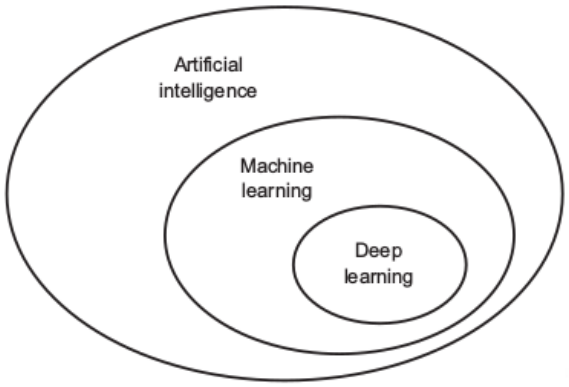


Figura 3.1: Inteligencia Artificial, Aprendizaje Automático y Aprendizaje Profundo. [Chollet 2017]

3.1.1. Inteligencia artificial

En 1956, los padres de la *Inteligencia Artificial*: Allen Newell, Herbert Simon, John McCarthy, Marvin Minsky y Arthur Samuel acuñaron el término *"Inteligencia Artificial"* como *"una rama de las Ciencias de la Computación que tiene como objetivo simular comportamiento inteligente en una computadora."* [Samuel 1962] [Minsky and Papert 1969]

En un principio dichos sistemas estaban dotados de una gran cantidad de reglas que los hacían muy efectivos para resolver ciertos problemas donde los pasos están bien definidos y son lógicos.

Entre las décadas del '50 y '80 se creyó que con un conjunto suficientemente grande de reglas se podría lograr una inteligencia al nivel de los humanos. No obstante ciertos problemas como el reconocimiento de objetos en imágenes o de transcripción de texto a partir de sonido son difíciles de tratar con este enfoque, a pesar de que los humanos lo pueden realizar sin dificultad. Y esencialmente la dificultad de estos problemas radica en que es complejo obtener un conjunto de reglas o pasos algorítmicos para resolverlos.

3.1.2. Aprendizaje automático

En 1962 Arthur Samuel en su ensayo *"Artificial Intelligence: A Frontier of Automation"* [Samuel 1962] propone construir algoritmos donde el flujo del programa dependa de ciertos pesos asignados por un mecanismo en función de la *performance* calculada, de modo que las máquinas "aprendan" de su experiencia. Éste fue un puntapié inicial para la fundación del campo del *aprendizaje automático*, el cual consiste en el estudio de algoritmos que mejoran automáticamente su rendimiento gracias a la experiencia.

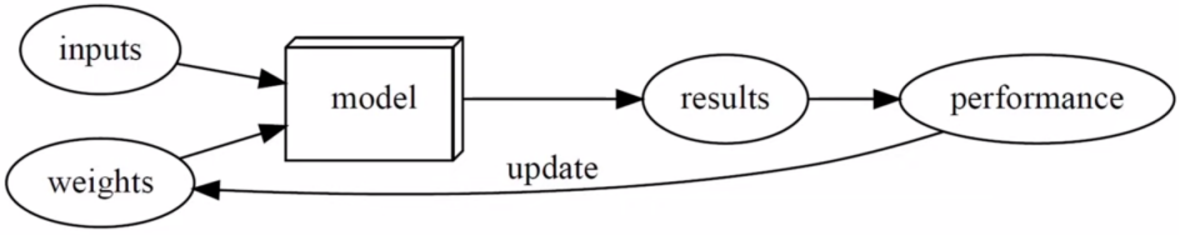


Figura 3.2: Esquema de la estructura propuesta por Arthur Samuel. [Howard and Gugger 2020]

Tom Mitchell define en su célebre libro *"Machine learning"* (1997, [Mitchell 1997]) al aprendizaje automático como:

> "Se dice que un programa de computadora aprende de la experiencia E respecto a una tarea T y una medida de desempeño P , si el desempeño medido con P en una tarea T , mejora con la experiencia E."

De este modo los algoritmos de aprendizaje automático construyen un modelo con el objeto de realizar predicciones o tomar decisiones sin haber sido expresamente programados para hacerlo. El modelo es construido en base a entradas suministradas.

La flexibilidad de las técnicas han ido aumentando y actualmente se pueden aplicar a gran variedad de problemas como reconocimiento de imágenes, sonidos, generación de texto, entre otros.
Dependiendo del tipo de experiencia suministrada se suele distinguir entre distintos problemas de aprendizaje [Russell and Norvig 2009]:

- **Supervisado:** se suministran los resultados esperados para las entradas ejemplo. El objetivo del modelo es "aprender" la relación entre las entradas y las salidas de modo que frente a nuevas entradas produzca razonables resultados. Es por ello que a menudo a estos últimos se los llamará también *predicciones*.
- **No supervisado:** sólo hay entradas (no hay salidas) y el objetivo es entender la estructura fundamental de los datos y aprender relaciones ocultas importantes.

Dependiendo de las predicciones del modelo en el caso supervisado hablaremos de:

- **Regresión:** si las predicciones son valores continuos.
- **Clasificación:** Si las predicciones pertenece a un dominio de valores discreto. A su vez hablaremos de *clasificación binaria* si las predicciones poseen sólo dos valores posibles.

En el presente trabajo trataremos un problema de clasificación binaria mediante *aprendizaje supervisado*.

3.1.3. Terminología del Aprendizaje Automático

En esta sección veremos con más formalidad los conceptos que atañen al *Aprendizaje Automático*. Además introduciremos la jerga específica utilizada en el mismo. Una vez leídas todas las definiciones

recomendamos volver a la sección 3.1.2 (*"Aprendizaje Automático"*) para constatar que detrás de la rigurosidad de símbolos y terminología específica se esconde sencillamente el modelo imaginado por Arthur Samuel en 1962. [Samuel 1962]

Como se aclaró en la sección anterior, en el presente trabajo trabajaremos con un problema de aprendizaje supervisado. En dichos problemas tenemos un conjunto de ejemplos D formado por entradas para las cuáles poseemos una (única) salida conocida. A este conjunto se lo denomina **conjunto de datos** (o ***dataset***):

$$D = \{(x, y)\} \quad D \subset A \times B,$$

donde x es un vector real de p dimensiones ($x \in \mathbb{R}^p$) llamado **variable independiente** o **vector de características**. A cada componente del mismo se lo llama **característica** (o ***feature***). Por otro lado, y es llamada **variable dependiente u objetivo** (o ***target***). En caso de tratarse de un problema de regresión y es una variable continua ($B \subseteq \mathbb{R}$). Si tenemos un problema de clasificación se la llama también **etiqueta** (o ***label***). En este caso B será el conjunto de clases posibles ($|B| \in \mathbb{N}$). Por ejemplo, $B = \{\text{“}gato''\text{, “}perro''\}$ para un clasificador que distinga fotografías de gatos y perros.

Llamaremos I al conjunto de todas las entradas y salidas posibles para nuestro problema. I generalmente es infinito puesto que todas las posibles entradas A suele ser infinito:

$$I = A \times B.$$

Volviendo al problema de clasificar imágenes de gatos y perros, A sería el conjunto (infinito) de todas las posibles imágenes que sean razonables recibir para dicho clasificador (imágenes donde haya algo discernible como gato o perro). A su vez, I será el conjunto de dichas imágenes junto su respectiva etiqueta.

Generalmente D es finito ($|D| \in \mathbb{N}$) ya que son los datos que poseemos (para los cuales conocemos certeramente una relación entre una entrada y su esperada salida). En este caso el conjunto de entradas disponibles D no es exhaustivo y existen muchas otras entradas factibles desconocidas que constituyen el **espacio problema**:

$$D \subset I.$$

En una aplicación real, los casos futuros a clasificar serán elementos de $I - D$.

El objetivo del problema es “aprender” una función $f : A \to B$ que le asigna a cada entrada una salida. El objetivo no es sólo que prediga bien D sino que posea una buena **generalización** al espacio problema I.

Como el conjunto de datos no es exhaustivo ($I - D \neq \emptyset$). Y existen infinitas entradas que no aparecen en el conjunto de datos, luego existen infinitas f consistentes con los datos (que los interpolan):

$$\mathbb{H} = \{f \mid f(x) = y \ \forall (x, y) \in D\}$$

Es por ello que debemos descartar algunas hipótesis, para lo cual se hacen suposiciones de la solución. Esto constituye lo que se llama **sesgo inductivo** o **base inductiva**.

Por ejemplo podemos hipotetizar que f es una combinación lineal de las características:

$$f(x) = \sum_{i=1}^{n} c_i.x_i$$

Nuestro **modelo** lo constituirán tanto dicha fórmula (producto de nuestras asunciones) como los valores de los coeficientes.

Las predicciones del modelo dependerán de la elección de dichos coeficientes, también denominados **parámetros** o **pesos** (o ***weights***). Para encontrar valores adecuados para los parámetros se utiliza

un **algoritmo de optimización**, teniendo en cuenta un subconjunto D_t de los datos disponibles D ($D_t \subset D$). Esto es lo que se conoce como **fase de aprendizaje**. Durante este proceso se habla de **ajuste de los datos de entrenamiento**, **ajuste de los pesos** o simplemente **ajuste del modelo**. A D_t se lo llama **conjunto de entrenamiento**. A continuación veremos por qué es importante no utilizar la totalidad de los datos disponibles para ajustar los pesos.

También, deberá existir una manera de medir la performance del modelo a partir de los resultados del mismo y de entradas y salidas conocidas. A esto se lo denomina **métrica**: es una función de las **etiquetas** y **predicciones**.

$$metric(f(x), y)$$

El objetivo del modelo es tener una buena performance sobre todo el conjunto de entrada I. Como se dijo, en la práctica los casos de $I - D$ pueden ser casos futuros sobre los que se utiliza el modelo para tomar decisiones en función de las predicciones.

Es muy importante contar con ejemplos adicionales D_v para los cuales se conozcan tanto entradas como salidas ($D_v \subset D$) y que no pertenezcan al conjunto que se utilizó para el ajuste del modelo ($D_t \cap D_v = \emptyset$). De modo que midiendo en D_v la performance podamos cuantificar la **capacidad de generalización** del modelo, entendiéndose a esta como la capacidad de extrapolar los resultados de *metric* a casos nos vistos durante el entrenamiento.

A dicho conjunto D_v lo denominamos **conjunto de validación**. Si el conjunto de validación ha sido bien elegido (más adelante veremos que $D_t \cap D_v = \emptyset$ es condición necesaria pero no siempre suficiente) las métricas calculadas en dicho conjunto serán una buena estimación de las métricas esperadas en el resto de los casos no observados:

$$\mathbb{E}_{(x,y)\in D_v}[metric(f(x), y)] \approx \mathbb{E}_{(x,y)\in E}[metric(f(x), y)]$$

Trasladado a un ejemplo práctico, esto quiere decir que nos dará una pauta confiable de la performance de nuestro modelo en la aplicación final. Es por ello que crear de forma correcta el conjunto de validación es crítico. La etapa de medir la performance del modelo sobre D_v suele llamarse **fase de evaluación**.

Las elecciones que constituyen la base inductiva del modelo (elección del modelo, variables de configuración del mismo, estrategias de entrenamiento, etc) constituyen "parámetros que actuarán sobre los parámetros del modelo". Es por ello que estas elecciones son denominadas **hiperparámetros**. Éstos no son modificados (o *ajustados*) por medio del mecanismo de optimización que actúa sobre los pesos, pero a menudo se prueban diferentes valores de hiperparámetros (entrenando diferentes modelos) y se elige el que tenga mayor performance en la fase de evaluación. Al usar el conjunto de validación de este modo el modelo finalmente elegido deja de ser independiente del conjunto de validación, por lo que se suele realizar una evaluación final sobre un tercer conjunto D_h ($D_h \subset D$ y disjunto respecto D_t y D_v) denominado **conjunto de test** para obtener la métrica de generalización del resultado final.

Es frecuente denominar **conjunto de desarrollo** a la unión del conjunto de entrenamiento y el conjunto de validación ($D_t \cup D_v$). El nombre proviene de que estos datos se utilizan durante el *desarrollo* de la solución: el conjunto de entrenamiento para ajustar los parámetros del modelo y el conjunto de validación para elegir los hiperparámetros. El conjunto de test, en cambio, se reserva exclusivamente para la evaluación final y no debe influir en ninguna decisión de diseño.

Luego de evaluado un modelo, está en condiciones de utilizárselo en la aplicación final. La acción de aplicar el modelo a un ejemplo al cuál no se conoce su resultado suele denominarse **inferencia**.

3.1.4. Dilema sesgo-varianza y sobreajuste

Sea:

- Y la variable dependiente a predecir.
- X los atributos (variable independiente) en los que se basa la predicción.

- f la función a predecir (relación subyacente entre X e Y).
- ϵ el ruido en los datos, distribuido normalmente con media 0 y varianza σ

Tenemos la fórmula:

$$Y = f(X) + \epsilon$$

Sea f el modelo que intenta aproximar f, el error cuadrático de predicción para un ejemplo x resulta ser:

$$Error(x) = \mathbb{E}[(Y - f(x))^2]$$

En [Hastie et al. 2001] se definen:

$$Sesgo = \mathbb{E}[f(x)] - y]$$

$$Varianza = \mathbb{E}[\ (f(x) - \mathbb{E}[f(x)])^2\ ]$$

$$ErrorIrreducible = \sigma$$

Reescribiendo la fórmula del error como:

$$Error(x) = Sesgo^2 + Varianza + ErrorIrreducible$$

En donde cada término del error puede ser interpretados como causas diferentes del mismo.

Haciendo un análisis de la composición del error, interpretando cada causa:

- **Error irreducible**: producto de la varianza en la función que se quiere predecir, ruido en los datos, etc. Está fuera de nuestro control.
- **Sesgo**: diferencia entre la predicción promedio de nuestro modelo, y el valor correcto que intentamos predecir. Puede ser pensado como el error causado por asunciones simplistas del modelo, como por ejemplo tomar que la distribución de los datos es lineal.
- **Varianza**: es la variabilidad de la predicción de un modelo, es decir, cuánto varía $f(x)$ alrededor de su media. Modelos con alta varianza "prestan mucha atención" a los datos de entrenamiento, corriendo el riesgo de no generalizar tan bien en datos no incluidos en el entrenamiento.

Cuando ajustamos los datos de entrenamiento con mayor precisión correemos el riesgo de que el modelo se adapte demasiado a los datos de entrenamiento, y no generalice bien en los datos de validación. Este escenario es denominado **sobreajuste** (o ***overfitting***). Cuando esto ocurra, las métricas mostrarán mucho peores resultados en el conjunto de validación que en el de entrenamiento.

En contraste si no se ajusta lo suficiente a los datos de entrenamiento poseerá un alto sesgo. Esto es lo que se conoce como **subajuste** (o ***underfitting***). A diferencia del sobreajuste, en este caso el error se manifestará tanto en la validación como en el entrenamiento.

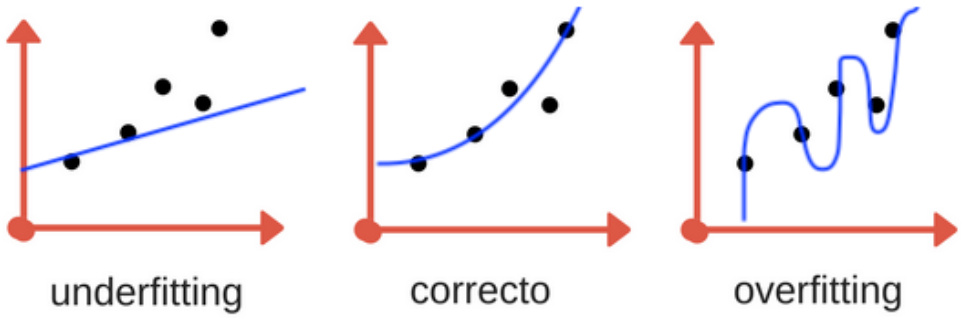


Figura 3.3: La imagen esquematiza que la elección de una base inductiva demasiado simple y poco expresiva puede ser el origen de un sesgo alto. A su vez, modelos complejos que se pueden ajustar demasiado a los datos de entrenamiento pueden generar sobreajuste.

Es por ello que los modelos más complejos, al tener más capacidad expresiva, pueden ajustar mejor los datos de entrenamiento (y potencialmente sobreajustar). El error de entrenamiento tiende a decrecer cuando incrementamos la complejidad del modelo.

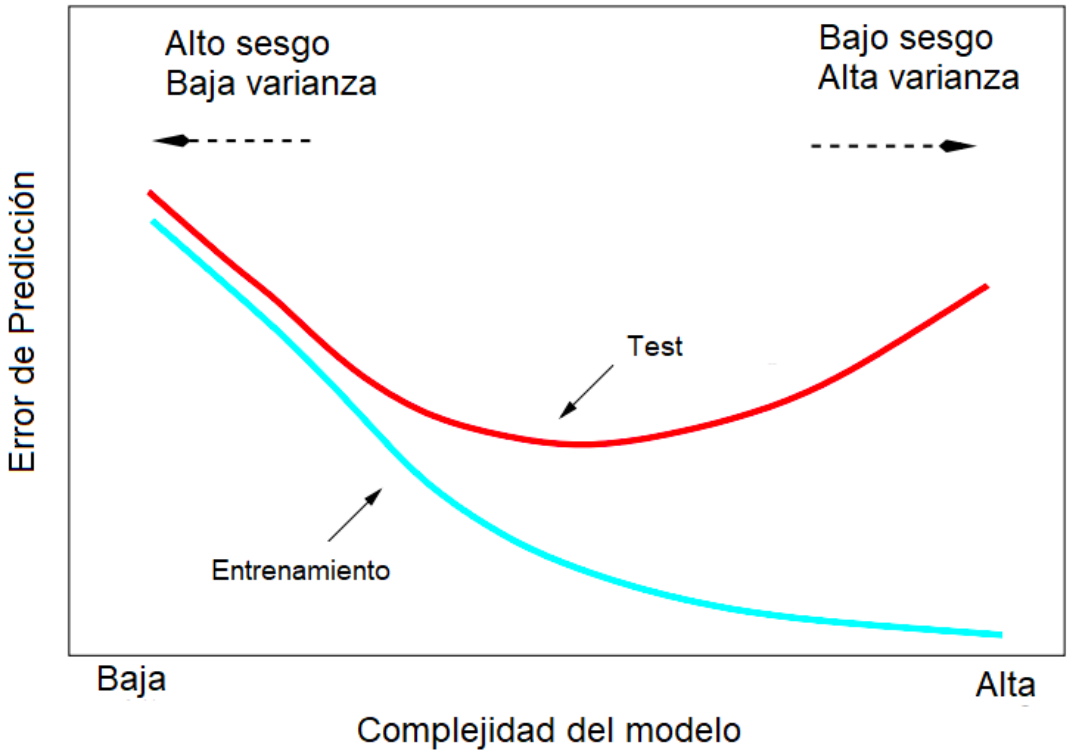


Figura 3.4: La gráfica esquematiza que cuando la complejidad de nuestro modelo crece, la varianza tiende a aumentar en tanto que el sesgo disminuye. Lo opuesto sucede cuando la complejidad del modelo disminuye. [Hastie et al. 2001]

Esto último no quiere decir que siempre la primer solución al sobreajuste sea reducir la complejidad del modelo. Como dijimos, esta última aumenta la capacidad expresiva y por ende la *capacidad* de sobreajustar. No obstante en métodos de optimización iterativos (como el empleado con las redes neuronales) podemos controlar la cantidad de ajuste por medio del número de iteraciones. Ya veremos que existen formas de combatir el sobreajuste y entrenar modelos más robustos llamados *técnicas de regularización*.

Más adelante veremos que reducir la complejidad de los modelos ante la presencia del sobreajuste no es -generalmente- la mejor opción en el caso del Aprendizaje Profundo. De hecho, la práctica demuestra que es la última de las opciones a intentar entre un gran abanico de soluciones. Como veremos en la subsección 3.3.12, en el aprendizaje profundo la gran complejidad de los modelos es lo que permite aproximar funciones complejas que reciban los *"datos crudos"* (por ejemplo valores de píxeles en caso de imágenes) y calculen las representaciones idóneas para el proceso de clasificación. Esto constituye el gran éxito del aprendizaje profundo por sobre anteriores enfoques que consistían en extraer características con algún algoritmo *ad-hoc* específico al dominio y tan solo luego utilizar un clasificador de aprendizaje automático.

El sobreajuste afecta a la mayoría de los métodos de Aprendizaje Automático. Es importante por ello contar con un conjunto de validación para medir las métricas elegidas en datos que no se usaron para entrenar.

3.1.5. Conjunto de validación

Como se dijo en secciones anteriores, se utiliza un **conjunto de datos de validación** D_v no incluídos en el entrenamiento para medir la capacidad de generalización del modelo.

Ya que nuestro objetivo final es aplicar el modelo en datos nunca antes vistos, D_v debe ser representativo del conjunto de entradas factibles E.

De manera obvia, los datos de validación deben no aparecer entre los datos de entrenamiento D_t:

$$D_v \cap D_t = \emptyset$$

A menudo se toma una partición aleatoria de los datos. Pero también debe evitarse cualquier correlación con los datos de entrenamiento D_t que no sea extensible al conjunto de entradas factibles E.

Un ejemplo dónde, claramente, no es válido partir aleatoriamente las entradas para construir los conjuntos de entrenamiento y validación es las series temporales (véase figura 3.5). De hacerlo es fácil para el modelo aprender valores certeros teniendo valores conocidos durante el entrenamiento dentro del entorno cercano del valor a predecir. Contrariamente a este escenario, en una aplicación real probablemente querramos predecir valores *futuros*. Por lo que una correcta partición tomaría un rango continuo de los últimos valores para confeccionar un conjunto de validación representativo.

Al ser un punto tan crítico la creación de un buen conjunto de validación suele ser fuente de numerosos errores en aplicaciones prácticas.

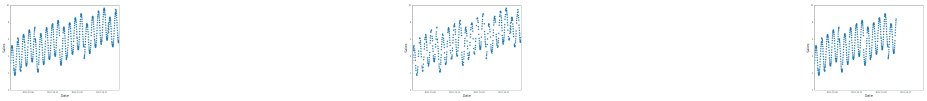


Figura 3.5: La primer figura muestra una serie temporal. La segunda muestra un subconjunto aleatoriamente seleccionado de ejemplos. La última muesta un subconjunto de entrenamiento correctamente seleccionado: siendo la validación un rango continuo de los últimos valores. Fuente: [Thomas 2017]

En conjuntos de datos académicos es frecuente que la separación en conjunto de validación ya esté dada con el mismo conjunto para que los resultados de diversas investigaciones sean comparables de forma correcta.

3.1.5.1. Validación cruzada

El método de validación cruzada (descripto en [Berrar 2018]) consiste en entrenar y evaluar modelos sobre diferentes particiones de entrenamiento y validación y combinar los resultados de los valores de las métricas obtenidos (por ejemplo tomando el promedio).

Existen diferentes estrategias de validación cruzada:

- **K-iteraciones** (o *K-folding*): consiste en particionar los datos disponibles en k grupos y hacer k iteraciones. En la i-ésima se utiliza el grupo i para validación y el resto se lo usa para entrenamiento.
- **Aleatoria**: consiste en hacer k iteraciones donde se elijen aleatoriamente la partición de validación. Como ventaja el tamaño de la partición no depende del número de iteraciones. Dentro de las desventajas destaca que existe la probabilidad de que existan ejemplos que no sean incluídos en ninguna validación.
- **Dejando uno afuera**: consiste en hacer N iteraciones (siendo N la cantidad de muestras disponibles). En la i-ésima iteración se utilizan todos los datos para entranar excepto el i-ésimo elemento del conjunto que se usa para evaluación.

La limitación de la validación cruzada es el gran esfuerzo computacional. Esto es debido a que se deben ejecutar k entrenamientos independientes.

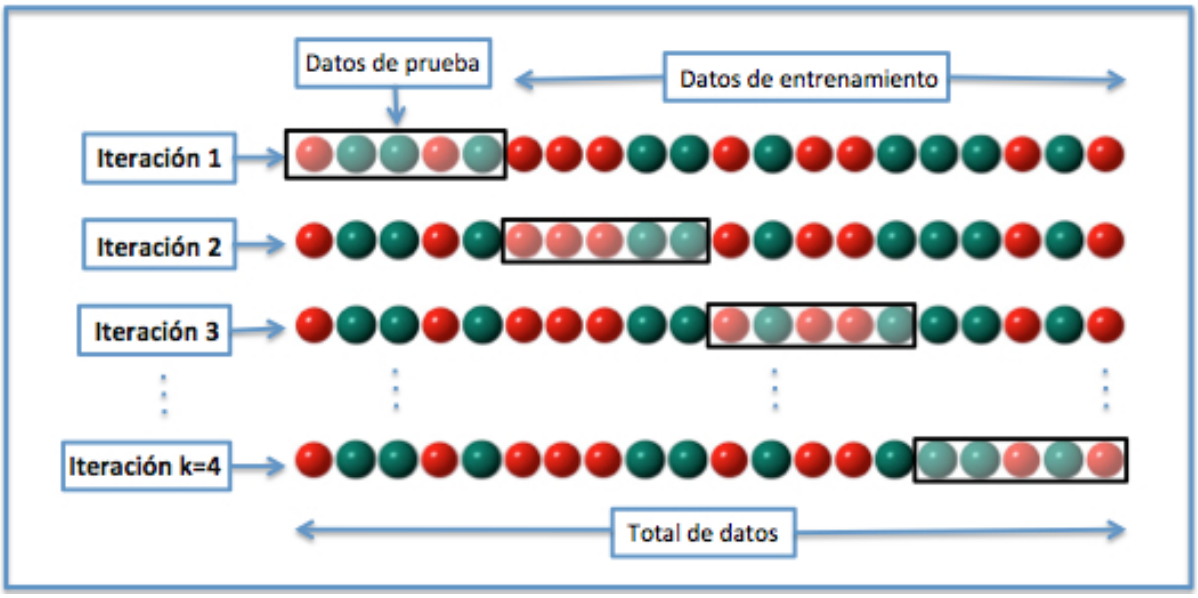


Figura 3.6: Método de validación cruzada de K-iteraciones. Fuente: [Joan.domenech91]

Por lo que se utiliza validación cruzada mayormente cuando se cuenta con pocos datos. Es muy útil, también, en casos donde el puntaje de validación suele no ser preciso debido a que varía demasiado al cambiar la partición de validación elegida (generalmente sucede cuando se posee pocos datos y el conjunto de validación es demasiado pequeño). Así, eligiendo diferentes particiones y promediando el puntaje, se obtiene un estimador de menor varianza del puntaje real o de generalización.

Un dato importante a tener en cuenta es que muchos esquemas de validación cruzada son sólo aplicable cuando uno puede particionar aleatoriamente los datos para obtener un conjunto de validación representativo.

3.1.6. Elección de hiperparámetros

Como se mencionó en anterioremente, generalmente se exploran diferentes elecciones de hiperparámetros. A este procedmiento a veces se lo conoce como **puesta a punto del modelo** o ***model tunning*** (no confundir con *fine tunning* que es la técnica de re-utilizar un modelo ya entrenado en otros conjuntos de datos, explicado más adelante en el trabajo).

Existen diferentes estrategias para la exploración de hiperparámetros tales como:

- ***Grid Search:*** se prueban todas las asignaciones posibles para conjuntos de valores propuestos para cada hiperparámetro.
- ***Random Search:*** se prueban asignaciones aleatorias de los hiperparámetros dentro de un conjunto de valores propuestos para cada uno.
- **Optimización Bayesiana:** utiliza el método bayesiano de optimización global para maximizar funciones (para una descripción detallada ver [Mockus 2012]). Suele obtener mejores resultados que el *Random Search* y requerir menor cómputo que *Grid Search* por su capacidad de razonar acerca de las asignaciones que no han sido probadas, y así elegir cuáles probar.
- **Algoritmos genéticos:** parten de un conjunto de asignación inicial y genera otras cambiando aletoriamente algunos valores de hiperparámetros. Luego de cada paso se queda con las mejores asignaciones (de mayor performance).
- **Manualmente:** En la práctica suele ser poco factible entrenar el modelo varias veces para demasiadas asignaciones de hiperparámetros. Esto es especialmente cierto en el *aprendizaje prufundo* donde el

tiempo de entrenamiento es considerable. Comúnmente se prueban manualmente hiperparámetros que son conocidos en la literatura que funcionan bien por defecto.

En general métodos exhaustivos como *Grid Search* no se utilizan en la práctica por la demanda de tiempo que conlleva. Se suele ver en la literatura científica y sólo tienen sentido si se desea arribar a conclusiones sobre determinados hiperparámetros (también es visto su uso en *competencias de modelos* donde diferencias mínimas en la performance permiten escalar en la tabla de posiciones).

Frecuentemente también se utilizan métricas y visualizaciones para analizar la estabilidad del entrenamiento, convergencia del proceso de optimización y otras heurísticas, por medio de las cuales podemos estimar si es necesario variar determinado hiperparámetro.

En el Capítulo 6 de FastBook [Howard and Gugger 2020], por ejemplo, se explica cómo elegir el umbral de aceptación para un problema de clasificación multicategoría dónde la salida del modelo son probabilidades. Para ello se visualiza cómo varía la tasa de aciertos (calculada en el conjunto de validación) para diferentes valores del umbral (ver figura 3.7). Ahí mismo se explica que, como *"regla heurística"* general, cuándo se eligen hiperparámetros de este modo raramente se estará sobreajustando el conjunto de validación si la gráfica de la métrica en función del hiperparámetros es suave.

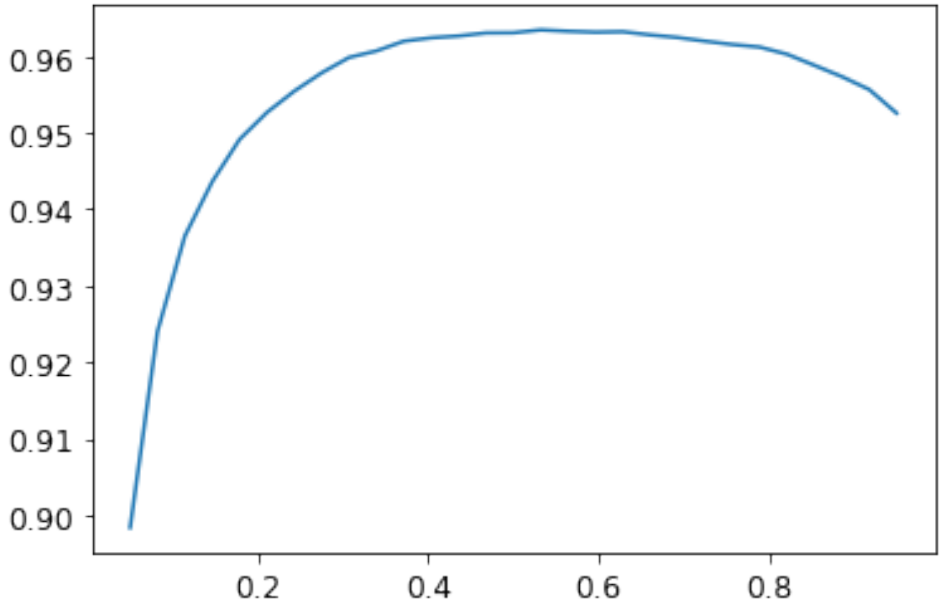


Figura 3.7: Tasa de aciertos en función al umbral de aceptación elegido para un problema de clasificación multi-categoría. Fuente: [Howard and Gugger 2020].

Veremos más adelante en el presente trabajo algunas técnicas populares en la práctica del aprendizaje profundo que se utilizan a menudo para elegir manualmente hiperparémetros, incluso en circunstancias como competencias de modelos dónde hilar fino en las métricas es muy importante.

3.1.7. Conjunto de test

Las diferentes asignaciones de hiperparámetros son comparadas teniendo en cuenta las métricas sobre el conjunto de validación. Si se realizan demasiadas pruebas se corre el riesgo de "sobreajustar los hiperparámetros" y que la performance que guia la elección de los mismos no generalice bien a conjuntos de datos no observados.

Como se explicó en secciones anteriores, se suele contar con un tercer conjunto de datos (además del de entrenamiento y de validación) denominado **conjunto de test**. El mismo se utiliza para realizar

una evaluación final no sesgada del modelo. Ésto puede parecer un despropósito pero es necesario para seguir una metodología correcta y mantenernos intelectualmente honestos.

3.1.8. Métricas

Ya hemos introducido el concepto de **métricas** en las secciones anteriores donde explicamos que son funciones para medir la performance del modelo.

Las **metricas** (pueden considerarse varias) son problema-dependientes debido a que dependen tanto de la naturaleza de las predicciones y de las entradas como del objetivo para el cuál se utilizará el modelo.

Deben poseer un significado interpertable por los usuarios en términos del uso que se le está dando al modelo.

No se debe confundir métrica con la *función de costo* que utiliza el mecanismo automático de optimización del modelo y a menudo posee propiedades matemáticas deseables por el mismo. El concepto se tratará más detalladamente cuando se hable de descenso por gradiente en redes neuronales (subsección 3.3.3).

3.1.8.1. Métricas en problemas de regresión

En problemas de regresión podemos encontrar las métricas:

- **Error cuadrático médio (MSE):** es una de las métricas más utilizadas en problemas de regresión puesto a qué es una función diferenciable. Pero penaliza los errores grandes, siendo suceptible a valores atípicos.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{f}(x_i))^2$$

- **Error absoluto médio (MAE):** no penaliza tanto a los errores grandes como MSE.

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{f}(x_i)|$$

- **Coeficiente de determinación (R^2)**: representa que proporción de la varianza de la variable dependiente y es explicada por las características de la variable independiente x.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{f}(x_i))^2}{\sum_{i=1}^{n}(y_i - \overline{y})^2}$$

  También puede ser pensada como una comparación del MSE de nuestro modelo con el del *baseline* trivial de predecir la media de las entradas.

3.1.8.2. Métricas en problemas de clasificación

En problemas de clasificación podemos encontrar las métricas:

- ***Accuracy*** es una de las métricas más utilizadas en problemas de clasificación. Consiste simplemente en la proporción de aciertos realizados. También se la conoce como **tasa de aciertos**.

$$Accuracy = \frac{1}{n}\sum_{i=1}^{n} Igual(y_i, \hat{f}(x_i))$$

$$Igual(a,b) = \begin{cases} 1 \; si \; a = b \\ 0 \; si \; a \neq b \end{cases}$$

- ***Top k - Accuracy:*** para problemas donde el número de clases es grande y existen varias fácilmente confundibles entre sí usualmente se modifica el problema de clasificación permitiendo a la predicción ser un conjunto de k elementos (para cierto k fijo) y se mide como acierto si la clase objetivo pertenece a este conjunto de predicciones.

$$Top\ k - Accuracy = \frac{1}{n}\sum_{i=1}^{n} Acierto(y_i, \hat{f}(x_i))$$

$$Acierto(etiqueta, predicciones) = \begin{cases} 1\ si\ etiqueta \in predicciones \\ 0\ si\ etiqueta \notin predicciones \end{cases}$$

- **Matriz de confusión:** a menudo resulta útil calcular una matriz donde los ejes sean las etiquetas y las predicciones. De modo de contener en la casilla $M_{i,j}$ cuántos ejemplos de la clase i han arrojado como resultado una predicción para la clase j. Los elementos de la diagonal representan los aciertos, mientras que los otros muestran errores. Es utilizada también debido a que confundir una clase por otra puede tener diferentes consecuencias prácticas y por lo tanto se puede preferir cierto tipo de errores a otros. Para comparar dos resultados y determinar cuál es mejor se pueden utilizar matrices de ponderación que le asignen a cada $M_{i,j}$ un costo $C_{i,j}$ ($C_{i,i} = 0$) de modo que el error quede definido como:

$$Error = \sum_i \sum_j M_{i,j} \times C_{i,j}$$

  De modo que $C_{i,j}$ refleje el costo práctico de cada tipo de error.

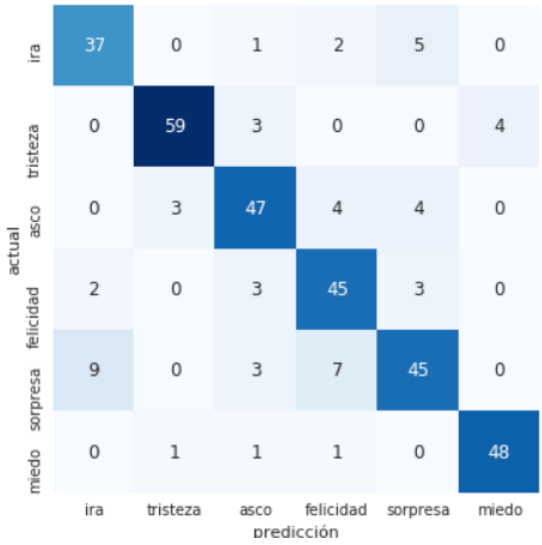


Figura 3.8: Ejemplo de una matriz de confusión para un problema de detección de emociones de 6 clases.

Dependiendo de la aplicación podemos querer optimizar diferentes métricas (y a menudo hay un *trade-off* entre ellas). Por ejemplo en un problema de clasificación binaria para detectar tumores querremos que no se le pase ningún tumor maligno como beningno, a costa de arrojar un número razonable de falsas alarmas.

Es importante **siempre calcular las métricas en el conjunto de validación**. Las métricas sobre el conjunto de entrenamiento tienen poco sentido, se monitorean con la única utilidad de detectar cuándo el modelo está sobreajustando.

3.1.8.3. Métricas en problemas de clasificación binaria

Como ya comentamos, en el presente trabajo trataremos un problema de clasificación binaria. Es por ello que haremos especial énfasis en las métricas que atañan a ésta subcategoría de problemas de clasificación.

En esta subcategoría de problemas suele denominarse a las clases como **positiva** y **negativa** [Sammut et al. 2012]. La decisión de a cuál clase definir como positiva y a cuál definir como negativa a veces es clara como por ejemplo en problemas de detección: la clase positiva será la etiqueta *detectado* y la negativa *no detectado*.

En nuestro caso querremos detectar errores en la confección de telegramas, buscando detectar *no coincidencias* de firmas. Por eso trataremos como positiva a las *no coincidencias*, ya que son las clases que se utilizará para desencadenar alertas en el sistema al cuál se pretende integrar la detección. En otros casos como en el problema de *"Verificación Automática de Firmas Manuscritas"* [Parodi 2014] se pretende verificar que la autoría de cierta firma es la que declara ser, comparándola con las de un autor registrado al que se le conocen ciertas firmas auténticas. En dicho trabajo se trabaja como clase positiva a la verificación de una firma como auténtica, o sea cuando coincide con las registradas del autor que declara ser. Es importante tener esto en cuenta a la hora de comparar métricas entre diferentes trabajos e implementaciones.

Luego de definidas las clases como positivas y negativas las predicciones del modelo pueden clasificarse en cuatro categorías en relación a su valor y al de la etiqueta respectiva según:

	Predicción Positiva	**Predicción Negativa**
Etiqueta Positiva	TP (Verdadero Positivo)	FN (Falso Negativo)
Etiqueta Negativa	FP (Falso Positivo)	TN (Verdadero Negativo)

Cuadro 3.1: Las abreviaciones TP, TN, FP, FN son populares en la literatura y provienen de los nombres en inglés, respectivamente: *True Positive*, *True Negative*, *False Positive* y *False Negative*

Definirimos también (para simplificar la notación en las fórmulas) las super-categorías (constituída por uniones de las cateogrías definidas):

- **Etiqueta Positiva**: conjunto de los datos que pertenecen a la clase positiva.

$$P = TP + FN$$

- **Etiqueta Negativa**: conjunto de los datos que pertenecen a la clase negativa.

$$N = TN + FP$$

- **Totalidad de datos**: conjunto de todos los datos usados para la evaluación.

$$Total = FP + FN + TP + TN$$

Luego se definen en función de las cardinalidades de dichas categorías diversas métricas para el problema de clasificación binaria (veáse [Sammut et al. 2012]), cada una con implicancias y significados distintos:

- **Tasa de aciertos** (o ***accuracy***): simplemente la proporción de ejemplos clasificados correctamente.

$$Accuracy = \frac{TP + TN}{Total}$$

- **Ratio de error** (o ***error rate***): es el concepto complementario al de tasa de aciertos, la proporción de errores cometidos.

$$Error\ Rate = \frac{FP + FN}{Total} = 1 - Accuracy$$

- **Exhaustividad** (***recall*** o **TPR** *True Positive Rate*): la proporción de ejemplo positivos detectados como tales.
$$Recall = \frac{TP}{TP + FN} = \frac{TP}{P}$$
En términos de probabilidad puede ser interpretado como la probabilidad de que nuestro modelo detecte un caso positivo.

- **Ratio de falsos negativos** (o **FNR** *False Negative Rate*): es el concepto complementario a *recall*:
$$FNR = \frac{FN}{TP + FN} = \frac{FN}{P} = Miss\ Rate = 1 - Recall$$
Puede ser interpretado como la probabilidad de que el modelo falle al detectar casos positivos. Por ello también se lo llama *Miss Rate*.

- **Especificidad** (***Specificity*** o **TNR** *True Negative Rate*): la proporción de ejemplo negativos detectados como tales.
$$Specificity = \frac{TN}{FP + TN} = \frac{TN}{N}$$
En términos de probabilidad puede ser interpretado como la probabilidad de que nuestro modelo detecte un caso negativo.

- **Ratio de falsos positivos** (o **FPR** *False Positive Rate*): es el concepto complementario a *especificidad*:
$$FPR = \frac{FP}{FP + TN} = \frac{FP}{N} = False\ Alerts = 1 - Specificity$$
Puede ser interpretado como la probabilidad de que el modelo falle al detectar casos negativos. Por ello también se lo llama *Falsas Detecciones* o *Falsas Alertas*.

- **Precisión** (*precision* en inglés): es la proporción de detecciones (predicciones positivas) que son correctas.
$$\text{Precisión} = \frac{TP}{TP + FP} = \frac{TP}{\text{Predicción Positiva}}$$
Puede interpretarse como qué tan probable es que una alerta por clase positiva esté bien, por eso también se lo denomina **poder predictivo positivo**.

3.1.8.4. Métricas en el problema de Verificación Automática de Firmas Manuscritas

En aplicaciones reales es importante conocer las implicancias prácticas de las métricas y definir objetivos razonables (no se suele desarrollar sin un objetivo ya que no se sabría cuándo considerar que el trabajo es suficientemente bueno). Los objetivos deben proveer garantías prácticas a los usuarios y dependerán del uso de la aplicación. Por ello, siempre hay que tener presente para qué se usará el modelo que pretendemos crear a la hora de elegir la métrica y qué significado nos provee dicha métrica.

En la bibliografía de *"Verificación Automática de Firmas Manuscritas"* [Parodi 2014] se trabaja con las métricas:

- **Tasa de Falsos Rechazos (FRR)**: porcentaje o ratio de rechazos de firmas auténticas.
- **Tasa de Falsa Aceptación (FAR)**: porcentaje o ratio de aceptaciones de falsificaciones.

El punto donde son iguales se suele llamar Tasa de error de intersección o ***Equal Error Rate* (EER)**. De tratarse de un dataset con clases balanceadas será igual a la tasa de aciertos (ver figura 3.9).

No es lo mismo rechazar una firma auténtica que aceptar una falsificación. Por ejemplo en una aplicación de detección automática de fraude se querrá priorizar no aceptar falsificaciones a costa de a veces arrojar falsos rechazos.

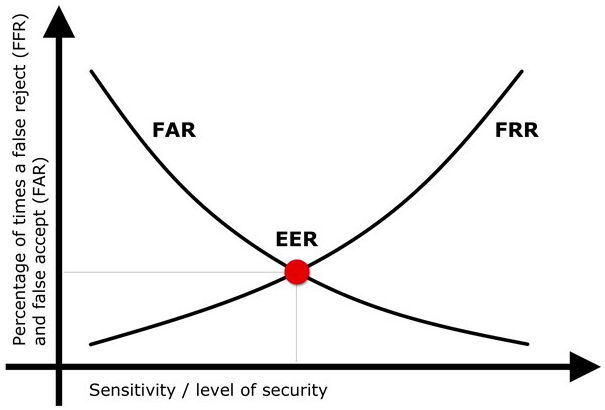


Figura 3.9: La tasa de error de intersección es la elección del umbral de sensibilidad que representa un balance equitativo entre falsos positivos y falsos negativos. Créditos de la imagen: *"FAR and FRR: security level versus user convenience"*

Como [Parodi 2014] toma como clase positiva las firmas auténticas (que son de quién declaran ser) resulta:

$$FRR = \frac{FN}{P} = 1 - Recall$$

$$FAR = \frac{FP}{N} = FPR = Falsas\ Alertas$$

En cambio, en nuestro problema particular tendremos pares de firmas y querremos saber si fueron hechas o no por la misma persona para detectar errores en la confección de los telegramas. En función de esto el sistema al cual se integrará el modelo levantará una alerta de posible error al usuario. Es por ello que, como se dijo en la sección anterior, trataremos como clase positiva a los pares de firmas *no coincidentes*. Bajo este enfoque las métricas nombradas para el problema de verificación automática de firmas se relacionan con las métricas del problema de clasificación binaria definido por nosotros de la siguiente manera:

$$FRR = Falsas\ Alertas = 1 - Specificity$$

$$FAR = Miss\ Rate = 1 - Recall.$$

Esto es consistente con que en nuestro sistema los rechazos (no coincidencias) generan alertas y las aceptaciones (coincidencias) dan lugar a la aprobación del telegrama.

3.1.8.5. Análisis costo-beneficio para las métricas en clasificación binaria

A menudo los métodos de aprendizaje automatizado para clasificación binaria producirán un puntaje o score entre $[0, 1]$ para cada instancia a clasificar. Este puntaje suele ser interpretado como la certeza del modelo o probabilidad de la instancia de pertenecer a la clase positiva. Luego el modelo aplica un umbral elegido $t \in [0, 1]$ para determinar la predicción:

$$model(x) = \begin{cases} Prediccion\ Positiva\ si\ score(x) > t \\ Prediccion\ Negativa\ si\ score(x) \leq t \end{cases}$$

Valores altos de t serán más conservadores a la hora de *detectar*, y sólo clasificarán como positivos ejemplos para los cuáles estén muy seguros que lo son. Es por ello que al incrementar el umbral aumentaremos el *precision*. Como contraparte al aumentar t habrá potencialmente más ejemplos positivos mal clasificados (falsos negativos), o sea disminuirá el *recall*.

Esta relación costo-beneficio entre *precision* y *recall* al variar el umbral de clasificación, también se evidencia entre *recall* y *False Alerts*: disminuyendo el umbral podemos hacer más propenso al modelo

a clasificar una instancia como positiva aumentando la cantidad de positivos correctamente clasificados pero potencialmente aumentarán también las falsas detecciones.

Las relaciones de compromisos especificadas en párrafos anteriores admiten ser analizadas gráficamente: variando el valor del umbral t se pueden obtener diferentes relaciones de compromiso entre las dos variables y obtener una gráfica 2D de una en función de la otra. Así se definen las curvas:

- **Curva ROC** (por su nombre en inglés *Receiver Operating Characteristic*) [Fawcett 2006]: graficando los valores de *Recall* y *False Alerts*. Se muestra un esquema de qué formas tienen las curvas en en la figura 3.10.
- **Curva DET** (por su nombre en inglés *Detection Error Tradeoff*) [Martin et al. 1997]: graficando los valores de *miss rate* y *falsas alarmas* y generalmente utilizando escala normal o logarítmica sobre ambos ejes. La información contenida es análoga a la de las curvas ROC debido a que el *miss rate* es el concepto complementario a *recall* pero la escala aplicada a los ejes hace que sean curvas más lineales y fáciles de analizar visualmente.
- **Curva Precision-Recall** [Akbani et al. 2004]: graficando los valores de *precision* en función del *recall*. Se muestra un ejemplo en la figura 3.11. Como se explicará más adelante es muy popular analizar esta relación de compromiso especialmente cuando se trabaja con conjuntos de datos desbalanceados.

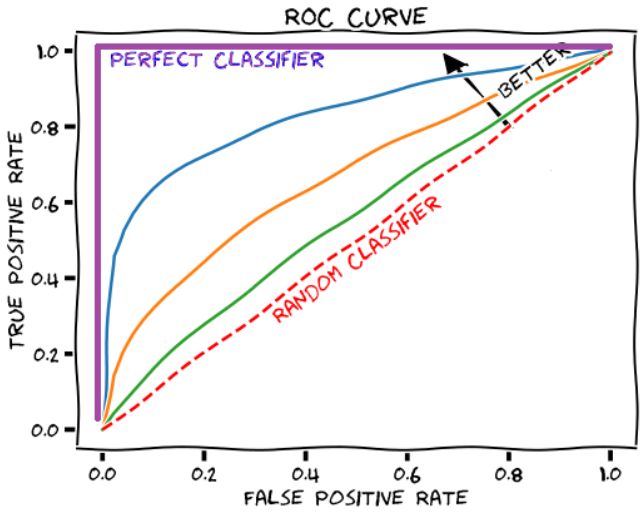


Figura 3.10: Figura esquemática donde ser observa como varían las curvas ROC de los modelos comparándolas con los casos extremos: el modelo de predicción aleatoria (peor modelo para la clasificación binaria) y el modelo ideal que siempre acierta. Créditos: The Machine Learners `www.themachinelearners.com/curva-roc-vs-prec-recall`

La elección de métricas adecuadas es todavía más compleja en **problemas desbalanceados**, es decir, en problemas donde una de las dos clases es mucho más probable que la otra. Por ejemplo, en diagnóstico médico o detección de fraude, a menudo existen en el conjunto de datos un número pequeño de ejemplos clasificados como positivos. Esto puede acarrear que un modelo que se entrene para maximizar la *tasa de aciertos* degenere en predecir siempre la clase más abundante (la negativa en los casos mencionados). No obstante, el *miss rate* es crítico en los ejemplos nombrados: ¡significará fraude no detectado o patologías existentes no diagnosticadas!.

Existen técnicas para corregir manualmente el *imbalance* de los datos como eliminar aleatoriamente parte de la clase mayoritaria (***undersampling***) [Japkowicz 2000] o repetir instancias de clase minoritaria (***oversampling***) [He and Garcia 2009] y técncias para que el clasificador pondere más una clase que

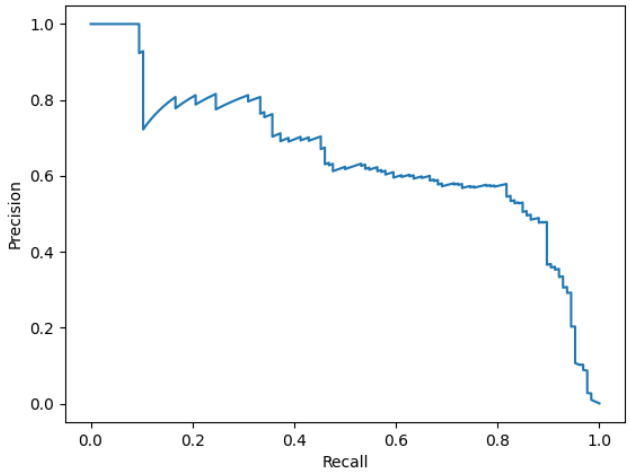


Figura 3.11: Ejemplo de una curva de *precision-recall*. Cada punto de la curva corresponde a la *precision* y el *recall* obtenidos para un cierto valor de umbral de clasificación t.

otra [Akbani et al. 2004]. Es de suma importancia tener en cosnideración que utilizar dichos métodos *en la evaluación del modelo* muchas veces constituye un **error metodológico grave**: no se puede modificar el conjunto de datos de evaluación, ya que el mismo deberá poseer la misma distribución del hipotético conjunto de datos futuro sobre el que se aplicará el modelo (o sea, los datos que aparecerán en la aplicación final). Por ende deberán también poseer el mismo grado de *imbalance*. Es por eso que las mencionadas técnicas suelen aplicarse sólo durante la fase de entrenamiento.

En éstos casos donde los datos son desbalanceados *por naturaleza* lo que debemos adaptar es la estrategia de evaluación eligiendo métricas robustas al desbalance. Analizar *precision* y *recall* suele ser una buena opción ya que evaluaremos detectar una cierta proporción de la clase positiva pese a que sea pequeña (*recall*) y tener cierta confianza en que las predicciones positivas son correctas (*precision*). Para conjugar estas dos métricas y obtener una medida de perforcmance numérica es común calcular el área debajo de la curva *Precision-Recall*, denominada **R-AUPRC**. Esta fórmula considera los valores de *precision* y *recall* para todos los valores de umbrales posibles.

En la práctica generalmente se considera al umbral como parte del modelo final ya que son las predicciones las que se utilizarán para tomar decisiones. Una estrategia que tiene en cuenta ambas métricas a la hora de elegir el umbral es pensar un umbral de *recall* o *precision* aceptado y maximizar la otra variable.

En el caso de nuestros errores en telegramas electorales, una alerta de *no coincidencia* (clase positiva) previene que se soliciten retransmisiones por telegramas mal confeccionados desde el centro de cómputo, lo cuál genera trabajo humano innecesario. Como estaremos satisfechos en reducir en $\times 5$ las retransmisiones, nos contentaremos con un *recall* del 0,8. A su vez, si el ratio de falsas alertas es alto, el usuario desestimará las alarmas, por lo que queremos que la *precision* sea de al menos 0,95 (de modo que a lo sumo 1 de cada 20 alarmas sean *falsas alarmas*).

Puede ser interesante disponer de una forma de comparar dos modelos de predicción (habiendo fijado el umbral para cada uno). Para ello se suele combinar *recall* y *precision* en una única métrica utilizando el ***F-score*** (o ***F-measure***), que es la media armónica de ambas:

$$F\text{-}score = \frac{2}{recall^{-1} + precision^{-1}} = 2\frac{precision.recall}{precision + recall}$$

A veces los conjuntos de datos son generados y poseen desbalance. Ésto es algo que ocurre en los problemas de similitud que se discutirán más adelante en la tesina: generalmente poseemos varios ejemplos que pertenecen a diferentes clases (firmas de un mismo autor, caras de una persona, etc) y

construimos nuestros conjuntos formando pares que constituyan *coincidencias* y *no coincidencias*. De este modo podemos correr el riesgo de generar conjuntos desbalanceados (por ejemplo si se toman todos los pares de elementos). En [Parodi 2014] se nombra el problema de desbalanceo en los conjuntos de datos (generados como parte del mismo trabajo) como limitante para obtener mejores métricas de error sobre la clase no predominante (sección 3.9 sobre trabajos futuros). Un muestreo incorrectamente imbalanceado en la evaluación (o sea que no sigue la distribución real de los datos a los cuáles el modelo pretende generalizar) puede constituir un error metodológico grave.

3.1.9. Importancia de la Metodología de Evaluación

La metodología de evaluación es la que define como evaluar una solución al problema. En Aprendizaje Automatizado se compone fundamentalmente de la confección del conjunto de test y de la elección de la medida de performance.

No es poco frecuente encontrar trabajos con errores de conjuntos de test mal confeccionados (por ejemplo con correlaciones no generalizables al conjunto total de datos), con errores en la elección de métricas (como usar métricas tales como tasa de aciertos para problemas desbalanceados), o incluso sin conjunto de test (sólo reportan datos de validación). Éstos errores invalidan totalmente los resultados del trabajo.

La solución a un problema y la evaluación al mismo tienen intereses contrapuestos: uno puede llegar a resultados sorprendentes -pero inválidos- con una solución si se la evalúa erróneamente. Es por ello que para garantizar que estamos haciendo un buen trabajo lo mejor es que el diseño de la metodología de evaluación del problema y las soluciones, sean responsabilidades y trabajos distintos e independientes.

Esto último sucede en el ámbito académico donde hay publicaciones que *presentan* un conjunto de datos con ciertas métricas (y muchas veces un *baseline*). Y luego existe un gran número de trabajos sucesivos que presentan soluciones y resultados sobre dicho problema.

También sucede en competencias de modelos donde el quién diseña el problema (incluyendo los conjuntos de datos y la manera de evaluar) y quién propone las soluciones son diferentes personas: los jueces y participantes. En las mismas, además, el puntaje sobre el conjunto de test suele ser oculto hasta terminada la competencia, evitando que se use la información de *feedback* para ajustar hiperparámetros. Esto presta una mayor garantía de confianza.

Cuando analizamos un trabajo que presenta una solución para un problema que él mismo define, hay que someterlo a un estricto escrutiño de que la metodología de evaluación sea correcta y haya sido correctamente implementada.

La metodología de evaluación es crítica en la práctica a la hora de desplegar soluciones que se basen en aprendizaje automático, incluso si se decide no desarrollar soluciones propias sino adquirirlas de terceros. Por ejemplo en organizaciones que van a subcontratar el proceso de desarrollo de modelos de Aprendizaje Automático, un conjunto de test correctamente construído y oculto al contratista (y preferentemente con un *baseline* conocido) da la pauta de qué tan bueno es el trabajo que se estará adquiriendo.

3.1.10. Reducción de la dimensionalidad

En ocasiones resulta útil reducir la dimensionalidad de los datos para, por ejemplo, poder analizarlos visualmente utilizando 2 o 3 dimensiones que sean graficables. Un método muy utilizado es el de PCA que elije las direcciones linealmente decorrelacionadas de mayor variabilidad de los datos.

El análisis de componentes principales (**PCA** por su nombre en inglés ***Principal Component Analysis***) es un método popular de extracción de atributos no supervisado [Han et al. 2012].

Podemos interpretar intuitivamente las componentes principales como las direcciones donde hay mayor variabilidad de los datos. Una vez encontradas dichas direcciones, por orden de importancia según mayor variabilidad de los datos, se puede proyectar los ejemplos, mediante un cambio de coordenadas, y quedarnos con las proyecciones respectivas a las k direcciones más importantes, y así reducir la cantidad p de atributos ($k < p$).

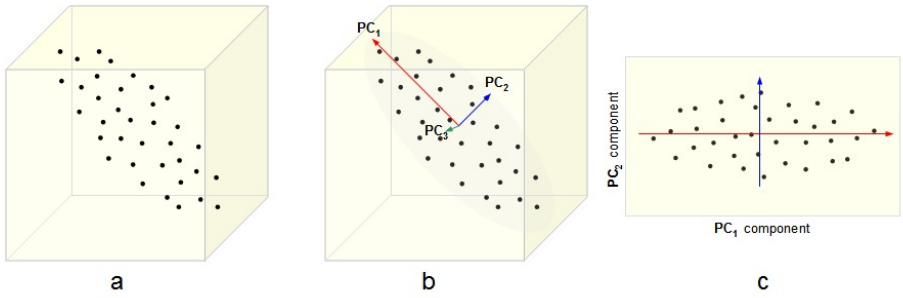


Figura 3.12: En (a) se muestra un conjunto de datos en tres dimensiones. En (b) se identifican las tres componentes principales (direcciones linealmente decorrelacionadas con mayor variabilidad de datos). En (c) se alinean los datos con las dos direcciones de mayor variabilidad (cambio de base), y se elimina la componente con menor variabilidad. Como consecuencia, la dimensionalidad de los datos se redujo en 1 y pueden ser graficados en el plano. Fuente: `cnx.org`

PCA se calcula en cada paso buscando el versor cuya dirección sea ortogonal a las direcciones de los versores ya calculados, y que maximice la varianza de las proyecciones de los datos sobre dicha dirección. Por construcción, el conjunto de versores generados constituye una base ortonormal.

Luego de calculadas k componentes principales se pueden proyectar los datos sobre dichas direcciones (cambio de base), ignorándose el resto de las componentes. La dimensionalidad se reduce a k, capturando -a veces también se dirá "explicando"- la mayor variabilidad de datos que es posible capturar -o "explicar"- con cambios de base.

También existen métodos de reducción de dimensionalidad que aplican transformaciones no lineales y a veces (dependiendo el conjunto de datos) pueden arrojar mejores resultados. Aún así PCA suele ser suficiente y es adoptado por su sencillez.

3.2. Clasificadores

En la presente sección se describen brevemente algunos clasificadores utilizados en la tesina para encontrar un primer punto de partida (*baseline*) contra el cuál comparar el desarrollo posterior.

Support Vector Machine (SVM) es utilizado en [Parodi 2014], sobre el cuál nuestro trabajo se ha basado fuertemente. También hemos experimentado utilizando ***Random Forest*** (RF) sobre las características extraídas en el mencionado trabajo.

Redes Neuronales Artificiales (a pesar de ser un clasificador) será desarrollado en la sección siguiente ya que es el método principal utilizado por nuestro trabajo.

3.2.1. Random Forest

3.2.1.1. Árboles de Decisión

Los árboles de decisión son un método simple que consiste en particionar recursivamente mediante hiperplanos el espacio de atributos de modo que quede finalmente partido en hiper-rectángulos [Hastie et al. 2001]. A cada región se le asigna un valor de predicción constante. A la hora de realizar una inferencia sobre un ejemplo x primero se ubicará la región a la que pertenece. Esto puede hacerse siguiendo las reglas de particionado del árbol de decisión binaria que generó las regiones. Finalmente, la predicción será la predicción constante de dicha región. En general el valor constante óptimo para cada región (ajustado sobre el conjunto de datos de entrenamiento) es tomar la media de la variable dependiente $\bar{y}$ en problemas de regresión que usan MSE como métrica, o tomar la etiqueta más frecuente (moda) de las y_i en problemas de clasificación que usan *Error Rate* como métrica. Ver figura 3.13.

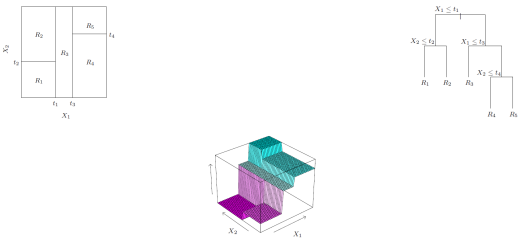


Figura 3.13: A la izquierda se muestra el espacio de atributos partido en regiones. En el centro se muestra el particionado recursivo (arbol de decisión binaria) que dio como resultado dichas regiones. A la derecha se muestra la función de predicción del modelo que asigna para cada región valores constantes.

3.2.1.2. *Classification And Regression Tree (CART)*

El algoritmo de CART es uno de los métodos más populares para construir árboles de decisión. Otras opciones son los métodos ID3 [Quinlan 1986], C4.5 [Quinlan 1993] y C5.0 [Kuhn and Johnson 2013].

Para explicar el algoritmo supongamos que tenemos N ejemplos de entrenamiento:

$$\{(x_i, y_i), 1 \leq i \leq N\}$$

A su vez consideremos que la variable independiente posee p atributos:

$$x_i = (x_{i,1}, x_{i,2}, ...x_{i,p}), 1 \leq i \leq N$$

Una partición sobre el atributo j y valor de división s define las regiones R_1 y R_2 según:

$$R_1(j, s) = \{x | x_{i,j} \leq s, 1 \leq i \leq N\}$$

$$R_2(j, s) = \{x | x_{i,j} > s, 1 \leq i \leq N\}$$

A su vez las predicciones para dichas regiones serán c_1 y c_2 calculadas como ya se explicó.

Para encontrar la mejor partición binaria se optimiza:

$$min_{j,s}[\sum_{x_i \in R_1(j,s)} criterion(y_i, c_1) + \sum_{x_i \in R_2(j,s)} criterion(y_i, c_2)]$$

Los criterios más usados son MSE para problemas de regresión o *Gini Impurity* o *Information Gain* [Raileanu and Stoffel 2004] para problemas de clasificación.

Para cada uno de los p atributos se puede ordenar en los ejemplos según su j-ésima feature con una complejidad de $\mathcal{O}(N\ log\ N)$ y luego encontrar el mejor valor de división s con una complejidad $\mathcal{O}(N)$. Haciendo esto para cada atributo j se puede encontrar la mejor elección de s sumando una complejidad total de: $\mathcal{O}(pN\ log\ N)$) en cada paso de subdivisión nueva del árbol. Incluso se puede ordenar una vez al principio y mantener p listas (una para cada atributo) de índices que apunten a los x_i ordenada según dicho j-ésimo atributo, ahorrando el factor $logN$ en cada paso.

El algoritmo CART encuentra la mejor partición binaria, se particionan los datos en las regiones resultantes y se repite recursivamente el proceso de división. La decisión de partir en la mejor partición en cada paso es una decisión *greedy*. Optimiza la mejor decisión local, o sea en un único paso de subdivisión, aunque puede no ser la mejor decisión global, o dicho de manera más coloquial "al largo plazo". De todas formas llega a muy buenos resultados de árboles de decisión.

Ventajas de los Árboles de Decisión:

- Poseen bajo sesgo, pués no hacen demasiadas asumpciones en la función objetivo.
- Son simples, no poseen demasiados hiperparámetros a ajustar.

- Son robustos al escalado de los atributos. No requieren normalizar o estandarizar los datos. Son insensibles a estos preprocesamientos puesto que sólo importa el orden de los características, no su escala.

Desventajas de los Árboles de Decisión:

- Los árboles profundos poseen alta varianza, por lo cuál, de no configurar bien los hiperparámetros que controlan su complejidad (cantidad mínima de nodos, profundidad máxima o aplicar técncias de podas) son propensos a sobreajustar.

- Al predecir valores constante para cada región son incapaces de descubrir tendencias que permitan extrapolar las predicciones a rangos de atributos que no estén dentro del rango de los atributos presente en el conjunto de entrenamiento; potencialmente arrojando resultados erróneos para ellos. Ver figura 3.14.

  A diferencia de lo que muchos creen, no es un problema que tengan sólo en problemas de regresión sino que se pueden pensar casos en donde afecten a problemas de clasificación (por ejemplo distinguir la clase de los (x_1, x_2) donde $|x_1 - x_2| \leq 1$). Y es que la escencia del problema radica en que las regiones con valores constantes no predicen bien ejemplos que poseean atributos "más allá" de los rangos de los atributos presentes en el entrenamiento.

3.2.1.3. *Random Forest*

Para mitigar el problema de la alta varianza de los árboles de decisión se puede recurrir a ***decision tree bagging***. En este método se entrena un número t de árboles de decisión, cada uno basado en diferentes subconjuntos de m ejemplos tomados del conjunto de datos de entrenamiento y luego (para la inferencia) se combinanan los resultados de las predicciones, por ejemplo, tomando la media o moda. Cuándo se combinan modelos se suele hablar de ***ensamble***.

Esta técnica es denominada ***Bagging*** y puede aplicarse sobre cualquier predictor [Breiman 1996]. Es un caso particular de la técnica denominada ***Bootstrap***, utilizada para encontrar buenos estimadores estadísticos sobre una población [Hastie et al. 2001]. Tiene la capacidad de reducir dramáticamente la varianza de predictores inestables como árboles de decisión sin introducir sesgo, lo cual conduce a mejores resultados. Una propiedad importante de los *ensambles* es que al incrementar el número t de estimadores base utilizados, el valor de la predicción no puede empeorar. Como contrapartida, la construcción será más lenta (un factor lineal respecto de t).

Random Forest es una modificación de un ***decision tree bagging*** donde además de la técnica de *bagging* se agrega que, cuando se crean los árboles de decisión, en cada nivel a subdividir se consideran aleatoriamente sólo k de los p atributos como candidatos. Ésto introduce ruido en el proceso que ayuda a que los t árboles generados no estén tan correlacionados, reduciendo aún más la varianza del *ensamble*. [Breiman 2001]. Valores pequeños de k reducirán la correlación entre cada par de árboles del ensamble, y por lo tanto, reducirán la varianza en promedio. Generalmente se toma $k = \sqrt{p}$.

OOB (*out of bag*): para cada ejemplo x_i del conjunto de datos, es posible construir una predición de *Random Forest* promediando sólo las predicciones de aquellos árboles que no incluyeron a x_i en el conjunto de entrenamiento. Esto es lo que se conoce como ***OOB score***. El *OOB score* constituye una métricas de validación correcta y es calculada haciendo uso de elementos incluídos en el conjunto de entrenamiento, lo que implica que no necesitamos reservar datos para la validación. ¡Podemos entranar con todos los datos!.

Es importante notar que si el *Random Forest* posee t predictores, durante el cálculo del *OOB score* se utiliza para cada ejemplo una cantidad menor de predictores (ya que se obvian para cada ejemplo los que lo utilizaron para entrenar). Es por ello que el *OOB score* será una métrica *pesimista* (se pueden esperar mejores resultados sobre un conjunto de test). También es importante aclarar que al igual que el método de *Cross-Validation*, *OOB Score* se basa en la asumpción de que un particionado aleatorio del conjunto de datos constituye una buen conjunto de validación. De no ser cierto esto observaremos un considerablemente mejor *OOB score* que los resultados observados sobre un conjunto de test bien

confeccionado.

Ventajas de *Random Forest*: tienen la simplicidad de los árboles de decisión (pocos hiperparámetros, insensible a la escala de atributos) y respecto a los mismos, resuelve el problema de sobreajuste causado por la alta varianza.

Desventaja de *Random Forest*: no solucionan el problema de extrapolación que poseen los árboles de decisión. Ver figura 3.14.

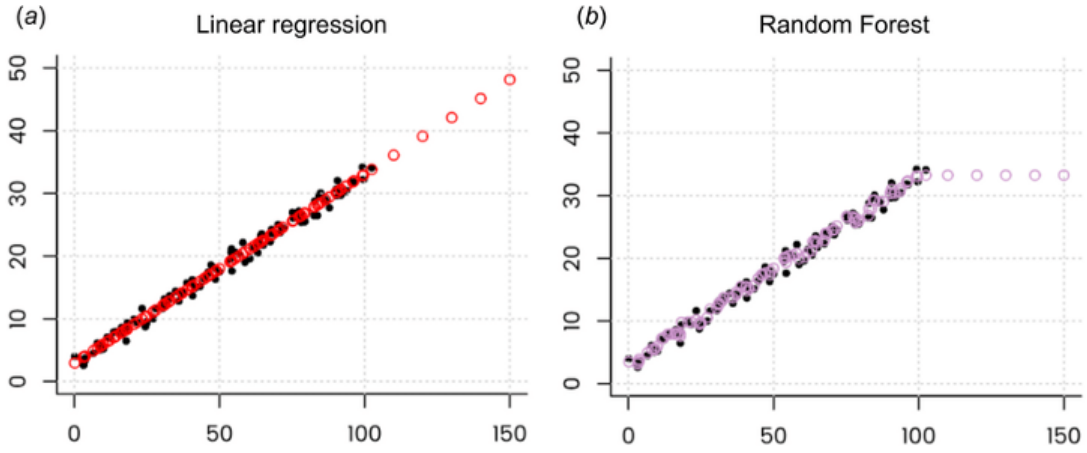


Figura 3.14: Ejemplo del problema de extrapolación que poseen los árboles de decisión y hereda el método de Random Forest. Créditos: Derrick Mwiti `https://neptune.ai/blog/random-forest-regression-when-does-it-fail-and-why`

3.2.2. Support Vector Machine

Las Máquinas de Vectores Soporte son un método de clasificación que está sustentada por un desarrollo matemático teórico de envergadura ya que se basa en aprendizaje estadístico o teoría VC (Vapnik-Chervonenkis) [Vapnik and Chervonenkis 1971], [Vapnik and Chervonenkis 1974].

Procederemos a explicar intuitivamente y de manera sencilla las ideas claves. Atiunque haremos referencias a las formulaciones matemáticas, no profundizaremos en ellas. El lector puede ampliar sobre el desarrollo del método leyendo [Cortes and Vapnik 1995] [Boser et al. 1992] o ahondar en el desarrollo matemático en [Hastie et al. 2001].

1. Una tarea de clasificacióin puede ser vista como la tarea de separar los espacios geométricos que ocupa cada clase dentro del espacio de atributos. Ver figura 3.15.
2. Podemos empezar tratando de utilizar hiperplanos de la forma $wx + b = 0$ para separar las clases. De todos los hiperplanos a elegir en nuestra labor de optimización encontraremos el que maximice el margen entre ambas clases: la mínima distancia entre un punto del conjunto de entrenamiento y el hiperplano separador. Ver figura 3.16.
3. La contra es que puede no existir un hiperplano que los separe, por ejemplo, si las clases se encuentran "demasiado superpuestas" en el especio de atributos. Ver figura 3.17.
4. Podemos debilitar el problema de separar las clases utilizando márgenes "relajados" y permitiendo a algunos puntos de entrenamiento estar dentro del lado equivocado del hiperplano. Ver figura 3.18.

   En la formulación matemática minimizaremos no sólo la distancia mínima al hiperplano separador sino también un término que pondere *"que tanto nos permitimos relajar los márgenes"*. De aquí sale el hiperparámetro C llamado *coste* o *regularización*.

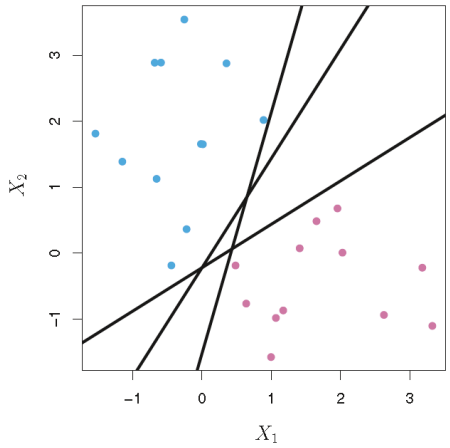


Figura 3.15: Ejemplo de clasificación binaria particionando el espacio geométrico de atributos por medio de separadores lineales. Créditos: Drew Wilimitis, `towardsdatascience.com`

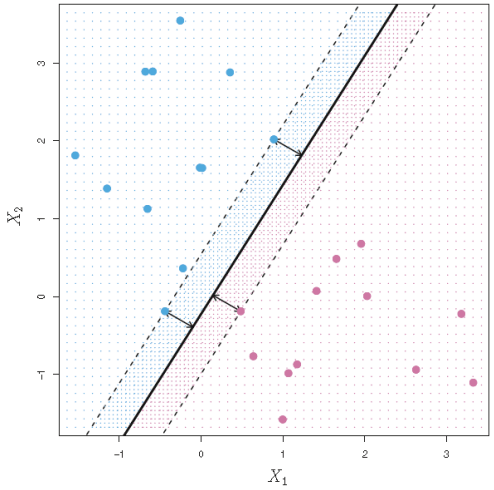


Figura 3.16: Ejemplo de hiperplano calculado por SVM. Los puntos que están a menor distancia del hiperplano separador se denominan vectores soporte. Créditos: Drew Wilimitis, `towardsdatascience.com`

5. Seguimos teniendo el limitante de que las fronteras son sólo lineales: puede no ser lo ideal para muchas distribuciones de datos. Es por eso que se introduce una función ϕ que mapee los datos de la variable independiente x a un espacio de mayor dimensionalidad donde potencialmente las clases serán separables por una frontera lineal. Así, trataremos de usar la formulación de hiperplano ya desarrollada hasta el punto anterior no para los datos $\{(x_i, y_i)\}$ sino para separar $\{(\phi(x_i), y_i)\}$. Ver figura 3.19.

6. El punto anterior no tiene sentido sin lo que se denomina **truco de kernel**: trabajando algebraicamente la fórmula a optimizar se llega a una expresión tal que nunca es necesario para la optimización calcular explícitamente el mapeo $\phi(x)$ sobre los ejemplos, sólo es necesario saber como calcular productos internos sobre el resultado de dichos mapeos (en el espacio de mayor dimensionalidad).

   O sea simplemente necesitamos saber como calcular:

   $$K(x_i, x_j) =< \phi(x_i), \phi(x_j) >$$

   Donde $<,>$ es la operación de producto interno. Dicha función K se denomina **función kernel**.

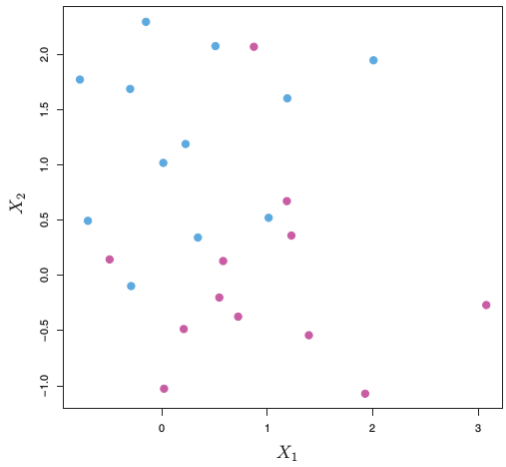


Figura 3.17: En este ejemplo de clases celestes y violetas, no pueden ser separadas por un hiperplano. Créditos: Drew Wilimitis, `towardsdatascience.com`

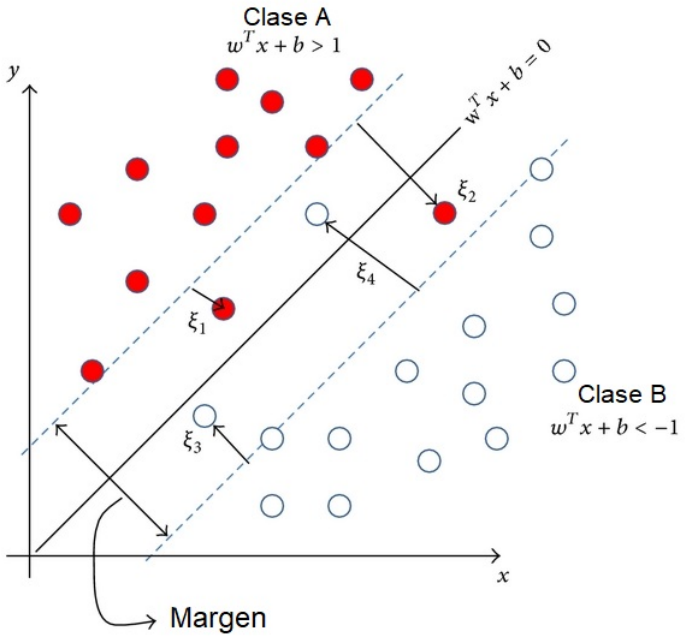


Figura 3.18: Los valores notados como ϵ_i constituyen una medida de margen relajado para cada ejemplo y se denominan *variables slack*. El hiperparámetro C permite regular cuán permisivo es el modelo a la hora de permitir errores de clasificación. Imagen tomada de [Fernandez-Lozano et al.].

Se desarrollan varias de estas funciones en la bilbiografía citada que poseen un sustento teórico relevante. Su elección y la de sus parámetros constituyen hiperparámetros de las SVM.

3.3. Redes Neuronales Artificiales

Las **Redes Neuronales Artificiales** (*RNA* por sus siglas en inglés) son un método de aprendizaje supervisado que deben su nombre a su raíz histórica: surgen a partir de diferentes trabajos que trataron de imitar el funcionamiento del cerebro humano. Sin embargo, la investigación moderna está guiada por

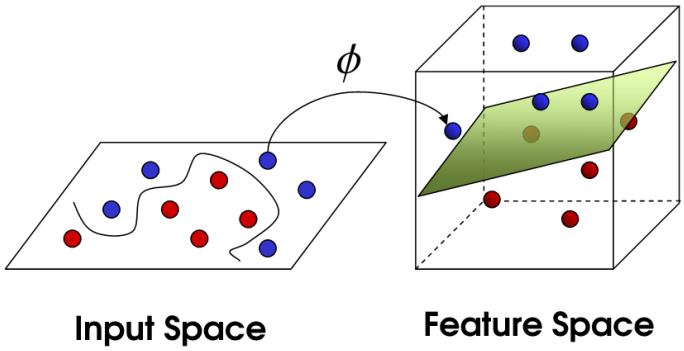


Figura 3.19: Luego de mapear los puntos por medio de una función no lineal a un espacio de mayor dimensionalidad pueden ser separados por un hiperplano. Remitiéndonos al espacio de atributos original el hiperplano constituye una superficie curva que particiona el espacio. Créditos: Drew Wilimitis, `towardsdatascience.com`

disciplinas matemáticas y de ingeniería, y el objetivo de las RNA no es modelar perfectamente el cerebro.

3.3.1. Un poco de historia

En 1943 el lógico Walter Pitts propone un modelo simplificado de la neurona cerebral representado por adiciones y la aplicación de un umbral. Intentaba imitar y analizar el comportamiento del cerebro humano en su trabajo *"A Logical Calculus of the Ideas Immanent in Nervous Activity"* [McCulloch and Pitts 1943]. Él creía que la neurona se podía modelar como adición de sus entradas y aplicación de una función de umbral para producir las respuestas debido a que la actividad neuronal poseía lo que describía como *"caracter de activación de todo o nada"* (ver figura 3.20).

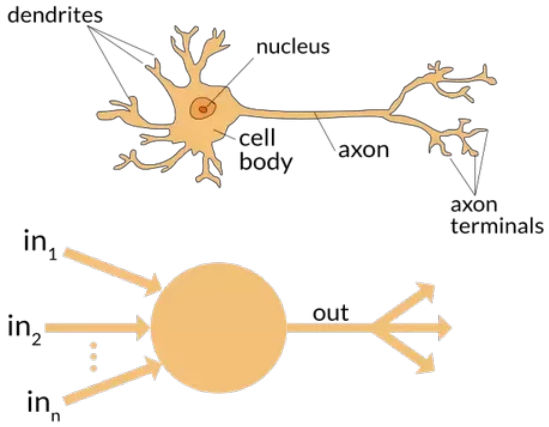


Figura 3.20: Imágen esquemática del modelo de neurona propuesto por [McCulloch and Pitts 1943]. Fuente: [Howard and Gugger 2020].

En 1958 el psicólogo Frank Rosenblatt desarrolla la neurona artificial (Mark I Perceptrón) con habilidad para aprender en *"The Design of an Intelligent Automaton"* [Rosenblatt 1956-1958]. El mismo poseía la siguiente formulación (ver figura 3.21) [Mitchell 1997]:

$$o(x_1, ..., x_n) = \begin{cases} 1 \; si \; w_0 + w_1 x_1 + w_2 x_2 + ... + w_n x_n > 0 \\ -1 \; en \; caso \; contrario \end{cases}$$

Los parámetros w_i se inicializarían de manera aleatoria y actualizarían iterativamente comparando la salida o del perceptrón con la salida esperadas y, siguiendo la siguiente fórmula:

$$w_i \leftarrow w_i + \Delta w_i$$

$$\Delta w_i = \eta(y - o)x_i$$

Donde x_i es el atributo respectivo al peso que se está actualizando y η una constante que denominada *learning rate*. En [Minsky and Papert 1969] se ha demostrado que de ser η suficientemente pequeña, en un problema binario linealmente separable el algoritmo converge a la solución óptima. El procedimiento es análogo al método de descenso por gradiente que se verá en las próximas secciones y en el cuál se basa la optimización de las actuales redes neuronales.

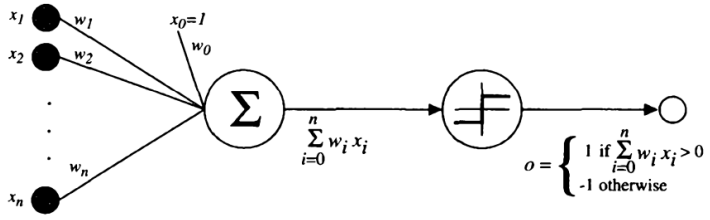


Figura 3.21: Representación gráfica de un perceptrón. Fuente [Mitchell 1997].

El preceptrón de Rosenbalt cayó en el olvido luego de que una publicación del profesor de MIT (colega de Rosenbaltt) Marvin Minsky demostrara que no podía representar simples funciones como XOR. Este perceptrón puede verse como un hiperplano partiendo el espacio de atributos de manera que produce una salida igual a 1 para instancias que se encuentran a un lado del hiperplano, y -1 al otro. Por eso la efectividad está limitada a conjuntos de datos linealmente separables.

Otro antecedente histórico de imitar la estructura de comportamiento del cerebro se haya en *"Parallel Distributed Processing (PDP)"* [Rumelhart et al. 1986d] donde se describe que de existir un sistema tal, poseería las siguientes características (todas características que poseen las redes neuronales actualmente):

1. Un conjunto de unidades de procesamiento.
2. Un estado de activación de las mismas.
3. Una función de salida de cada unidad de procesamiento.
4. Un patrón de conectividad de dichas unidades.
5. Una regla de propagación de las activaciones a través de la red de conexiones (forma de transformar las salidas de una unidad en entradas de la otra).
6. Una regla de activación para combinar las entradas de las unidades, el estado actual y producir las salidas de la unidad.
7. Una regla de aprendizaje donde se modifican los patrones de conectividad por medio de la experiencia.
8. Un entorno en donde el sistema opera.

Durante la década del 80' las redes neuronales vuelven a cobrar cierta tracción en el mundo científico a raíz de que se demostrara que agregar una segunda capa de neuronas evitaría el problema identificado por Minsky [Cybenko 1989]. Véase figura 3.22

En 1986 se publicaron los reconocidos trabajos sobre el método de *backpropagation* donde se establece el algoritmo de optimización que hasta el día de hoy es utilizado (con algunas variaciones y ajustes) para entrenar las redes neuronales de muchas capas [Rumelhart et al. 1986c] y [Rumelhart et al. 1986b].

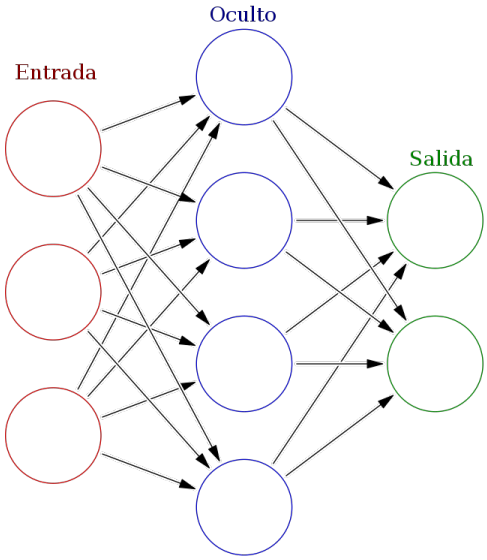


Figura 3.22: Esquema de red de una capa oculta. Las entradas se combinan linealmente y se le aplica una función no lineal para obtener las activaciones ocultas (valores intermedios) que luego se vuelven a combinar linealmene y aplicar una función no lineal para obtener las salidas o predicciones. Créditos de la imágen: `es.wikipedia.org/wiki/Red_neuronal_artificial`

No obstante actualmente se sabe que esta técnica fue varias veces re-descubierta e incluso existe un antecedente que data de 1960 [Rumelhart et al. 1986a].

Si bien teóricamente cualquier función continua puede ser aproximada por redes con una capa oculta con un número suficiente de neuronas se falló en entender la naturaleza fundamentalmente empírica del aprendizaje automatizado y poco se exploró aproximadamente hasta 20 años después sobre el potencial de agregar más capas. A fines de los 90' se publica el famoso trabajo [LeCun et al. 1998] que mejora sustancialmente la performance utilizando un modelo relativamente sencillo (LeNet-1) y sin extracción manual de caraceterísticas. Fue este el nacimiento de las redes convolucionales que revolucionaron el mundo de la visión computarizada.

3.3.2. Formulación actual

La formulación actual de las Redes Neuronales Artificiales no dista demasiado del modelo simplista de red de una capa oculta, descripto en la subsección anteior. Ver figura 3.22.

Supongamos que la función a aproximar es:

$$\hat{f} : \mathbb{R}^{n_{in}} \rightarrow \mathbb{R}^{n_{out}}$$

O sea que tenemos n_{in} atributos de entrada y n_{out} de salida.

Una Red Neuronal Artificial consiste escencialmente en transformar el vector de entradas $\{x_i\}_{i=1}^{n_{in}}$ aplicando sucesivamente funciones no lineales f^j denominadas **capas**. Estas funciones tienen la forma:

$$f^j : \mathbb{R}^{w_{j-1}} \rightarrow \mathbb{R}^{w_j}$$

O sea, son simples transformaciones de vectores en vectores. Notaremos $\bar{h}^j = \{h_i^j\}_{i=1}^{w_j}$ el resultado de la j-ésima capa. Los resultados de cada capa son denominados **activaciones** [Nielsen 2015].

Definiendo $\bar{h}^0 = x$ que constituye la entrada de nuestra red y a menudo se la llama **capa de entrada**.

A la dimensión de la capa $\bar{h}^j$, notada w_j se la conoce como **anchura de la capa**.

Cada una de dichas funciones (capas, de ahora en más) consiste en una transformación afín de la salida de la capa anterior (o el vector de entrada para el caso de la primer capa), seguida de una función no lineal ϕ^j (denominada **función de activación**).

Los modelos donde la información fluye desde la entrada hasta la salida sin retroalimentaciones se denominan ***feed forward*** (ver figura 3.23).

Suponiendo que tenemos m capas, el vector de las $\{h_i^m\}_{i=1}^{n_{out}}$ constituyen la salida de nuestra red. El número m de capas se denomina **profundidad**.

Podemos formular una red neuronal *feed forward* más formalmente -y de forma concisa- como:

$$\bar{h^0} = \bar{x}$$
$$\bar{h}^{j+1} = f^{j+1}(\bar{h}^j)$$
$$out_i = h_i^m \ 0 \leq i < n_{out}$$

Donde:

$$f^j(\bar{h}^{j-1}) = \phi^j({W^j}^\top \bar{h}^{j-1} + b^j)$$

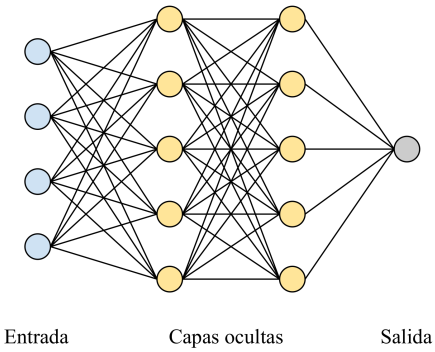


Figura 3.23: Esquema de un modelo de red neuronal *feed forward*. Análogamente a los modelos de una única capa oculta, las entradas se combinan linealmente y se le aplica una función no lineal para obtener activaciones ocultas. Existe un vector de activaciones ocultas por cada capa de valores intermedios (capa oculta) que resulta de combinar linealmente y aplicar una función no lineal sobre la capa anterior. Finalmente la última capa constituye la salida del modelo. Imágen extraída de [Pividori 2019]

Los valores de las matrices $W^j \in \mathbb{R}^{w_{j-1} \times w_j}$, $b^j \in \mathbb{R}^{w_j}$ son los parámetros a entrenar y se denominan respectivamente **pesos** y ***biases*** en la jerga del aprendizaje profundo. Son responsables del cambio de dimensión de una capa a otra de activaciones aplicando una transformación afín. La función de activación ϕ^j generalmente es una simple función $\mathbb{R} \to \mathbb{R}$ que es aplicada de forma independiente componente a componente.

Fijado el número m de capas, sus respectivas anchuras w_j y funciones de activación ϕ^j queda definida "el esquema" de la red: la cantidad de parámetros y su relación con las entradas queda fija. Las salidas solo dependerán de la elección de los parámetros. Ésto es lo que se conoce como **arquitectura** del modelo o red.

~~**Nota:** El Aprendizaje Profundo posee una jerga entrevesada para conceptos realmente simples. A menudo ésto ocurre a raíz de llamar de manera *vistosa* ciertos conceptos que ya existían para que suenen más innovadores a lo largo de las publicaciones que los han popularizado. Hay que evitar que esta jerga científica nos haga perder de vista la simpleza de los conceptos subyacentes del Aprendizaje Profundo: **función** de activación no es más que una función no lineal, **activaciones** son resultados parciales. En las secciones subsiguientes se seguirán introduciendo conceptos con nombres estrambóticos que tienen significados muy simples como las **unidades rectificadoras** que son simplemente la función $max(0, x)$. ¡Los principios matemáticos detrás de las Redes Neuronales Artificiales son realmente simples!~~

Observación: Con los conceptos definidos tenemos un panorma de Red Neuronal Artificial que satisface casi todos los puntos que en *"Parallel Distributed Processing (PDP)"* [Rumelhart et al. 1986d] se planteaba como necesario para *"imitar al cerebro humano"*. Tan sólo nos falta tener una forma de evaluar la experiencia y una regla de aprendizaje, que es lo que veremos en las secciones siguientes.

3.3.3. Descenso por gradiente

En esta sección describiremos la **regla de aprendizaje** utilizada para *ajustar* el modelo. Definida la arquitectura de una red neuronal, resta elegir valores óptimos de los parámetros W^j, b^j. Por simplicidad notaremos para esta sección al conjunto de todos los parámetros (*pesos* y *biases*) con W. Primero que nada necesitamos definir precisamente qué queremos optimizar. Supongamos que poseemos una función J para medir el error de nuestro modelo al aproximar la función objetivo dada una determinada asignación de pesos:

$$J : \mathbb{R}^{|W|} \to \mathbb{R}_0^+$$

Por ahora podemos pensar que J calculará las predicciones (salidas de la red) para cada ejemplo del conjunto de datos y lo comparará *de alguna forma* con las salidas esperadas.

Nota: notar que J recibe como entrada los *parámetros* del modelo. El conjunto de datos de evaluación (que será provisto como entrada al modelo) se considera fijo para el análisis de J. El conjunto de datos no es una entrada de la función J.

Las funciones que se utilizan para medir el error, comparando las entradas y las salidas son llamadas **funciones de costo**. En la sección siguiente se habla de qué funciones de costo se usan para cada problema y qué diferencia tiene con el concepto ya definido de *métricas*.

El método de actualización se basa en el gradiente[1] de los pesos en función del costo:

$$\nabla J(W) = \left(\frac{\delta J}{\delta W_0}, \frac{\delta J}{\delta W_1}, ..., \frac{\delta J}{\delta W_m} \right)$$

Gráficamente puede interpretarse al gradiente como la dirección (en el espacio de pesos) donde ocurre el mayor incremento de J de manera local (dentro de un entorno infinitamente pequeño). De manera opuesta $-\nabla J(W)$ determina la dirección de mayor decremento local.

El algoritmo de descenso por gradiente usa esto y actualiza los pesos "moviéndose" en la dirección opuesta al gradiente (ver figura 3.24):

$$W_i \leftarrow W_i - \eta \nabla J_i(W)$$

η es una constante positiva llamada ***learning rate*** que determina cuánto se modifica W en la dirección y magnitud del gradiente del costo. Notar que ésta fórmula es similar a la de actualización del original Perceptrón [Mitchell 1997].

El algoritmo converge cuando se alcanza un punto crítico (gradiente cero). Notar que en casos donde la función no es convexa es posible que el algoritmo converja a **mínimos locales** (que no sean el mínimo global) o **puntos de ensilladura** (ver figura 3.25).

En la práctica las funciones de costo poseen formas irregulares como muestra el trabajo [Hao Li et al. 2018], con varios mínimos locales (ver figura 3.26). Pero se ha corroborado que no encontrar mínimos globales no constituye un problema siempre que se encuentren mínimos locales con bajo costo [Goodfellow et al. 2016]. Además [Dauphin et al. 2014] muestra que para muchas funciones no convexas de alta dimensionalidad, los mínimos locales son infrecuentes respecto a los puntos ensilladura.

3.3.3.1. Sobre funciones de costo "*realistas*"

Las funciones de costo dependen tanto de la arquitectura como del conjunto de datos. Esta característica conceptual tiene una implicancia metodológica en el estudio de las funciones de costo, ya que si bien la arquitectura la controlamos y podemos variar, esto no posible con el conjunto de datos. Las entradas del modelo no poseen una distribución arbitraria sino que está restringido al dominio particular de estudio.

Si estamos entrenando con el conjunto de fotos de ***ImageNet*** [Deng et al. 2009], las imágenes esperadas en una aplicación final serán fotografías. No esperaremos, por ejemplo, una imágen de un espectrograma de sonido u obras de arte.

[1] Recordar que el concepto de *gradiente* para una función $\mathbb{R}^n \to \mathbb{R}$ es un vector donde cada componente es la derivada de la -única- salida respecto de cada componente de la entrada. [Apostol] [Deisenroth et al.]

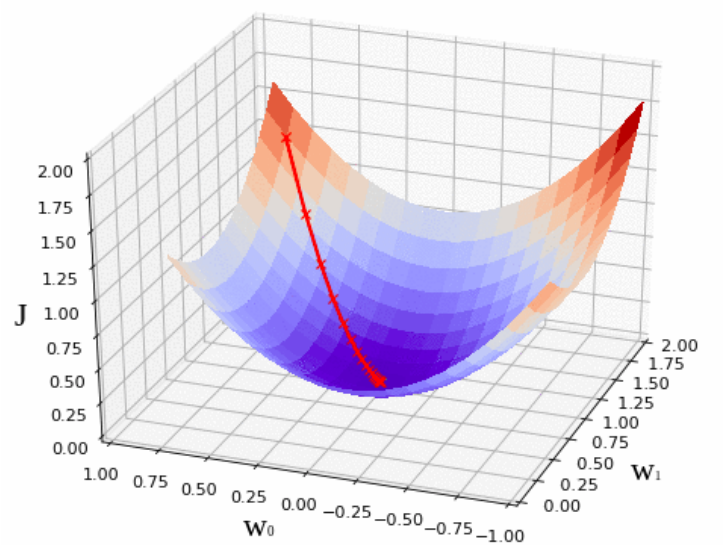


Figura 3.24: Ejemplo de descenso por gradiente en un espacio de dos parámetros para una función de costo convexa. En la práctica los espacios de pesos son de altísima dimensionalidad y las funciones suelen no ser suaves y poseer mínimos locales y puntos de ensilladura. Imágen extraída de [Pividori 2019]



Figura 3.25: En la figura izquierda podemos ver un ejemplo de función de dos parámetros no convexa. Se señalaron los mínimos locales. En la figura derecha se muestran los dos tipos de puntos críticos: mínimos y máximos locales, y puntos de ensilladura. El máximo global es un caso particular de éstos. Imágen extraída de [Pividori 2019] y [Zanetti 2019].

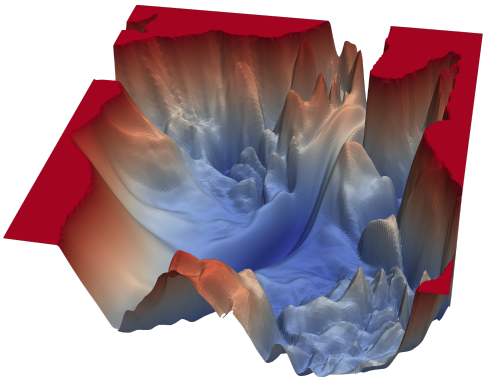


Figura 3.26: Visualización en dos dimensiones de la función de costo sobre un conjunto de datos reales. Fuente: [Hao Li et al. 2018].

Este concepto es clave ya que a veces se tratan de hacer análisis teóricos sobre funciones de costo arbitrarias cuando las conclusiones clave se han derivado más bien de análisis empíricos de funciones de costo *"realistas"*, o sea, considerando solamente conjuntos de datos reales.

Otra consecuencia es que la función de costo durante la evaluación del modelo no es la función de

costo sobre la totalidad del espacio de entrada que idealmente querríamos minimizar. Es importante contar con un conjunto de validación correctamente diseñado que nos garantice que la evaluación sea extrapolable y el modelo generalice bien. Lo mismo ocurre entre las funciones de costo respectivas al conjunto de entrenamiento y evaluación.

La limitación señalada en el párrafo anterior es intrínseca a la finitud de nuestros datos. Es por ello que los resultados han demostrado que más importante que encontrar mínimos globales, es el hecho de encontrar **mínimos suaves**, de modo que pequeñas diferencias entre las funciones de costo (al tratarse de conjuntos de datos distintos y por ende funciones distintas) supongan pocas variaciones entre los respectivos costos. Ver figura 3.27.

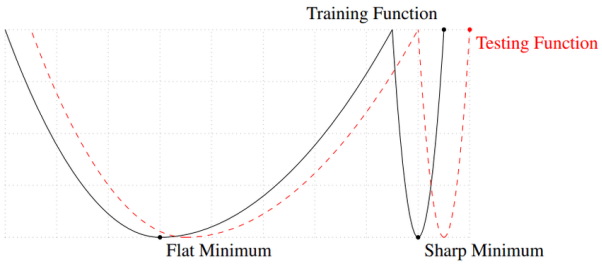


Figura 3.27: Los mínimos suaves supondrán, probablemente, cambios menores en el resultado del costo producto de las variaciones entre funciones de costo al variar los datos, y por ende presentan un **mayor poder de generalización** que los mínimos afilados. Fuente: [Howard and Gugger 2020]

3.3.3.2. Descenso por gradiente estocástico

Computar el gradiente del error sobre todo el conjunto de entrenamiento es costoso computacionalmente. En la actualidad los conjuntos de datos pueden ser muy grandes: ¡ya el conjunto de datos de imágenes *ImageNet* ([Deng et al. 2009]) contiene 14 millones de imágenes de resolución considerable! (generalmente se trabaja con recortes de 224x224 píxeles). Si se gasta demasiado tiempo para *cada actualización* de los pesos el método sería demasiado lento.

Una solución consiste en estimar el gradiente de la función de costo J teniendo en cuenta sólo un subconjunto de los datos de entrada (usualmente denominado ***mini-batch*** B). Como ya dijimos, la función de costo depende de los datos. Podemos pensar, entonces, que estamos calculando el gradiente de la función de costo definida con ese subconjunto de datos J_B:

$$B \subset D_t$$

$$w \leftarrow w - \eta \nabla J_B(w)$$

Se particiona el conjunto de entrenamiento D_t en *mini-batches* y para cada uno se ejecuta un paso denominado **iteración de entrenamiento**: se evalúa el modelo, se calcula el gradiente del costo y se actualizan los pesos. Las iteraciones de todas las particiones constituyen lo que se denomina **época de entrenamiento**. La cantidad de épocas es la cantidad de veces que el modelo ajusta el conjunto de datos de entrenamiento, por lo que al aumentarlo hará más probable que el modelo comience a sobreajustar.

La elección del **tamaño del *mini-batch*** supone una relación costo beneficio ya que a medida que lo incrementemos:

- Cada iteración de entrenamiento consumirá más tiempo y memoria.
- El gradiente calculado $\nabla J_B(w)$ será más estable y representará una mejor estimación del gradiente verdadero $\nabla J(w)$ (sobre todo el conjunto de datos).

Como el subconjunto de datos que se le suministra al modelo en cada iteración de descenso por gradiente es aleatorio, el método se denomina **descenso por gradiente estocástico**. La estocasticidad introducida hace que J_B sea distinto entre iteración e iteración y entre épocas sucesivas (ya que cambia B). Éstas variaciones suelen ser útiles para evitar mínimos filosos que no generalizan bien.

El hecho de entrenar en ***mini-batch*** en vez de utilizar un único ejemplo no solo genera gradientes más estables (buenos estimadores del gradiente real) sino que se presta a paralelizar en **GPU** los cómputos: lo que ha permitido un enorme salto en el desarrollo del Aprendizaje Profundo.

3.3.3.3. Importancia de la elección adecuada del *learning rate*

La elección del *learning rate* ha sido por tiempo uno de los hiperparámetros más importantes a la hora de entrenar redes neuronales. Como podemos ver en la figura 3.28, de ser demasiado pequeño puede generar convergencia muy lenta y de ser demasiado grande también, e incluso, el proceso puede diverger.

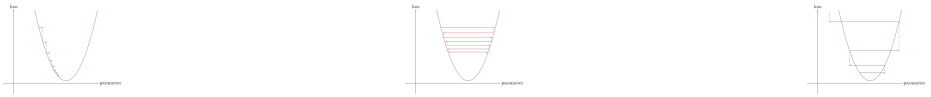


Figura 3.28: La elección del *learning rate* es importante para la convergencia del método de manera eficiente. Fuente: [Howard and Gugger 2020]

Más adelante se mostrarán optimizaciones más avanzadas que se basan en descenso por gradiente pero ajustan el *learning rate* conforme avanza el entrenamiento. También existen heurísticas que permiten tomar decisiones razonables para la elección del mismo.

Es importante volver a resaltar que como la estabilidad de los gradientes calculados dependen del **tamaño de *mini-batch***, de éste dependerá el valor óptimo de *learning rate*.

3.3.4. Función de costo

En el capítulo anterior asumimos que poseíamos una función que calculase el error para una asignación de parámetros determinada:

$$J : \mathbb{R}^{|W|} \to \mathbb{R}_0^+$$

A dicha función la nombramos **función de costo**. Luego introducimos un algoritmo para optimizarla, que se basa en el cálculo de ∇J.

¿Por qué no usar las ya introducidas **métricas** como funciones de costo? Una de las razones es que sus gradientes no poseen buenas propiedades por lo que no se ajusta bien a nuestro algoritmo de aprendizaje.

Las métricas tales como la *tasa de aciertos* son funciones escalonadas: un pequeño cambio en los pesos generará un cambio en las probabilidades de la predicción pero muy probablemente no cambiará la clase predecida, y por ende en la tasa de aciertos no se modificará. O sea el gradiente será nulo.

En cambio se utilizan como funciones de costo:

- En problemas de **regresión** el ya definido **Error Cuadrático Medio** (**MSE**).

  Además de ser una función diferenciable, presenta buenas propiedades de concavidad.

- En problemas de clasificación binaria ***Binary Cross Entropy*** [2] (**BCE**):

$$BCE(y, p) = \sum_{x_i \in D_v} y_i \; log(p_i) + (1 - y_i) \; log(1 - p_i)$$

  Observación: como $y_i \in \{0, 1\}$ (clasificación binaria), las expresiones y_i y $1 - y_i$ simplemente cumplen el rol de "elegir" que término sumar: $log(p_i)$ o $log(1 - p_i)$ respectivamente.

  Intuitivamente podemos pensar que si el *log* no estuviera es una *"tasa de aciertos"* continua que mide que distancia hay entre la probabilidad predecida y la predicción correcta. El efecto del *log* hace que la función pondere adecuadamente la diferencia entre clasificar, por ejemplo, con un error de 0,001 y 0,01 (en términos absolutos no es mucho pero en términos de error es 10× mejor).

[2] Centraremos nuestro análisis en *BCE* ya que, como se dijo, el problema a tratar en este trabajo es de clasificación binaria. No obstante el concepto de *BCE* se extiende fácilmente (y también sus propiedades) a la función de costo *Cross Entropy Loss* para problemas de clasificación de cantidad arbitraria de clases.

Es un caso particular de la función de entropía cruzada (en inglés *Cross Entropy Loss*), utilizada en probabilidad para calcular diferencia entre distribuciones, con bases en la teoría de la información [Deisenroth et al.].

Otro sustento matemático a la elección de dichas funciones de error es que sus fórmulas se derivan de los estimadores de máxima probabilidad (*Max Likelihood Estimators*).

3.3.4.1. Linearidad de gradientes respecto a última capa de activaciones

Una propiedad interesante surge de analizar $\frac{\delta J}{\delta z}$ siendo z la última capa de activaciones.

En problmeas de clasificación generalmente tendremos: [3]

$$J(W) = BCE(y, sigmoid(z))$$

Y los gradientes respecto a la última capa de activaciones serán:

$$\frac{\delta J}{\delta z} = sigmoid(z) - y$$

En problemas de regresión simplemente será:

$$J(W) = MSE(y, z)$$

Y los gradientes respecto a la última capa de activaciones serán:

$$\frac{\delta J}{\delta z} = 2(z - y)$$

En ambos casos los gradientes respecto a la última capa de activaciones serán proporcionales a la diferencia entre la predicción y el objetivo. Ésto hace que dicho gradiente que se "propagará" (como ya veremos en el capítulo de *backpropagation*) hacia las capas anteriores no poseerá saltos bruscos: se *suavizará* al aproximarse al objetivo tanto como se acerque al mismo. Sería problemático, si por el contrario aumentara exponencialmente al alejarse del objetivo, ya que, intuitivamente hablando, podría dar pasos tan grandes que saltase muy por ensima del mínimo y diverger (ver figura 3.28). Notar que también sería un problema si no disminuyese conforme se acercase al mismo, ya que en un entorno próximo al mismo son necesarios pasos cada vez más pequeños.

3.3.5. Funciones de activación

Notar la importancia de aplicar funciones no lineales: de no hacerlo la combinación sucesiva de transformaciones afines no sería otra cosa que una transformación afín, y por lo tanto sería inútil la aplicación de sucesivas capas: el modelo sería en su conjunto una simple regresión lineal.

Diferentes funciones de activación utilizadas son:

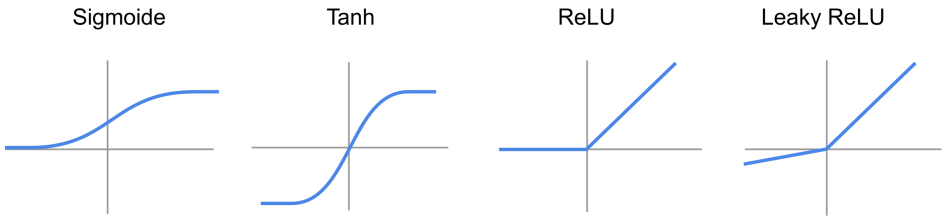


Figura 3.29: Diferentes alternativas para la función de activación.

[3] La nombrada *función sigmoide* es una función de activación utilizada frecuentemente en la última capa para problemas de clasificación binaria. Se verá con más detalle en 3.3.5.

- **Sigmoide** (σ):

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Las salidas de las neuronas sigmoideas no están centradas en el cero. Esto puede causar que el gradiente sea siempre positivo o negativo, causando inestabilidad en la actualización de los pesos.

Suele utilizarse, no obstante, sobre la última capa de activaciones (como última función de activación) para problemas de clasificación binaria. De este modo las predicciones son números en el rango $[0, 1]$ (consistentes con el hecho de reflejar probabilidades de pertenecer a la clase positiva o negativa). Además, como ya se explicó, posee buenas propiedades con respecto a *BCE* al ser utilizada como última función de activación.

- **Tangente hiperbólica** :

$$tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{1}{1 + e^{-2x}} - 1 = 2\sigma(2x) - 1$$

$Tanh$ es una sigmoide escalada y corrida. Resuelve el problema de centro distinto de cero.

Históricamente estas dos funciones surgieron como primeras alternativas por ser diferenciables en todo su dominio, y en continuidad con el concepto de neurona encendida/apagada que poseía el perceptrón originalmente.

Pero éstas funciones poseen el problema que el gradiente (derivada) es prácticamente nulo para valores de fuera del rango de [-2, 2]. Este problema se conoce como "saturación del gradiente". Ésto puede hacer muy pequeña la actualización de los pesos hacia capas anteriores (lo que se conoce como *señal de backpropagation*) en los casos donde la activación escape al mencionado rango. Este comportamiento es denominado como problema de desvanecimiento del gradiente (en inglés, *gradient vanishing*) y es sobretodo relevante en las actuales *arquitecturas profundas* (que poseen un gran número de capas). Intuitivamente "no les importa demasiado" que los pesos de capas anteriores cambien ya que esto no genera cambios sustanciales en la función de activación. Es por eso que ya no se las utiliza como funciones de activación para capas intermedias.

- ***Rectified Linear Unit*** (ReLU):

$$ReLU(x) = max(0, x)$$

Las ReLUs fueron introducidas en [Nair and Hinton 2010] y sus variantes adquirieron mayor popularidad, mostrando resultados superadores en el entrenamiento de redes profundas debido a que:

- Resuelven el problema de saturación del gradiente.
- Involucran operaciones simples (función $max(0, x)$ a diferencia de sigmoide o $tanh$ que involucran exponenciales), siendo computacionalmente menos costosas.
- En [Krizhevsky et al. 2012] se muestra que son 6 veces más rápidas que sigmoide o $tanh$ para converger utilizando el método de descenso por gradiente estocástico.
- En [Glorot et al. 2011] muestran que las neuronas con ReLUs representan un mejor modelo de las neuronas biológicas y aprenden mejor que $tanh$ o sigmoide las representaciones dispersas.
- Es simple demostrar y ver (como veremos cuando hablemos de teorema de aproximación universal) que cualquier función puede ser aproximada como combinación lineal de ReLUs

Las ReLUs poseen un problema denominado "neuronas muertas": el gradiente es 0 en el rango $(-\infty, 0)$ lo que anula la propagación a las capas anteriores y esto puede generar que conjuntos de neuronas de diversas capas no se activen. Y además, como tampoco actualizan sus respectivos pesos difícilmanete volverán a activarse.

Leaky ReLU introducidas en [Maas et al. 2013] y [Xu et al. 2015] pretende solucionar este problema agregando una pequeña pendiente en el rango $(-\infty, 0)$.

- ***Leaky ReLU***:

$$Leaky\ ReLU(x) = \begin{cases} \alpha x\ si\ x < 0 \\ x\ si\ x \geq 0 \end{cases}$$

Con:

$$0 < \alpha \ll 1$$

En general las ReLUs se siguen usando porque *Leaky ReLU* no ha mostrado una mejoría consistente.

- ***Softmax***: suele utilizarse en la última capa en problmeas de clasificación para transformar activaciones arbitrarias en probabilidades.

$$\sigma : \mathbb{R}^K \to [0,1]^K$$
$$\sigma(z)_i = \frac{e_{z_i}}{\sum_{j=1}^{K} e_{z_j}}$$

Intuitivamente puede pensarse que:

- "positiviza" las activaciones y las mapea al rango $[0, 1]$.
- hace que la suma de las probabilidades sea 1 (notar que sirve para problemas de clasificación donde la respuesta es una única clase, no múltiples clases).
- amplifica las diferencias entre las activaciones diferentes (tiende a tratar de escoger una única opción).

La elección de esta función tiene bases en la teoría de probabilidad.

El gradiente en la última capa de activaciones es lineal a la diferencia entre las predicciones y las etiquetas cuando se utilzia *softmax* como última función de activación junto a *BCE* como función de costo (al igual de lo que se mostró que pasaba con *sigmoid*). Cómo se explicó, esto es una propiedad deseada.

3.3.6. Teorema Aproximación Universal

Como se dijo en la subsección 3.3.1, el Perceptrón de Rosenblatt de una sóla neurona cayó en olvido luego de que Marvin Mibsky mostrara que funciones simples como XOR no podían ser representadas. En la década del 80' las redes neuronales se popularizaron nuevamente luego que se demostrara que se podía solucionar dicho problema agregando más de una capa.

Los *Teoremas de Aproximación Universal* son una familia de teoremas que demuestran las capacidadades de las redes neuronales de aproximar *casi* cualquier función. No entraremos en detelle sobre la formulación matemática exacta de dichos teoremas (el lector interesado puede encontrar una explicación muy amena en el capítulo 4 de [Nielsen 2015]).

Sólo nombraremos que se demuesta que es posible aproximar cualquier función continua en un espacio compacto de $\mathbb{R}^p$ a un grado de precisión arbitrario con:

- **Dos capas + anchura arbitraria**: dado un número suficiente de neuronas intermedias.
- **Anchura limitada + profundida arbitraria**: con redes suficientemente profundas.

El primer caso es muy fácil de demostrar utilizando funciones de activación ReLUs. En la figura 3.30 se muestra un esquema intuitivo. La idea subyacente es dividir el espacio compacto en intervalos y aproximar la función utilizando rectas adecuadas en cada intervalo. A su vez esto es fácil de lograr eligiendo e incorporando a la ecuación $ReLUs$ escaladas y corridas convenientemente para que generen dichas rectas lineales, siempre teniendo en consideración las $ReLUs$ ya elegidas.

¡Notar que los teoremas no dan ninguna garantía de que el algoritmo de entrenamiento pueda aprendar esa función! E incluso siempre tendremos el limitante de la finitud de nuestros datos: no existe un procedimiento universal para elegir una función que generalice a puntos fuera del conjunto de entrenamiento [Wolpert 1996].



$$
\begin{aligned}
n_1(x) &= Relu(-5x - 7.7) \\
n_2(x) &= Relu(-1.2x - 1.3) \\
n_3(x) &= Relu(1.2x + 1) \\
n_4(x) &= Relu(1.2x - .2) \\
n_5(x) &= Relu(2x - 1.1) \\
n_6(x) &= Relu(5x - 5)
\end{aligned}
$$

$$
\begin{aligned}
Z(x) = &-n_1(x) - n_2(x) - n_3(x) \\
&+ n_4(x) + n_5(x) + n_6(x)
\end{aligned}
$$

Figura 3.30: La imagen muestra la aproximación de un polinomio de tercer grado con una "red de dos capas". Fuente: `towardsdatascience.com/can-neural-networks-really-learn-any-function-65e106617fc6`. Autor: Brendan Fortuner

Grandes errores se han cometido en el desarrollo del aprendizaje profundo por no entender que se trata de una disciplina práctica. En la década del 80' muchos científicos, en base al resultado de los teoremas de aproximación de dos capas, no experimentaron con redes más profundas. En la práctica éstas han mostrado una contundente mayor eficiencia produciendo resultados increíbles [Goodfellow et al. 2016].

Debido al éxito de entrenar redes neuronales con muchas capas es que también nos referimos con un nombre propio al campo que estudia las redes neuronales profundas: **Aprendizaje Profundo**,

3.3.7. Algoritmo de *backpropagation*

En esta sección continuaremos explicando cómo se optimizan los parámetros. Explicaremos como se calculan los gradientes para cada peso siguiendo un algoritmo conocido como ***backpropagation***.

Ya se mostró que la arquitectura de nuestra red puede verse como composición sucesiva de funciones sobre los datos de entradas. Es natural entonces que para calcular los gradientes nos basaremos en la **regla de la cadena**. La misma nos dice que si tenemos:

$$E_1(w) = f(E_2(w))$$

Luego:

$$\frac{\delta E_1}{\delta w}(w) = \frac{\delta f}{\delta E_2}(E_2(w)).\frac{\delta E_2}{\delta w}(w)$$

Utilizamos la notación de Leibinz extendida para resaltar en qué punto se evalúa cada derivada.

Nota: cuando hablamos de gradientes en redes neuronales nos referimos a *valores* y no a las *funciones*. A diferencia de lo que sucede en matemáticas donde es frecuente querer encontrar o calcular la función de gradiente o derivada, en cada iteración del entrenamiento, sólo necesitamos la derivada del error respecto a cada peso **aplicada en la asignación actual de pesos**.

En un principio, cuando las arquitecturas eran simples, dichas derivadas se calculaban con una fórmula codificada a mano para cada matriz W^j, b^j. Pero si se observa con atención, la regla de la cadena nos da una forma inductiva de calcular los gradientes del error respecto de cada capa, haciendo uso de los gradientes ya calculados de capas posteriores.

Matemáticamente si tenemos las capas f^1, f^2 ... f^n y las aplicamos sucesivamente según:

$$out = f^n(...f^2(f^1(x)))$$

Y recordamos la notación para las activaciones tal como la definimos en la subsección 3.3.2:

$$\bar{h^0} = \bar{x}$$

$$\bar{h}^{j+1} = f^{j+1}(\bar{h}^j)$$

Luego podemos calcular los gradientes $\frac{\delta J}{\delta h^j}$ como:

$$\frac{\delta J}{\delta h^j} = \frac{\delta J}{\delta h^{j+1}}(h^{j+1}).\frac{\delta h^{j+1}}{\delta h^j}(h^j)$$

Esta fórmula nos da una forma constructiva (algorítimica) de calcular los gradientes respecto de cada una de las activaciones, empezando por las capas más profundas (más cercanas al resultado) y conociendo la derivada $\frac{\delta h^{j+1}}{\delta h^j}(h^j)$ de la fórmula que computa una activación a partir de la anterior.

Una vez que los gradientes respecto a cada capa h^j han sido calculados, se puede computar de manera simple el gradiente respecto a los parámetros (pesos W^j y *biases* b^j) intervinientes en las respectiva capa. [LeCun et al. 2015]. Simplemente, conociendo la derivada de ϕ^j, podemos calcular $\frac{\delta J}{\delta W^j}$ y $\frac{\delta J}{\delta b^j}$ -en función del ya conocido $\frac{\delta J}{\delta h^j}$- valiéndonos de la fórmula de la capa:

$$\bar{h}^j = f^j(\bar{h}^{j-1}) = \phi^j({W^j}^\top \bar{h}^{j-1} + b^j)$$

En éste cálculo la información fluye "*hacia atrás*" y es por eso que se denomina **algoritmo de *backpropagation*** [Rumelhart et al. 1986c]. En contraste a éste concepto se llama ***forward propagation*** al cáclulo de las predicciones, ya que la información en este caso fluye "*hacia adelante*".

Notar que en los cálculos intervienen los valores h_j, lo que significa que muchas veces será necesario guardar el valor de las activaciones (o al menos cierta información de ellas) para el cálculo posterior de los gradientes.

Nota: ¡El algoritmo de backpropagation es un caso de uso de programación dinámica! Estamos usando los resultados de instancias más chicas, o sea de las capas más cercanas al error, para calcular los casos "más complejos" de las capas menos profundas.

El mismo razonamiento es fácilmente generalizable a cualquier grafo acíclico de computaciones. Ésto es lo que usan los frameworks actuales para calcular los gradientes de manera transparente al usuario: en general no necesitaremos calcular ningún gradiente a mano. Cada una de las funciones definidas, definen un método para calcular los gradientes respecto de sus entradas utilizando como dato los gradientes respecto a esa misma capa. Los frameworks calculan un grafo de ejecucción que lleva el rastro de cómo se fue calculando determinado resultado. De modo que una vez calculado el error (o la función sobre la que queremos calcular los gradientes) es fácil calcular el gradiente de dicha función respecto cada capa (y luego de cada parámetro) haciendo uso de programación dinámica (*backproapagation*) sobre el DAG de ejecucción que calculó el error. Ver esquema en la figura 3.31

3.3.8. Extensiones al algoritmo de descenso por gradiente

En esta subsección explicaremos algunas optimizaciones que se hacen al algoritmo de descenso por gradiente estocástico para mejorar su estabilidad y convergencia. Las mismas, conforme avanza el entrenamiento, van adaptando y calculando valores que influyen en la actualización de los pesos.

Generalmente estas optimizaciones ya vienen integradas en las bibliotecas que permiten crear y entrenar modelos de redes neuronales (como Pytorch y Tensorflow) y han sido utilizadas en el presente trabajo.

Notaremos t al número de iteración y w_t a los pesos al comienzo de la t-ésima iteración.

Una lista más extensa y comparaciones de las diferetnes optimizaciones se puede encontrar en [Ruder 2016].

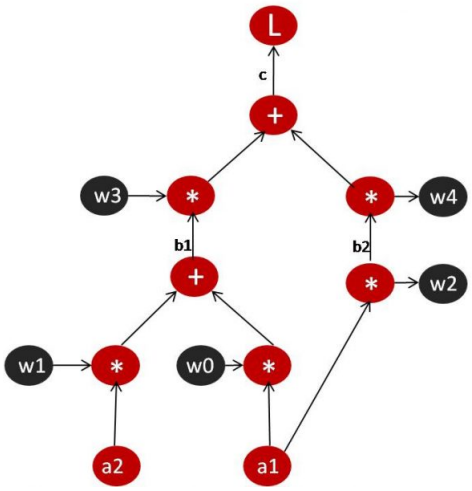


Figura 3.31: Ejemplo de grafo de ejecución construído en base al cálculo de un expresión. Créditos de la imágen: `mlearninglab.com/2019/04/14/autograd-una-de-las-claves-de-la-flexibilidad-de-pytorch/`

3.3.8.1. Momentum

Esta extensión fue propuesta en [Rumelhart et al. 1986]. Consiste en "recordar" información de las actualizaciones de pesos de iteraciones anteriores, calculando una media móvil exponencial de las actualizaciones m_t, ponderada por un hiperparámetro β:

$$m_t = \beta\ m_{t-1} + (1-\beta)(-\eta\ \nabla J(w_t))$$

$$w_{t+1} = w_t + m_t$$

Intuitivamente puede pensarse que el descenso por gradiente tendrá una *inercia a la dirección que ya poseía*, de ahí que se lo llame **momentum** (por momento de inercia).

3.3.8.2. RMSProp

RMSProp (por *Root Mean Square Propagation*) fue introducido por [Hinton 2014]. Propone utilizar un *learning rate* para cada parámetro de modo que los que *"necesitan cambiar más"* posean valores más altos.

Esto lo logra mirando los gradientes, asumiendo que parámetros con gradientes más cercanos a 0 necesitan un *learning rate* más alto para salirse de zonas planas. En cambio, si los gradientes son altos en valor absoluto significa que estamos en zonas empinadas, dónde conviene moderar el *learning rate* para evitar divergencia. El factor que mide cuánto se tiene que reducir el *learning rate* se lo nota v_t (y es diferente para cada peso).

Para que v_t no sea ruidoso (cambie demasiado entre iteraciones) y, en cambio, determinar una tendencia general del valor absoluto de los gradientes, se calcula la siguiente media móvil exponencial:

$$v_t = \beta v_{t-1} + (1-\beta)(\nabla J(w_t))^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla J(w_t)$$

Se agrega ϵ por razones de estabilidad numérica para evitar un denominador nulo.

3.3.8.3. Adam

Debe su nombre a *Adaptive Moment Estimation* y es una combinación de Momento y RMSProp propuesta en [Kingma and Ba 2014].

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\nabla J(w_t)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)\nabla (J(w_t))^2$$

Adam utiliza la media móvil no sesgada en su fórmula final. De modo que corrije el sesgo a 0 existente en las medias móviles desde el inicio, al ser inicializadas en 0. Este sesgo es especialmente fuerte si β_1, β_2 son cercanos a 1 (como los valores que sugiere [Kingma and Ba 2014]).

$\hat{m}_t$ y $\hat{v}_t$ son los estimadores no sesgados respecto a las medias móviles m_t y v_t:

$$\hat{m}_t = \frac{m_t}{1 - (\beta_1)^t}$$

$$\hat{v}_t = \frac{v_t}{1 - (\beta_2)^t}$$

Finalmente la actualización de los pesos se calcula según:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}}\hat{m}_t$$

3.3.9. Inicialización de los parámetros

Como explica el capítulo 17 de [Howard and Gugger 2020], es importante la inicialización correcta de los pesos, sobretodo en redes profundas ya que se calculan muchas multiplicaciones sucesivas de las entradas con ellos. De inicializarlos muy pequeños (en valor absoluto) puede que las activaciones se anulen luego de sucesivas capas. Además, de iniciarlizarlos con valores absolutos demasiado altos (como las ReLU no limitan la saldia de cada capa), puede que incrementen exponencialmente el valor de las activaciones llegando a resultados computacionalmente inmanegables (valores *nan*, de *"not a number"*) causando errores de desbordameinto (*overflow errors*)

En [Glorot and Bengio 2010] se determina que la escala adecuada para inicializar los pesos es $\frac{1}{\sqrt{n_{in}}}$ siendo n_{in} la cantidad de entradas de la capa. La **inicialización Xavier** inicializa los pesos en valores aleatorios con distribución gaussiana de media 0 y varianza $\frac{1}{\sqrt{n_{in}}}$.

Este factor depende del tipo de función de activación utilizada y [Glorot and Bengio 2010] lo han probado para *tanh*. En [He et al. 2015] (mismo trabajo que introduce las redes residuales) determina que la escala en caso de utilizar *ReLU* es $\sqrt{\frac{2}{n_{in}}}$. Esta es la escala que utiliza el método de **inicialización Kaiming**.

3.3.10. Mejorando la generalización - técnicas de regularización

Otro desafío que supone el entrenamiento de redes neuronales es el del **sobreajuste**. Su gran complejidad, consecuencia de la gran cantidad de parámetros posibilita aprender funciones complejas pero también hace que sean más suceptibles a sobreajustar.

A veces se piensa -erróneamente- que la solución adecuada en estos casos es simplificar el modelo (reducir la cantidad de parámetros), la práctica del aprendizaje profundo desvela que es la última de las opciones a la hora de combatir el sobreajuste.

Existen una multitud de técnicas conocidas cómo **técnicas de regularización**, que permiten modificar el algoritmo de aprendizaje para producir modelos que generalicen mejor, sin achicar la cantidad de parámetros.

En [Howard and Gugger 2020] se dice que si no podemos entrenar un modelo hasta que el mismo empiece a sobreajustar no estamos aprovechando en su totalidad la capacidad del mismo.

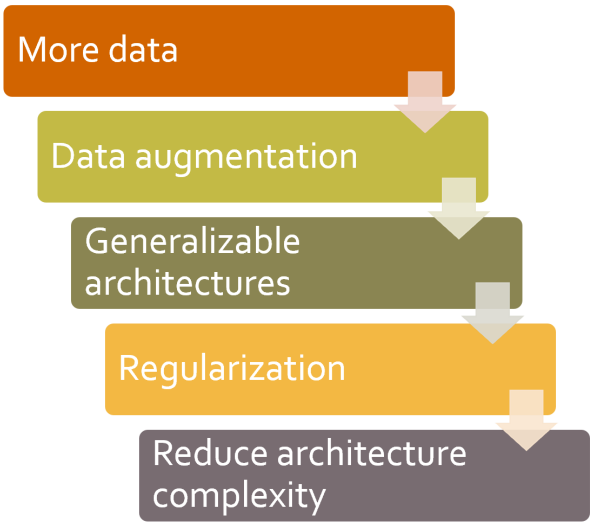


Figura 3.32: [Howard and Gugger 2020] nos sugiere en su capítulo 5 seguir esta línea de acción ante el problema del sobreajuste. Fuente: [Howard and Gugger 2020].

También indica que la manera correcta de detectar el sobreajuste es **cuando el error de validación empieza a empeorar** (y no comparando el valor de la función de costo en el entrenamiento y la validación como muchos hacen). Después de todo, si el error de validación sigue disminuyendo y el conjunto de validación está bien confeccionado, significa que nuestro modelo está mejorando.

Ante la presencia del sobreajuste, reducir la complejidad del modelo tiene que ser la última opción en redes neuronales (ver figura 3.32): [Howard and Gugger 2020] sugiere intentar antes con técncias de *data augmentation*, agregar regularización en la arquitectura (como capas de *batch normalization* y *dropout*) y aumentar luego los valores de regularización (incrementar los hiperparámetros que influyen en ella como λ de *weight decay*, o el p de las capas de *dropout*). Todas estas técnicas se verán en las subsecciones siguientes.

3.3.10.1. *Data augmentation*

El proceso de ***Data Augmentation*** consiste en generar datos sintéticos durante la fase de entrenamiento. Muy a menudo al contar con pocos datos, el modelo ajustará demasiado los datos de entrenamiento disponibles, lo que dará lugar (como ya se ha explicado en secciones anteriores) al sobreajuste.

Existen métodos de generar datos sintéticos que son dependientes de la naturaleza de los datos. Por ejemplo en el caso de imágenes es común generar más datos aplicando transformaciones como rotación, traslación, escalamiento, espejamiento o diferentes tipo de deformación mientras que no cambien sustancialmente la semántica de la imágen (ver ejemplo figura 3.33). Estas transformaciones son combinables. Debe tenerse en cuenta que tengan sentido en el contexto del problema elegido. Por ejemplo si estamos reconociendo objetos espejar es una operación válida, mientras que si estamos reconociendo texto no lo será.

También existen métodos estadísticos generales de *data augmentation*. Por ejemplo el agregado de ruido en función a la distribución de los datos, que son aplicables a diversos tipos de datos (imágenes, sonido, vector de características, etc). Además existen métodos más avanzados como las Redes Generativas Adversiales capaces de aprender la distribución intrínseca de los datos, y a partir de ello generar datos sintéticos.

Un método simple que arroja muy buenos resultados en problemas de clasificación, utilizado en Aprendizaje Profundo, es la técnica de **Mixup** [Zhang et al. 2017] que consiste en generar nuevos ejemplos a partir de hacer un promedio ponderado entre dos datos (ver ejemplo figura 3.34). El mismo promedio, con igual ponderación, se aplica a las etiquetas para generar una nueva etiqueta sintética



Figura 3.33: La librería fastai provee una forma de realizar *Data Augmentation* en la GPU, acelerando considerablemente el entrenamiento. Fuente: [Howard and Gugger 2020].



Figura 3.34: Un ejemplo de la técnica *Mixup* que combina ponderadamente dos imágenes (estación de gasolina y templo). Fuente: [Howard and Gugger 2020]

respectiva. Ver formulación matemática en la figura 3.35.

Contribution Motivated by these issues, we introduce a simple and data-agnostic data augmentation routine, termed *mixup* (Section 2). In a nutshell, *mixup* constructs virtual training examples

$$\tilde{x} = \lambda x_i + (1-\lambda)x_j, \qquad \text{where } x_i, x_j \text{ are raw input vectors}$$
$$\tilde{y} = \lambda y_i + (1-\lambda)y_j, \qquad \text{where } y_i, y_j \text{ are one-hot label encodings}$$

Figura 3.35: La formulación matemática de *Mixup* según [Zhang et al. 2017].

3.3.10.2. *Batch normalization*

Un problema que causa entrenamientos infecientes es el denominado ***"internal covariance shift"*** explicado en [Ioffe and Szegedy 2015]: la entrada de cada capa cambia conforme se entrenan las capas anteriores, por lo cuál le costará aprender los pesos definitivos hasta que las capas anteriores hayan *convergido suficientemente*. Podemos pensar intuitivamente que desde la perspectiva de cada capa es como si *"le cambiaran el conjunto de datos"* todo el tiempo.

Una ingeniosa manera de ver esto es en las gráficas propuestas en el capítulo 13 de [Howard and Gugger 2020] donde se grafica mediante mapa de color los histogramas de los valores de activación en función a iteraciones de entrenamiento. Como se puede ver en la figura 3.36 las activaciones comienzan casi todas en cero. Luego, cíclcicamente, incrementan su valor exponencialmente y vuelven a cero (como si volvieran a comenzar).

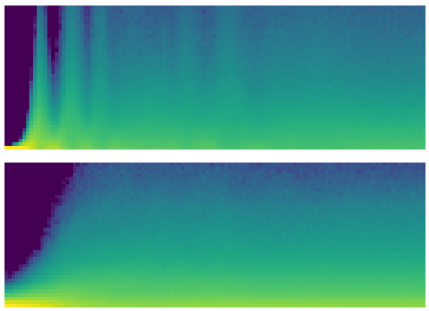


Figura 3.36: Se grafica para cada valor de activación (eje y) su cantidad en un hisotrama (color, el cero corresponde a colores más oscuros), conforme avanzan las iteraciones (eje x). En la primer figura se muestra un entrenamiento sin regularización. Se puede observar como las activaciones aumentan muy velozmente y vuelven a cero cíclicamente. En la segunda figura, luego de aplicar *batch normalization*, este comportamiento desaparece: el aumento de los valores de las activaciones es más lento y consistente. Fuente: capítulo 13 de [Howard and Gugger 2020].

Como solución a eso [Ioffe and Szegedy 2015] propone normalizar cada capa de activaciones para que tengan media 0 y varianza 1. De modo de reducir ese cambio en la varianza de capas internas (*internal covariate shfit*). Sobre la salida de activaciones h_i de cada mini-batch se calcula su media μ y varianza σ^2 y se normaliza según:

$$BN(h_i) = \frac{h_i - \mu}{\sigma}$$

Para no limitar la capacidad del modelo a producir activaciones más grandes, se introducen dos parámetros *entrenables* γ y β. Serán optimizados por el método de entrenamiento como cualquier otro peso. Así se permite *aprender* la media y varianza de la salida:

$$out_i = \gamma\ BN(h_i) + \beta$$

El método de Batch normalization hace más estable el entrenamiento, permitiendo incrementar el *learning rate* y entrenar más rápidamente (menos iteraciones). También parece generar modelos que generalizan mejor, probablemente por la introducción de estocasticidad (que el modelo tiene que superar volviéndose más robusto) por la variabilidad de μ y σ^2 de las activaciones entre diferentes *mini-batches*.

En el figura 3.36 es evidente el cambio en la estabilidad de las activaciones conforme avanzan las iteraciones luego de aplciar *batch normalization*.

3.3.10.3. *Weigth decay*

Técnica introducida por [Krogh and Hertz 1991] que consiste en añadir un término al error que penalice la suma de los cuadrados de los pesos:

$$J_{Weight\ Decay}(W) = J(W) + \lambda \sum W_i^2$$

λ es un hiperparámetro que controlará la cantidad de penalización aplicada. Valores más altos entorpecerán un poco más nuestro entrenamiento pero harán que el modelo resultante generalice mejor.

Esto incentiva elegir pesos pequeños. La razón detrás de ello es que pesos grandes generan mínimos afilados en la función de costo que, como se mostró, generalizan mal. Un argumento intuitivo se expone en la figura 3.37.

Es fácil ver que el término de penalización en la función de costo agregará un término de $2\alpha W_i$ sobre los gradientes de cada peso. En la práctica no se añade explícitamente el término a la función de costo sino que se modifica el cálculo de la actualización de los pesos agregando un factor de W_i:

$$\Delta W_i = -\eta \nabla J(W)_i - \eta \lambda w_i$$

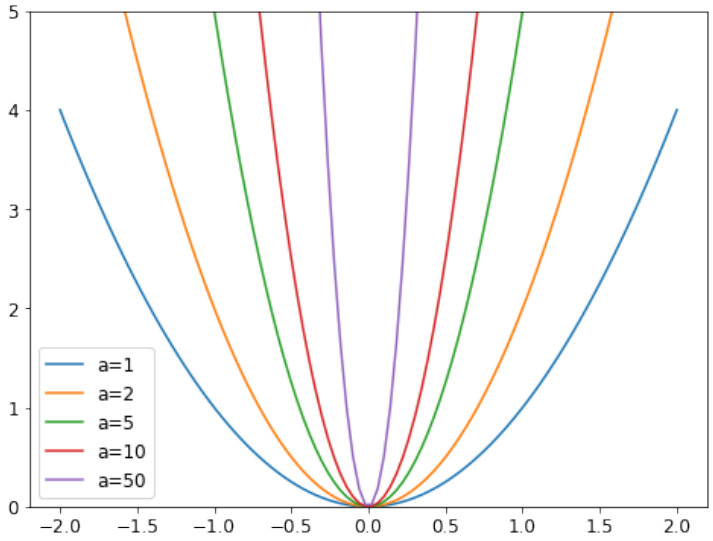


Figura 3.37: Al aumentar w en la fórmula wx^2 la parábola se vuelve más "afilada". Fuente: capítulo 8 [Howard and Gugger 2020].

O también se lo puede agregar en el cálculo del gradiente:

$$\nabla \hat{J}(W)_i = \nabla J(W)_i + \lambda w_i$$

Ambas son equivalentes para el método de descenso por gradiente estocástico pero en el trabajo desarrollado por [Loshchilov and Hutter 2019] se recomienda la segunda cuando se aplica en combinación con las técnicas momentum o Adam, debido a que es más fiel a la formulación original, y también muestra mejores resultados prácticos.

3.3.10.4. *Dropout*

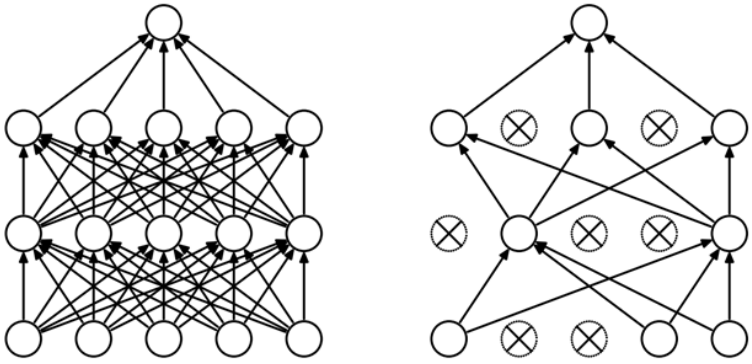
Es una técnica de regularización introducida en [Srivastava et al. 2014] que consiste en desactivar aleatoriamente ciertas neuronas (llevando su respectiva activación a 0) durante el entrenamiento. Ver figura 3.38.

La técnica funciona asombrosamente bien y ha generado un salto en el estado del arte del aprendizaje profundo. La intuición detrás de ella es que:

- Mejora la cooperación entre neuronas impidiendo que intrincadamente neuronas de diferentes capas aprendan funciones de "memorización" del conjunto de entrenamiento. Incentivando que aprendan a extraer verdaderas características útiles y robustas.
- Introducen ruido en las activaciones, haciendo que se aprendan características más robustas en cada capa.

El *dropout* se aplica sobre determinadas capas e incluso se ha mostrado su utilidad aplicándolo sobre la entrada para introducir ruido. Ésto puede pensarse como un caso de *data augmentation*.

Notar que el apagado de las neuronas de una capa con probabilidad p durante el entrenamiento requiere que se escale las salidas restantes en $\frac{1}{1-p}$. De lo contrario la capa posterior recibiría en promedio p activaciones en 0 más durante el entrenamiento que durante fase de validación o durante la inferencia, lo que generaría diferentes valores promedios de activación en el entrenamiento respecto de validación, y arruinaría totalmente la generalización.



(a) Standard Neural Net (b) After applying dropout.

Figure 1: Dropout Neural Net Model. **Left**: A standard neural net with 2 hidden layers. **Right**: An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

Figura 3.38: Imagen esquemática extraída de *"Dropout: A Simple Way to Prevent Neural Networks from Overfitting"* [Srivastava et al. 2014].

3.3.11. Configurando el *Learning Rate*

Como se explicó en el apartado 3.3.3.3, el *learning rate* es uno de los parámetros más importantes en el Aprendizaje Profundo: de ser muy pequeño el modelo puede tardar demasiado en entrenar, y de ser muy grande el entrenamiento puede diverger.

Optimizadores tales como Adam (explicado en el apartado 3.3.8.3) permiten escalarlo adecuadamente según el valor del gradiente. Pero notar que aún utilizando Adam, es necesario la elección del *learning rate* inicial. Siendo el escalamiento aplicado por Adam independiente del valor inicial elegido.

Otras optimizaciones como *Batch Normalization* (apartado 3.3.10.2) y *Weight Decay* (apartado 3.3.10.3), mitigan los riesgos de divergencia existentes derivados de utilizar valores de *learning rate* elevados. No obstante, no resuelven el problema de escoger un valor inicial.

3.3.11.1. Seteando el *Learning Rate* inicial

Cansado de que el modo de hallar el *learning rate* óptimo sea cuestión de prueba y error, generalmente utilizando métodos como *Grid Search* que requieren re-entrenar el modelo varias veces (consumiendo demasiado tiempo), Leslie N. Smith[4], en [Smith 2015], describe un ingenioso método para encontrar el *learning rate* inicial que denominó *Range Test*. [5] El mismo consiste en probar *learnings rates* incrementales (desde un valor pequeño como 10^4) e ir aumentándolo conforme avanzan las iteraciones (para siguientes *mini-batches*) hasta que el valor de la función de costo empieze a aumentar demasiado indicando divergencia (ver figura 3.39). Luego, se corrobora empíricamente que un buen *learning rate* inicial resulta de elegir un orden de magnitud por debajo del mínimo o un valor luego del cual se presente una caída abrupta del costo. Intuitivamente, para este valor la función de costo sigue decreciendo.

3.3.11.2. *Cycle Learning Rates*

Una técnica utilizada es hacer descender el *learning rate* a medida que avanza el número de iteraciones, intuitivamente, para que al acercarse al minimo no se "saltase" por ensima de él.

[4]**Nota:** Leslie N. Smith se doctoró en Química Cuántica en la Universidad de Illinosis y es investigador del *Naval Research Laboratory* de los Estados Unidos. Se introdujo al Aprendizaje Profundo en 2013 y sus estudios han contribuído en hallar técnicas para entrenar más rápido y con menos datos. Este es un paso fundamental a la hora de democratizar la Inteligencia Artificial y volverla más accesible a más personas (y no solo un puñado de grandes compañías). Veáse esta entrevista que hizo para Jeremy Howard: `youtube.com/watch?v=dxpyg3mP_rU`

[5]También es conocido bajo el nombre de *Learning Rate Finder* por la biblioteca de Aprendizaje Profundo FastAI, la cual popularizó el método usándolo en su implementación.

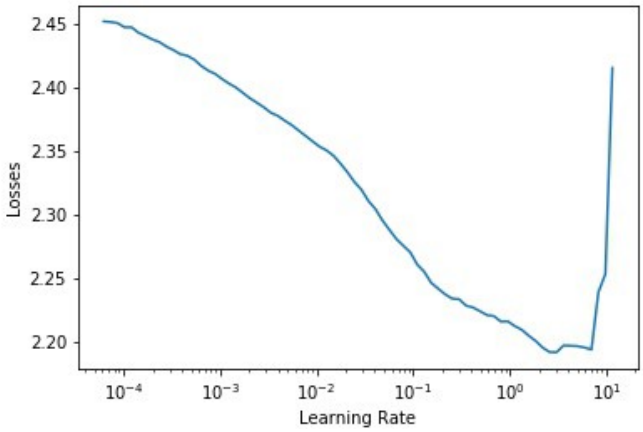


Figura 3.39: La librería FastAI provee un método denominado *Learning Rate Finder* que ejecuta el test descripto en [Smith 2015] y muestra la gráfica de la función de costo a medida que se aumenta el *learning rate*. En este caso particular una elección de *learning rate* posible oscila entre 10^{-1} y 0,5 (un orden de magnitud menor que el mínimo). Fuente: fastai

En [Smith 2015] se sugiere hacer oscilar el valor del *learning rate*, en cada época, entre dos umbrales (ver figura 3.40). La experimentación muestra que así se logran convergencias más rápidas.

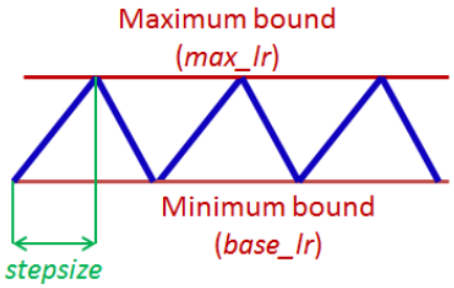


Figura 3.40: Imagen esquemática extraída de [Smith 2015]. Se halla empíricamente que incrementando y decrementando cíclicamente el *learning rate* se obtienen entrenamientos más veloces y modelos que generalizan mejor.

Una de las razones intuitivas es que permite superar puntos de ensilladura (ver ejemplo figura 3.41). Los puntos de ensilladura son un obstáculo para el método de descenso por gradiente, ya que son zonas planas, difíciles de superar, y por ende, donde el entrenamiento se ve realentizado. Además, según reporta [Dauphin et al. 2015], son mucho más frecuentes que los mínimos locales en las topologías de funciones de costo.

En [Smith and Topin 2017a], se ha reportado mejoras significativas en la velocidad de entrenamiento aplicando ésta técnica (utilizando en CIFAR-10 y CIFAR-100 y la familia de ResNets). El autor ha denominado a este fenómeno "Super - Convergencia". [Smith and Topin 2017b]

3.3.11.3. *One Cycle Policy*

Finalmente en [Smith 2018], L. Smith sugiere un política de único cíclo por época para variar el *learning rate*: comenzando con valores pequeños, aumentando al principio del ciclo y disminiyéndolo sobre el final (ver figura 3.42.a). Además fija el tamaño del ciclo un poco menor al número de iteraciones de una época, y en las últimas iteraciones disminiye progresivamente el *learning rate* hacia 0 (esta idea está inspirada en *"Simulated Annealing"* [Aarts and Korst 1988]).

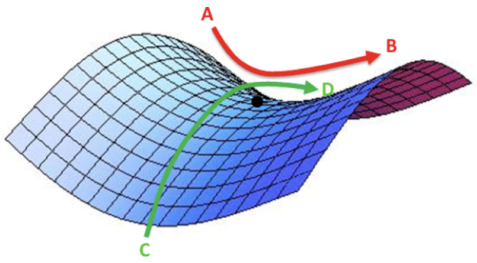


Figura 3.41: Punto ensilladura para una entrada 2-dimensional. Créditos de la imágen safaribooksonline



Figura 3.42: En la imágen podemos observar cómo varía el *learning rate* (izquierda) y *momentum* (derecha) según la Política de Único Cíclo descripta en [Smith 2018]. Créditos de la imágen: *"Finding Good Learning Rate and The One Cycle Policy"*, autor Nachiket Tanksale.

En [Smith 2018] se corroboran mejores resultados de variar el *momentum* de manera inversa a la variación del *learning rate*: comenzando por el umbral máximo, reduciéndolo y luego aumentándolo (ver figura 3.42.b). Para finalmente utilizar el valor máximo de *momentum* para la fase de *annealing* del *learning rate*

La intuicion detrás de ello es que en los momentos donde el *learning rate* se está aumentando para superar puntos de ensilladura, uno desearía que el algoritmo de descenso por gradiente rápidamente vaya hacia nuevas direcciones, para lo cual es necesario que tenga baja inercia.

Otra forma de entender por qué es razonable variar el *momentum*, se desprende de observar la fórmula del mismo (presentada en el apartado 3.3.8.1) y notar que $1 - \beta$ (β es el *momentum*) tiene un impacto similar a λ (*learning rate*) en la actualización de los pesos:

$$m_t = \beta \; m_{t-1} + (1 - \beta)(-\eta \; \nabla J(w_t))$$

Por lo que es razonable que si en nuestra política variamos λ querremos que β variase en sentido opuesto (y que no nos "contrarreste" la variación).

En [Smith 2018] se reportan mejoras en la velocidad de entrenamiento de *ResNet-50* para el dataset CIFAR-10 y CIFAR-100 de hasta $\times 10$. Además se obseva que los modelos generalizan mejor (presentan mejores resultados de validación). Esto es atribuído, a que entrenando con *learnings rates* más altos se encuentran mínimos más suaves, ya que de encontrar uno empinado probablemente se "salte" fuera del mismo. Como se explicó en el apartado 3.3.3.1 los mínimos suaves generalizan mejor. Una vez que -aumentando progresivamente el *learning rate* para evitar divergencia- se encuentran zonas con mínimos más suaves, la fase de disminución progresiva hace que encontremos la mejor parte de dicha zona (la de menor costo).

3.3.12. Aprendiendo las representaciones

Anteriormente al Aprendizaje Profundo los sistemas solían tener una fase de extracción de características que luego eran suministradas a un algoritmo de Aprendizaje Automático para entrenar [Stevens and Antiga 2019]. La primer fase era dependiente del dominio: necesitaba ingeniería de características propia para cada problema (para imágenes se utilizaban filtros detectores de bordes, cantidad de agujeros, etc).

Por el contrario, en el Aprendizaje Profundo las características adecuadas son *aprendidas* (calculadas automáticamente) por el modelo mismo. El cual, a medida que es entrenado, produce mejores características.

Usualmente las representaciones aprendidas son mejores que las que puede escribir un experto manualmente. Ver figura 3.43.

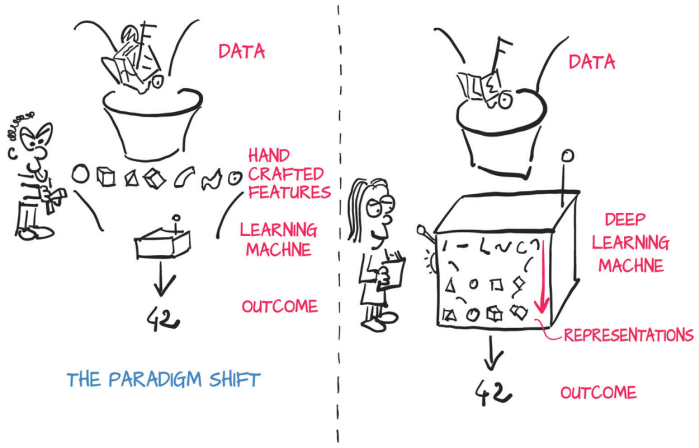


Figura 3.43: Imagen que esquematiza el cambio de paradigma en el Aprendizaje Automático: a la izquierda un experto del dominio se ocuapaba de programar manualmente la extracción de características. A la derecha, con el paradigma del Aprendizaje Profundo, puede focalizarse en que el modelo entrene adecuadamente para aprendarlas. Fuente: [Stevens and Antiga 2019]

El hecho de contar con Redes Neuronales de muchas capas posibilita que las primeras aprendan representaciones más básicas, sobre las cuáles se basen las representaciones más refinadas de las capas subsiguientes. Posibilitando un paradigma que consiste en suministrarle los datos crudos al modelo (por ejemplo los píxeles de la imágen) y que no sea necesario conocimiento específico del dominio para realizar la extracción de características manualmente. Haciendo así mucho más fácilmente trasladable los métodos entre diferentes dominios de aplicación.

Éste ha sido la clave del éxito del Aprendizaje Profundo y lo que ha popularizado a las Redes Neuronales. Y fue posible gracias a que se empezó a experimentar con más de una sóla capa oculta.

Los siguientes factores han sido clave en el desarrollo de las Redes Neuronales profundas:

- El aumento en la capacidad de cómputo (sobretodo el uso de GPU).
- Atreverse a experimentar con más de una capa oculta (pese a las malas interpretaciones del Teorema de Aproximación Universal).
- El surgimiento de arquitecturas con conexiones más dispersas entre capas sucesivas: las Redes Convolucionales (que se verán en la siguiente sección), que posibilitó agregar más capas con menor costo computacional.

3.3.13. Redes Neuronales Convolucionales

Nuestro trabajo se enfocará en **Redes Neuronales Convolucionales** (*CNN* del inglés *Convolutional Neural Network*), arquitectura que posibilitaron modelos muy profundos de Redes Neuronales, constituyendo el primer gran éxito del Aprendizaje Profundo [LeCun et al. 1998].

Se basan en dos observaciones claves, particularmente válidas para problemas de análisis de imágenes:

- Las características son *locales*. La salida de cada capa puede ser construída teniendo en cuenta solamente un área acotada de la capa anterior.
- Las características extraídas poseen la restricción de *invariancia espacial* ([LeCun 1989]): cuestiones tales como detección de bordes o texturas tienen sentido re-aplicarlas en toda la imágen. Luego, los pesos que se aplican a una zona de la imágen puede ser compartidos en otras zonas.

Estas restricciones permiten disminuir la conectividad y compartir pesos, reduciendo el número de parámetros y de computaciones realizadas en cada capa. Así, se vuelve manejable computacionalmente trabajar con muchas capas.

Además se reduce la complejidad del modelo, reduciendo el espacio de hipótesis bajo asumpciones razonables, lo que logra una mejor generalización [Denker and Schwartz 1987] [Patarnello and Carnevali 1987].

Así, en redes convolucionales[6], las activaciones serán matrices de dimensiones (W, H, C): el ancho, la altura y la cantidad de canales respectivamente. Como capa de entrada se suministrará los pixeles de la imagen (la cantidad de canales en esta capa serán los canales de color RGB). No trabajamos con un simple vector de características, ya que nos sirve guardar el concepto de localidad (entre pixeles y las características cercanos entre sí) en la imágen original.

3.3.13.1. Convoluciones

La **convolución** es el tipo de conexión restringida entre activaciones sucesivas. Consiste simplemente en una combinación lineal de las características en una sección rectangular (w, h) de la capa anterior (también se suma un parámetro de *bias* b). Ver figuras 3.44 y 3.45.



Figura 3.44: Ejemplo de filtro convolucional $(3, 3)$. Fuente: [Dumoulin and Visin 2018]

Los valores de ponderación que determinan cómo se combinarán linealmente las entradas para formar las salidas son compartidos por cada salida con diferente (W, H) (según [LeCun 1989]). La matriz (w, h) con dichos valores se denomina ***kernel***.

De tener múltiples canales de entrada $(C > 1)$, tendremos un filtro convolucional o *kernel* para cada uno. A su vez, cada canal de salida tiene un conjunto de filtros y *bias* independiente.

Las convoluciones ya se aplicaban en el procesamiento clásico de imágenes y eran llamadas también ***filtros***. Se usaban valores de *kernels* que se sabían útiles para la extracción de ciertas características (como detección de bordes horizontales, diagonales o verticales).

En el caso de las CNN, los *filtros* serán entrenados, pretendiendo que se aprendan los valores que le resulten útiles. Además, la aplicación sucesiva de capas convolucionales apunta a que la componibilidad de los filtros sucesivos genere que la complejidad semántica de las características que se extraiga vaya en aumento con la profundidad.

3.3.13.2. Aritmética de las convoluciones

Para cada canal de la capa resultado se suman convoluciones distintas (con parámetros de *kernel* distintos) de cada uno de los canales de la capa de entrada.

Si tenemos una capa de entrada de tamaño (W, H, c_{in}) y le aplicamos un filtro de dimensión (c_{out}, c_{in}, w, h) (c_{in} canales de entrada, c_{out} canales de salida y tamaño de kernel $w \times h$), se obtendrá una capa de salida de dimensión $(W - w + 1, H - h + 1, c_{out})$[7].

Recordar que, además, tenemos un término de *bias* por canal de salida (c_{out} *biases*, en este caso).

En el caso de las convoluciones, el número de parámetros de una capa sólo depende del tamaño del filtro: $(c_{out} \times c_{in} \times w \times h)$ y el del *bias*: c_{out}. A su vez, el número de computaciones sólo depende del producto entre el tamaño del filtro y el tamaño de la entrada. En el caso *fully-connected*, tanto la cantidad de parámetros como el número de computaciones era del orden del producto de los tamaños de las activaciones sucesivas (debido a la matriz W).

[6] En el trabajo nos referiremos siempre a redes convolucionales 2-dimensionales, que son las que aplican al análisis de imágenes. Los conceptos explicados son extrapolables a entradas de más o menos dimensiones (por ejemplo existen aplicaciones de 4D CNN para videos o 1D CNN para series temporales).

[7] Esto es considerando el caso de cero de *padding* y uno de *stride*. Conceptos que se explican en la subsección siguiente.

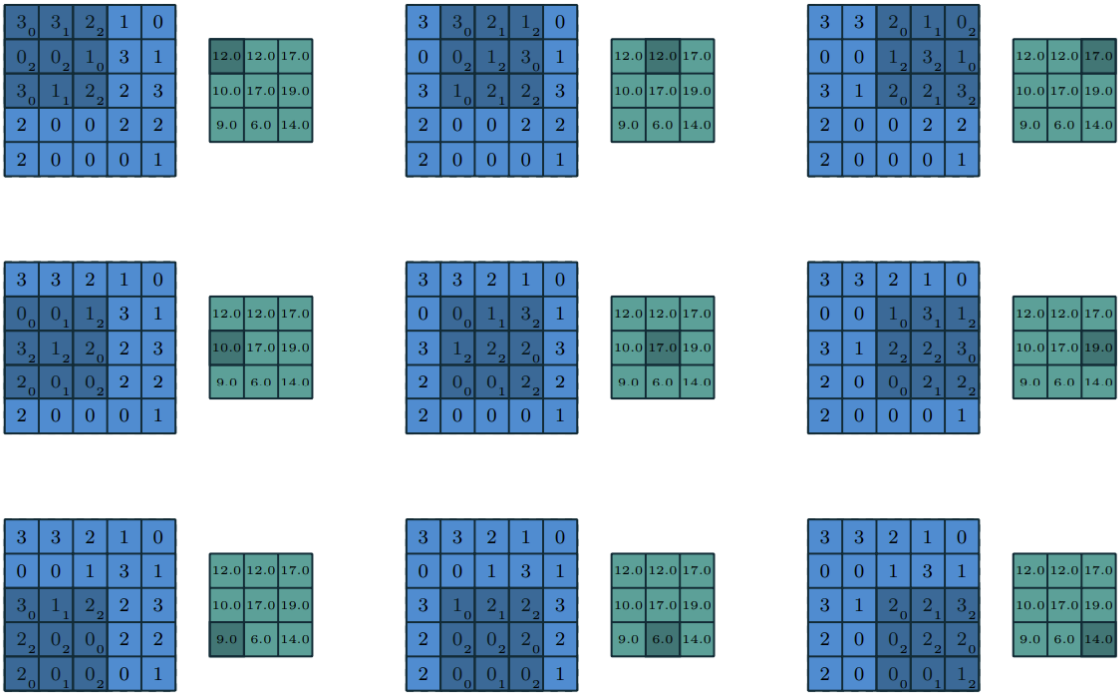


Figura 3.45: El filtro de la imágen 3.44 es aplicado a una matriz de entrada $(5, 5)$ (color azul), produciendo una salida de $(3, 3)$ (color aguamarina). Se han oscurecido respectivamente la zona rectangular de la matriz de entrada que interviene en la operación en cada paso, y la posición de la matriz de salida que constituye el resultado. Por razones de simplicidad hemos obviado el *bias* b. Fuente: [Dumoulin and Visin 2018].

Esta reducción en el número de parámetros y el número de computaciones de cada capa permite entrenar modelos de muchas más capas.

Recordemos la fórmula para las capas densas (o *fully-connected*) vista en la subsección 3.3.2:

$$f^j(\bar{h}^{j-1}) = \phi^j({W^j}^\top \bar{h}^{j-1} + b^j)$$

La convolución es un caso particular de multiplicación de matrices, con restricciones sobre W.[8]

En la imagen 3.46.a se desgloza una convolución sobre una entrada de dimensión $(3, 3)$ sobre la que se aplica un *kernel* de $(2, 2)$ produciendo una salida de $(2, 2)$. Las combinaciones lineales dan lugar a las ecuaciones que se muestran en la figura 3.46.b. En la figura 3.47 se muestra dicha operación como multiplicación de matrices. Notar que la matriz de la transformación (respectiva a W de la fórmula vista anteriormente) posee ceros en determinadas entradas y los valores de ciertas entradas son compartidos, pueden variar según el entrenamiento pero deben ser iguales (ésto es denominado en la jerga del Aprendizaje Profundo como *shrared weights* o *tied weights*).

3.3.13.2.1. *Padding* y *Stride*

El *padding* y el *stride* son hiperparámetros que configuran la convolución. Con la fórmula vista hasta ahora, el tamaño de la capa de salida está determinado por el de la entrada y el del *kernel*. El propósito

[8] Este hecho no debe sorprendernos ya que la convolución es una transformación lineal. Y las transformaciones lineales pueden ser expresadas como multiplicación de matrices.



Figura 3.46: A la izquierda el desgloce de la aplicación de un filtro convolucional. A la derecha las ecuaciones lineales que se desprenden de dicha convolución. El filtro está notado con las letras α, β, γ y δ. La matriz de entrada corresponde a las letras de la A a la J. La matriz de salida corresponde a las letras P, Q, R y S. Fuente: *"CNNs from different viewpoints"*. Autor: Matthew Kleinsmith.



Figura 3.47: Las ecuaciones de 3.46.b vistas como multiplicación de matrices. Fuente: *"CNNs from different viewpoints"*. Autor: Matthew Kleinsmith.

del *padding* y el *stride* es tener más flexibilidad a la hora de incidir en el tamaño de la salida.

El ***padding*** consiste en agregar rellenos (generalmente 0) en los bordes de la capa de entrada. Se agrega *padding* para aumentar el tamaño de la salida.

Si pensamos a la aplicación de la convolución como un parche rectangular (el *kernel*) que se va moviendo sobre la entrada (como una ventana deslizante), el ***stride*** determina el tamaño del paso con el que avanza cada vez que se desplaza (ver figura 3.48). Se agrega *stride* para reducir el tamaño de la salida.

Tanto el *padding* como *stride* pueden ser diferentes en el ancho o alto (aunque generalmente siempre se trabaja con cuadrados). En [Dumoulin and Visin 2018] se muestra que si una dimensión de entrada posee tamaño i, y se aplica un *kernel* de tamaño k, p de *padding* y s de *stride*, el tamaño o de la salida obtenida es:

$$o = \left\lceil \frac{(i + 2p - k)}{s} \right\rceil + 1$$

3.3.13.2.2. *Pooling*

Dada una entrada de tamaño (W, H, C) las capas de *pooling* la reducen a una capa de tamaño (W', H', C) (con $W' < W$, $H' < H$), tomando el máximo (*max-pooling*) o calculando el promedio (*average-pooling*) de regiones rectangulares.

La reducción trabaja como una "ventana deslizante" que se va desplazando (al igual que el parche en la convolución) sobre la entrada. Ver figura 3.49.

Recomendamos [Dumoulin and Visin 2018] para profundizar en el entendimiento de las convoluciones. Allí se explica y se esquematizan estos conceptos con puntillosidad.

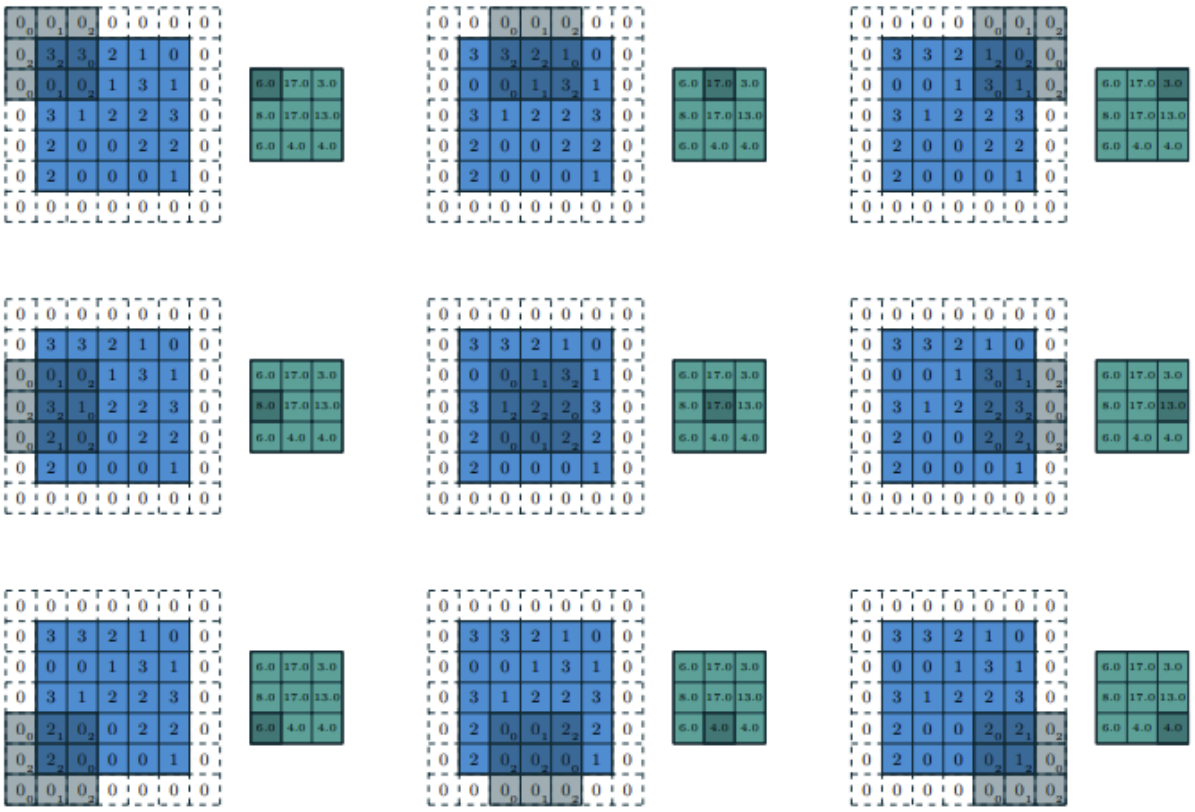


Figura 3.48: Ejemplo de aplicación de un *kernel* convolucional de tamaño $(3, 3)$ sobre una entrada de tamaño $(5, 5)$ aplicando *zero-padding* de $(1, 1)$ y *stride* de $(2, 2)$. Fuente: [Dumoulin and Visin 2018]

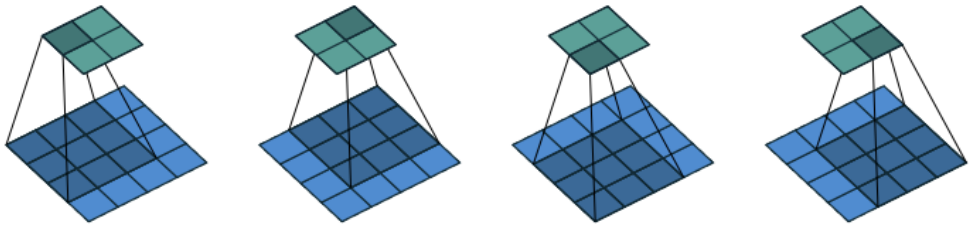


Figura 3.49: Las capas de *pooling* se calculan con la misma lógica de "ventana deslizante" de las convoluciones. Fuente: [Dumoulin and Visin 2018]

3.3.13.3. *Receptive field*

Se conoce como **área de recepción** (*receptive field*) al área de la imagen original que incide sobre el cálculo de cierta activación. En [Howard and Gugger 2020] se muestra el esquema presentado en la figura 3.50, donde se puede ver cómo a medida que aumenta la profundidad el *receptive field* es más grande. O sea que una gran porción de la imágen interviene en el cálculo de cada activación (característica) de las capas profundas. Es de esperar entonces que al aumentar la profundidad, será necesario aumentar la cantidad de filtros (canales), porque la sección de la entrada correspondiente que es representada en cada posición (w, h) de la activación, es de dimensionalidad mayor. Y por ende se necesitará, probablemente, más *features* para describirla.

Como se verá en el apartado 3.3.13.4, las activaciones de capas más profundas corresponden a características semánticamente más complejas. Es de esperar también, que sean necesarios más filtros en

las capas más profundas ya que el número de características semánticamente complejas probablemente sea mayor que el de características más simples.

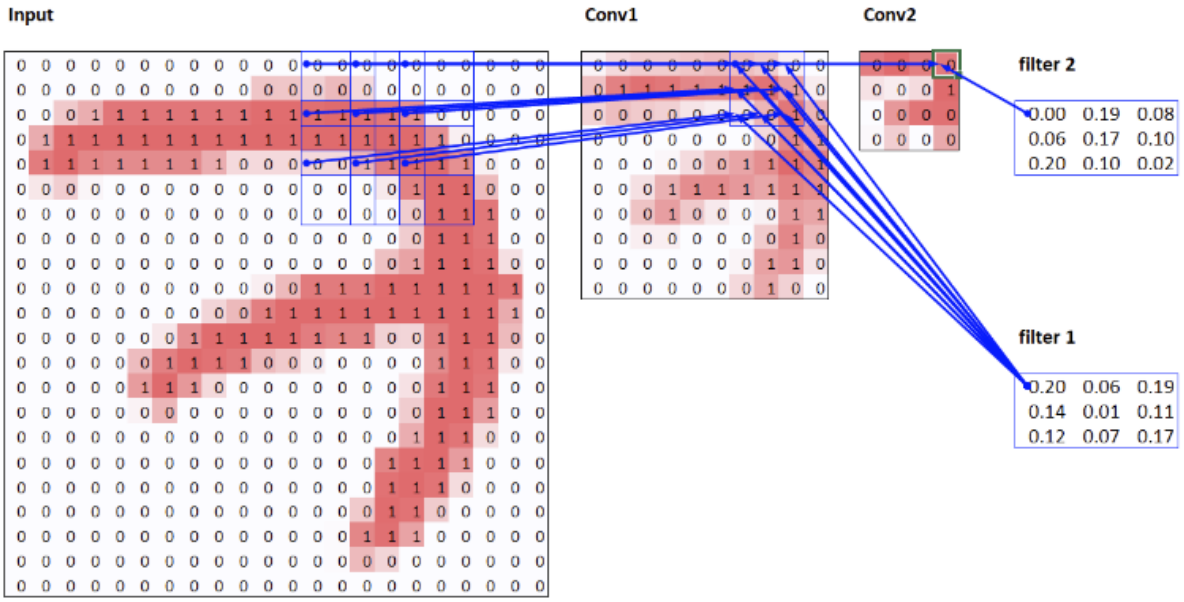


Figura 3.50: En [Howard and Gugger 2020] se implementan dos capas de convoluciones en planillas de cálculo, con el fin de visualizar como el procesamiento se aplica sobre la capa de entrada (*Input*) obteniendo sucesivamente las activaciones de cada convolución (*Conv1* y *Conv2*).
También se muestra la zona respectiva de la imágen para cada elemento de la salida (en azul, el *receptive field*) y su relación con los filtros.
Fuente [Howard and Gugger 2020].

A su vez en las primeras capas, el tamaño de la capa de entrada es grande, por lo que utilizar pocos filtros para ellas reducirá considerablemente el número de computaciones (que, como se dijo, es del orden del producto entre el tamaño de la capa de entrada y el tamaño del filtro).

3.3.13.4. Visualizando las características aprendidas

Las características aprendidas capa por capa fueron estudiadas por [Zeiler and Fergus 2014] para el modelo de cinco capas *AlexNet* que ganó la competencia de *ImageNet* 2012 [Hinton et al. 2012]. Se graficó para cada capa, la intensidad de los pesos de los filtros y las partes respectivas de la imágenes (*receptive fields*) para las cuáles los filtros mostraban una mayor activación (ver figura 3.51).

Se observa cómo las primeras capas aprenden a reconocer características básicas (diagonales, horizontales y verticales de ciertos colores), las capas intermedias características más complejas (espirales, curvas y diferentes texturas), hasta las últimas capas que aprenden a extraer conceptos semánticos de alto nivel (flores, perros, personas, vehículos, etc).

Muchas de las características aprendidas por el modelo son parecidas a las que se solían extraer manualmente por algoritmos anteriores de análisis de imágenes.

El trabajo nos muestra no solamente una forma de "visualizar qué aprenden los filtros" en las *CNN*, sino también prueba el hecho de que las representaciones aprendidas son patrones deseables, componibles y de creciente complejidad.

3.3.13.5. Redes completamente convolucionales

Una diferencia entre las capas convolucionales y las densas (*fully-connected*) está en cómo se relaciona el tamaño de la capa de entrada, el tamaño del *kernel* (los parámetros) y el tamaño de la capa de salida.

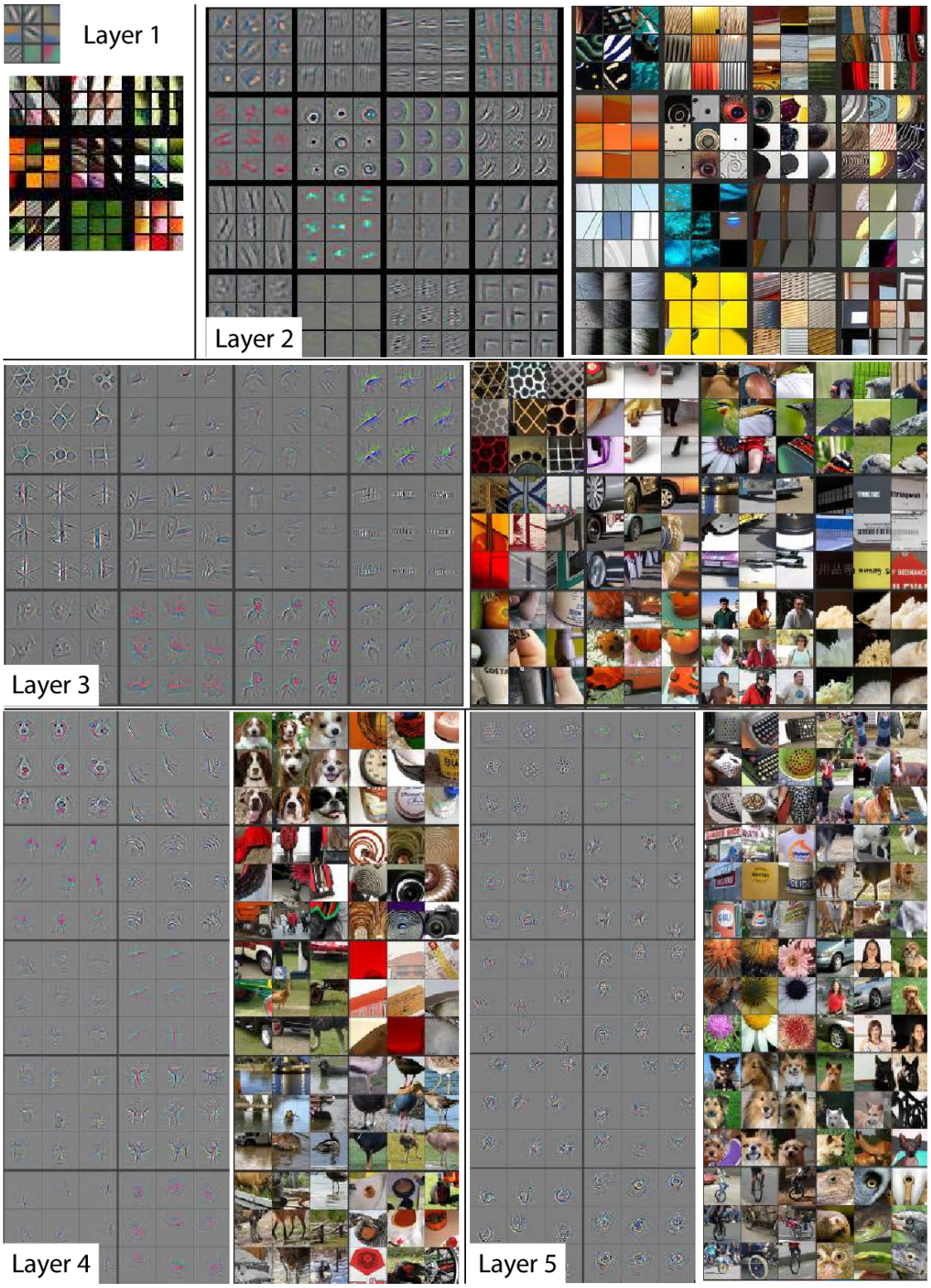


Figura 3.51: En gris, el valor de los filtros convolucionales. En colores, los parches de *receptive field* que más activaron a los filtros respectivos. Fuente: [Zeiler and Fergus 2014]

Para una convolución dada, el tamaño de la entrada no depende de la misma (puede aplicarse a diferentes tamaños de entrada). Pero fijada la convolución, el tamaño de la salida será función de la entrada.

En el caso de las capas densas, dada una capa (dado W y b), el tamaño de la entrada y la salida está completamente determinado: sólo puede aplicarse a entradas de un único tamaño, produciendo salidas de un tamaño también determinado.

A la hora de diseñar arquitecturas de Redes Neuronales siempre se debe pensar cómo se transforma la entrada (de cierto tamaño) en la salida (del tamaño de la predicción deseada).

Arquitecturas como *VGG* (ganadora de *ImageNet 2013* [Simonyan and Zisserman 2014]) apilan cierto número de convoluciones (como se dijo, con número de filtros creciente) para luego aplanar las características y aplicar una sucesión de capas densas sobre las mismas. Ésto hace que, si bien las convoluciones iniciales pueden ser aplicadas a imágenes de cualquier tamaño, la entrada de las capas densas deba ser (como se dijo) fijo. Y por ende el tamaño de la entrada original también deba ser fijo.

Una manera de evitar esta dependencia respecto del tamaño de la entrada (W, H, C), consiste en apilar tantas capas de *stride* (o *pooling*) como sean necesarias para reducir el tamaño de la salida convolucional a $(1, 1, C')$ y luego aplicar capas densas para obtener la predicción del tamaño deseado c_{out}. De este modo el tamaño de las capas densas sólo dependerá de C', y en última instancia del número de canales de la entrada C, pero será independiente de W y H. Pero nuevamente, esto es sólo posible si conocemos el tamaño de la entrada de antemano (de lo contrario no sabríamos *cuántas* capas convolucionales con *stride* aplicar).

La solución ha sido transformar la salida de la pila de convoluciones de tamaño (W', H', C') haciendo un *Pooling Adaptativo*,[9] lo cuál escencialmente acumulará para cada canal todos los valores de las activaciones (a lo alto y ancho de la última activación), de modo de obtener una salida de tamaño $(1, 1, C')$. Así, la arquitectura se mantiene totalmente desacoplada del tamaño de entrada, ya que la capa de *pooling* funciona para cualquier (W', H') porque es simplemente acumular los valores de cada canal usando cierta operación.

Éstas redes se denominan **Redes Completamente Convolucionales** (*FCN*, en inglés *Fully Convolutional Network*).

Luego la salida de dimensión C' puede ser llevado a la predicción c_{out} utilizando capas densas. Notar que C' no varía si se utilizan entradas de diferente ancho o alto, ya que los canales son independientes de W y H. Sólo depende de la cantidad de canales de entrada C.

La primera parte de la arquitectura (hasta la salida de dimensión C') es denominada cuerpo (***body***). Y la segunda parte (capas densas) es llamada cabeza (***head***).

Estas redes pueden ser entrenadas para imágenes de cierto tamaño y usadas en imágenes de otro tamaño (notar que la cantidad de canales de la entrada debe ser el mismo).

Nota: la última capa del *body* combina los filtros independientemente de su posición. Es importante que el dominio donde se aplique sea válido la hipótesis de *invariancia traslacional*.

3.3.13.6. *ResNets*

En 2015 los autores de [He et al. 2015] observaron, trabajando con las ya presentadas redes *VGG*, que al agregar más capas convolucionales obtenían peores valores del error y la causa no era el sobreajuste, puesto que lo mismo sucedía durante el entrenamiento (ver 3.52).

Tampoco se debía a que el cambio de arquitectura restringiera el espacio de hipótesis puesto que las capas innecesarias bien podrían aprender la identidad ($W = I$ y $b = \overline{0}$). Pero esto no sucedía: por alguna razón el algoritmo de optimización no encontraba esta solución.

Luego reemplazaron la capa de convolución:

$$y = conv(x)$$

Por la siguiente formulación:

[9] Éstos, en vez de poseer un tamaño de ventana fijo, tienen un tamaño de salida fijo y utilizan para ello la ventana necesaria.

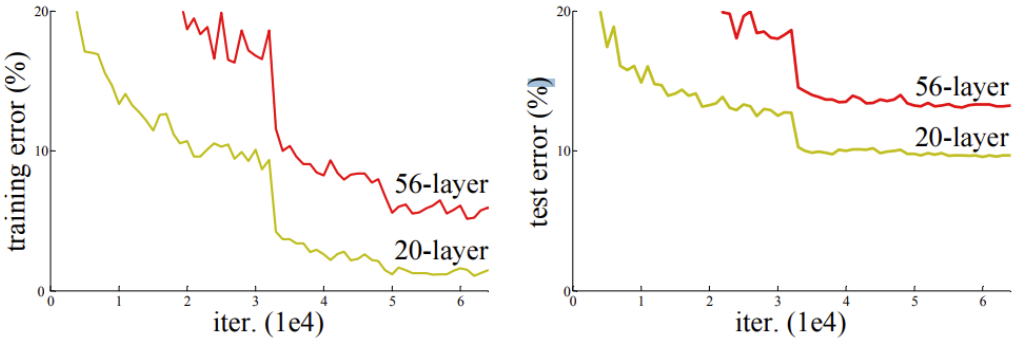


Figura 3.52: Observación clave que condujo al desarrollo de redes residuales. Fuente: [He et al. 2015]

$$y = conv(x) + x$$

Además, inicializaron el parámetro γ (de *batch normalization*) en 0. De modo que el entrenamiento podía variarlo pero inicialmente la capa era en sí la identidad. Ésta idea dio lugar al bloque que denominaron *skip-connection* (ver esquema 3.53).

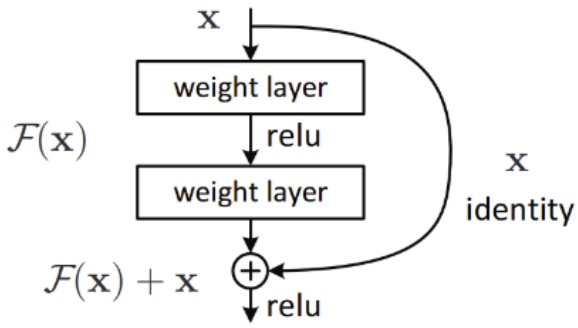


Figura 3.53: El bloque usado en [He et al. 2015] usa dos convoluciones sucesivas. La idea básica es la misma que se ha explicado: las convoluciones ahora deben aprender el *residuo* de la función objetivo (menos la identidad). De modo que aprender la identidad resulta trivial. Fuente: [He et al. 2015]

En este bloque se basa la familia de *Residual Neural Network* (*ResNets*) ganadoras de la competencia ImageNet 2015 (*error rate* 3,57 %), acabando con la dominancia de las *VGG*. [He et al. 2015]

El estudio posterior ha constatado la superioridad de estas arquitecturas. En [Hao Li et al. 2018] se muestra que el uso de *skip-connections* genera funciones de costo más suaves, con menos mínimos empinados (ver figura 3.54) y por ende contribuyen a una mejor generalización.

En [He et al. 2018] se estudian algunos ajustes que perfeccionan la arquitectura de las *ResNets*. En el trabajo se utilizan estos ajustes (incorporados a la librería FastAI). Por ejemplo se utilizan simples convoluciones en las primeras capas para mantenerlas más simples (ya que como se dijo son computacionalmente de mayor costo porque aplican a una entrada más grande). Y se reemplazan las *skip-connections* por *Bottlneck Layers* (ver esquema 3.55) que son más rápidos permitiendo agregar más filtros.

3.3.13.7. *Fine-tuning*

El hecho de que las características aprendidas sean patrones deseables y componibles (como se vio en el apartado 3.3.13.4) da apoyo teórico a que puedan ser reutilizadas, entre problemas similares.

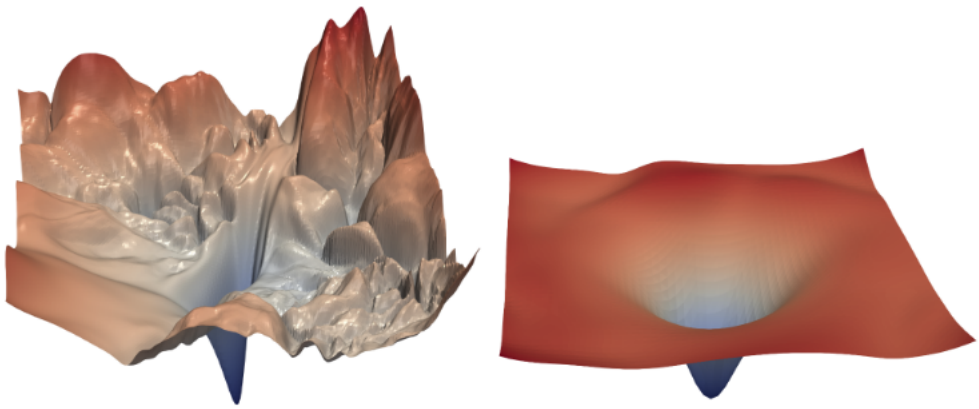


Figura 3.54: Por medio de métodos de visualización lograron una representación bidimensional de la función de costo. A la derecha se ve como resulta la superficie de costo al utilizar *skip-connections*
Fuente: [Hao Li et al. 2018]

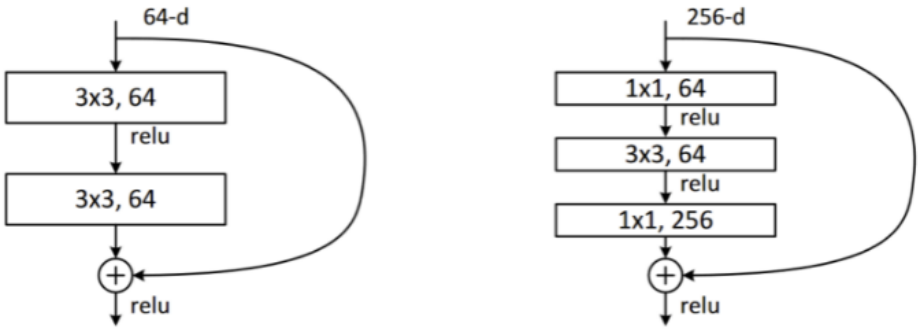


Figura 3.55: [He et al. 2018] reemplaza las *skip-connections* (izquierda) por *Bottlneck Layers* (derecha)
Fuente: [He et al. 2018]

Se conoce como ***fine-tunning*** a la técnica de transferencia del aprendizaje (*transfer learning*) donde los parámetros son entrenados partiendo de la base de un modelo que ya ha sido entrenado para un conjunto de datos diferente (e incluso para un problema diferente) al que se quiere resolver.

Así, por ejemplo, si se quiere hacer un clasificador de fotografías de tipos de automóviles, se pueden obtener resultados con precisión del estado del arte aún sin contar con demasiados datos, y entrenando unas pocas épocas, si re-entrenamos un modelo ya entrando sobre el *dataset ImageNet* (por ejemplo podríamos usar *ResNet-50*). La técnica incluso sirve para problemas diferentes a la clasificación (por más que el problema original de *ImageNet* es entrenar un clasificador), ya que lo transferible son las de características extraídas en las capas intermedias. Por ejemplo se pueden entrenar modelos para problemas de regresión, *tagging* multi-clase, etc.

En general cuando se aplica *fine-tunning* se conserva la parte de la arquitectura que hemos denominado *body* y se coloca un nuevo *head*. Ésto es debido a que el *head* es problema-específico, ya que depende del tamaño de la salida c_{out}. Además, las características codificadas por las últimas capas tendrán que ver con las cuestiones específicas del problema y lo que se quiere transferir son las representaciones que son trasladables entre los dos problemas.

La librería *FastAI* posee la funcionalidad de fácilmente congelar los pesos del *body*, de modo de entrenar unas épocas modificando el *head* solamente (ya que al ser inicializado de cero, contendrá pesos aleatorios). Esto tiene sentido ya que no queremos que la señal de *backpropagation* de un *head* totalmente

aleatorio arruine las representaciones útiles que intentamos transferir. Además el congelamiento de las primeras capas reduce el problema de *internal covariance shift*, ya que sólo varía el *head* y no las capas anteriores (su entrada). Hay quienes incluso pre-computan las activaciones de salida del *body*, ya que no variarán durante esta fase, para no volverlas a calcular. Luego podemos descongelar los parámetros del *body* y entrenar todo el modelo en su conjunto. [Howard and Gugger 2020]

En línea con el argumento del párrafo anterior [Yosinski et al. 2014][10] propone asignar *learning rates* diferenciados durante el proceso de *fine tunning* de modo que las primeras capas (más cercanas a la entrada) no varíen tanto como las últimas. Ésto está en consonancia con el hecho de que las representaciones de las primeras capas son conceptos más simples y serán más extrapolables, en cambio las capas posteriores (incluso del *body*) son más complejos y tendrán que re-adaptarse al problema específico.

3.3.14. Arquitecturas de redes siamesas

Como se discutió en el capítulo 5, muchos problemas de similitud de imágenes (por ejemplo *Face Verification* o similitud de firmas) no se resuelven entrenando un clasificador con una clase por identidad, sino aprendiendo una *representación* y una *métrica* que permitan comparar pares de ejemplos. Las **redes siamesas** constituyen una arquitectura estándar para este tipo de problemas [Face Verification LeCun 2005]. La idea central es utilizar un mismo *encoder* convolucional G_W para mapear cada imagen *img* a un vector de características de dimensión fija (un *embedding*):

$$z = G_W(img) \in \mathbb{R}^d,$$

compartiendo los pesos W entre ambas ramas de la red (lo que en la jerga se conoce como *tied weights*).

A partir de estos *embeddings* se define una función de distancia $d(z_1, z_2)$ (por ejemplo la distancia euclídea o la distancia coseno) y se entrena el modelo para que firmas (o imágenes) de la misma clase queden "cerca" en el espacio de representación, y las de clases distintas "lejos". Desde esta perspectiva, el clasificador binario que decide si dos firmas son de la misma autoría o no pasa a ser un umbral sobre la distancia:

$$autoresNoCoinciden(img_1, img_2) = \begin{cases} \text{Verdadero} & \text{si } d(G_W(img_1), G_W(img_2)) > \tau \\ \text{Falso} & \text{si } d(G_W(img_1), G_W(img_2)) \leq \tau \end{cases}$$

para algún umbral τ elegido según los criterios de costo–beneficio discutidos en 3.1.8.5. En nuestro caso particular, este esquema permite aprovechar el mismo *encoder* convolucional para firmar autores nunca vistos durante el entrenamiento, algo que sería muy difícil de lograr con un clasificador "uno contra todos" por autor.

3.3.14.1. Triplet loss

Una forma efectiva de entrenar redes siamesas es mediante funciones de costo diseñadas específicamente para *aprendizaje de métricas*. Entre ellas, una de las más utilizadas es la ***triplet loss***. En lugar de trabajar con pares (img_1, img_2), se consideran tripletas (a, p, n):

- a (*anchor*): una firma de referencia.
- p (*positive*): una firma del mismo autor que a.
- n (*negative*): una firma de un autor distinto.

Sea $z_a = G_W(a)$, $z_p = G_W(p)$ y $z_n = G_W(n)$ sus *embeddings*. La triplet loss impone que la distancia entre el ancla y el positivo sea menor, al menos por un margen $\alpha > 0$, que la distancia entre el ancla y el negativo:

$$L(a, p, n) = \text{máx}\left(0,\ d(z_a, z_p) - d(z_a, z_n) + \alpha\right).$$

[10] Jason Yosinski et al. desarrolló este método originalmente para el problema de NLP ULMFiT. Cómo muchos conceptos de Aprendizaje Profundo, es extrapolable a otras áreas.

Intuitivamente, el costo es cero siempre que z_a esté "más cerca"de z_p que de z_n por al menos α, y positivo en caso contrario. El entrenamiento busca minimizar la suma (o el promedio) de $L(a, p, n)$ sobre un conjunto de tripletas construidas a partir del *dataset*.

Cabe notar que esta formulación resulta menos restrictiva que otras funciones de costo que fuerzan a los ejemplos de una misma clase a colapsar en un único punto del espacio de representación. La triplet loss únicamente exige que los ejemplos positivos estén más cerca del ancla que los negativos, permitiendo así que las muestras de un mismo autor mantengan cierta dispersión entre sí. Esta flexibilidad es deseable en dominios como el de firmas manuscritas, donde existe variabilidad intra-clase natural y no resulta razonable pretender que todas las instancias de un autor converjan a una representación idéntica.

En la práctica es crucial la estrategia con la que se generan dichas tripletas: si se eligen tripletas "fáciles" (para las cuales la desigualdad ya se cumple ampliamente), el gradiente será casi nulo y el modelo aprenderá poco; por el contrario, seleccionar tripletas demasiado difíciles puede llevar a inestabilidades numéricas o a un entrenamiento que no converge. Existen esquemas *offline* (las tripletas se pre–generan) y *online* (se seleccionan dinámicamente tripletas informativas dentro de cada *mini-batch*), que se describen con más detalle en el capítulo de Experimentos. En nuestro trabajo utilizamos variantes de *triplet loss* precisamente para forzar que firmas de la misma autoría colapsen en regiones compactas del espacio de representación y que firmas de diferentes autores queden claramente separadas, lo que se traduce luego en mejores métricas de *precision* y *recall* en el problema binario definido en 4.3.

Capítulo 4

Formulación del problema

AGREGAR LA FALTA DE FIRMAS!!

En el presente capítulo se formula la problemática a resolver y se eligen las métricas de satisfactibilidad.

En el capítulo 5 hacemos un sondeo de soluciones a problemas similares e introducimos los conjuntos de datos que luego utilizaremos en nuestros experimentos.

La metodología para llegar a los resultados y el detalle de los mismos se encuentran en el capítulo 6.

4.1. Preliminares

El trabajo se enmarca en el proyecto de incorporación de dispositivos tecnológicos que asistan el proceso electoral impulsado por el Gobierno de Santa Fe a través de las Secretarías de Tecnologías para la Gestión. Y más especificamente, del sistema de escrutinio provisorio con asistencia digital[1].

Actualmente el Sistema de Escaneo y Transmisión de Telegramas (*SETT*) y Sistema de Asistencia a las Autoridades de Mesa (*SIAAM*) proveen soluciones robustas de asistencia digital al proceso de escrutinio provisorio demostrando (véase [Informe 2019 UNR GSF]):

1. Aumentar el cubrimiento de mesas en el escrutinio provisional.
2. Disminuir el porcentaje de votos "a determinar en el escrutinio definitivo".
3. Aumentar considerablemente la celeridad en la carga de telegramas.
4. Disminuir las solicitudes de re-confección de telegramas.

4.2. Sobre los telegramas de votación en Santa Fe

Hoy por hoy se detecta cuando el telegrama está en el centro de cómputo. Si el sistema de escaneo contara con un detector de firmas y similaridad se podrían reconocer antes y sería probable que se arreglar.

En las elecciones provinciales de Santa Fe, cada mesa de votación realiza un recuento de las actas emitidas en las mismas y se confeccionan telegramas con un formato específico con los resultados. Los mismos consisten en una tabla con la cantidad de votos para cada partido y cargo a elegir. Se puede ver un ejemplo en la figura 4.2.

El telegrama debe ser firmado por las autoridades (presidente y auxiliares) y los fiscales de la mesa respectiva. De poseer varias páginas la firma debe estar en todas y coincidir. En la figura 4.1 se puede ver el recorte de la sección de firmas, de dos páginas distintas que pertenecen a un mismo telegrama.

Luego los telegramas son transmitidos al centro de cómputo del escrutinio provisorio. Operadores humanos llevan la labor de carga de datos, realizándose chequeos de consistencia. Si se detectan errores

[1] En [UNR GSF] podemos encontrar una detallada descripción del Proceso de Escrutinio Provisorio Asistido Digitalmente. En [UNR GSF QRs] se encuentran las especificaciones técnicas de los *QRs* utilizados para la verificación de consistencia de los datos registrados en los telegramas.

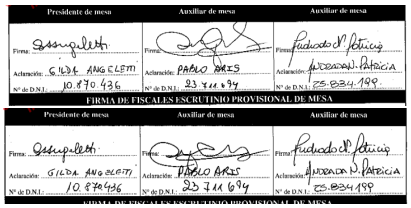


Figura 4.1: Recorte de la sección de firmas de dos páginas de un mismo telegrama. Para ver un telegrama completo ir al a figura 4.2. En las secciones donde aparecen firmas también aparecen otros trazos del formulario como la línea de puntos. Fuente:

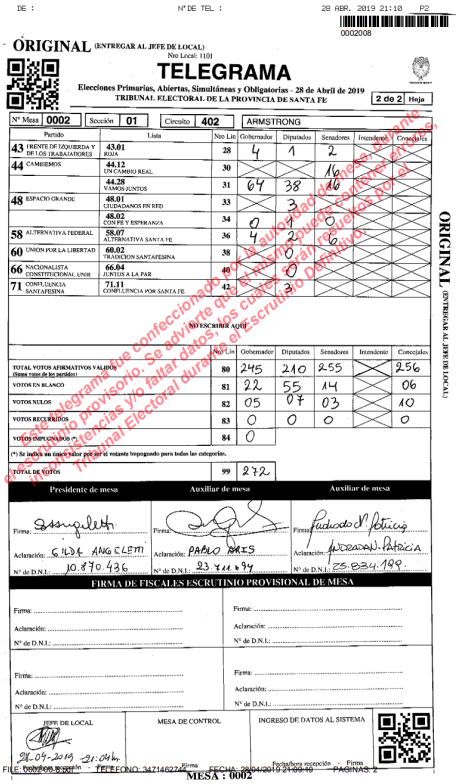


Figura 4.2: Ejemplo de un telegrama electoral de las elecciones primarias de 2019. El formato es conocido meses antes de las elecciones. Los códigos QR de las esquinas sirven para enderezar las fotografías de los telegramas. Fuente: [UNR GSF]

en esta fase puede pedírsele a las autoridades de la mesa que realicen nuevamente la confección del telegrama y/o el recuento según sea necesario. Nos referiremos a estos eventos como **retransmisiones**. Las autoridades de mesa deben esperar la confirmación del centro de cómputo de que el telegrama ha sido chequeado y cargado para considerar su labor finalizada.

Según surge de las estadísticas del informe [Informe 2019 UNR GSF] (sección 7.3), de poderse automatizar los procesos de:

1. Detectar que firmas de las autoridades de mesa estén presentes y coincidan en todas las páginas de un mismo telegrama.

2. Detectar los números en grillas de telegramas de votación (y hacer chequeos de consistencia, como por ejemplo verificación de totales).

Se podrían introducir chequeos automáticos antes de que el telegrama sea enviado al centro de cómputo logrando:

- Aumentar la celeridad en la carga de telegramas en el centro de cómputo.
- Disminuir las solicitudes de reenvío de telegramas.

En definitiva, disminuir en gran medida el trabajo humano.

Nos enfoquermoes en el primer problema: reconocer firmas y constatar coincidencia entre ellas.

4.3. Definiendo los objetivos

Como veremos en el capítulo 5, existen variedad de problemas similares relativos a firmas manuscritas.

Como el diseño de los telegramas se conoce de antemano, y las firmas que se quieren analizar aparecen en posiciones determinadas de la página, no es necesario resolver el problema general de *Signature Recognition*. Nos bastará recortar dicha sección de la página y determinar si hay o no una firma. Esto, hasta hace muy poco tiempo atrás consistía en un problema para nada trivial, sobretodo puesto que en la sección aparecen otros trazos como partes del formulario. No obstante, con el estado actual de los analizadores de imágenes hemos podido resolverlo sin demasiado ajetreo y hemos extendido y focalizado el trabajo de la tesina en el problema -mucho más desafiante- de similitud de firmas.

El problema de determinar si dos firmas son el mismo *tipo de firmas* (fueron realizadas por el mismo autor) puede expresarse más formalmente como el predicado[2]:

$$autoresCoinciden(firmaA : Imagen, firmaB : Imagen) \rightarrow Bool$$

Es así que el problema puede ser tratado como un problema de clasificación binaria donde las imágenes de ambas firmas constituyen la entrada.

Además, como se detalló en el apartado 3.1.8.3, al querer detectar *no coincidencias* (ya que las mismas constituyen un error en la confección del telegrama analizado), trateremos a las mismas como clase positiva. Así nuestro clasificador tratará de computar la siguiente función:

$$autoresNoCoinciden(firmaA : Imagen, firmaB : Imagen) \rightarrow Bool$$

4.3.1. Modelado del problema

A diferencia del problema de *signature verification* donde hay que determinar si una firma pertenece al autor que declara ser (conociendo previamente ciertas firmas para cada autor), en nuestro problema es posible (y probable) que las firmas sobre las que aplicaremos nuestra solución sean de autores completamente distintos a los que usamos para entrenar. Es por ello que no podemos entrenar un clasificador binario para cada autor como se hace en algunos trabajos relacionados.

Nuestra entrada serán pares de firmas y nuestra salida un booleano que indicará si es clasificado como coincidencia o no. Formalmente, una solución al problema consiste en un par de funciones:

- $fit(firmas, clases)$: entrena la solución utilizando un conjunto de imágenes para las cuales se conocen sus respectivos autores.
- $predict(img_1, img_2) \rightarrow Bool$: determina si dos firmas son una coincidencia o no.

4.3.2. Definición de los experimentos

Cada experimento consiste en una 3-upla $(dev, test, test_ds)$:

[2]En nuestro caso contamos con imágenes de las firmas manuscritas, pero existen otros ámbitos donde se cuenta también con información dinámica (temporal) acerca de cómo fue generado el trazo. También existen conjuntos de datos para este otro problema.

- **dev** y **test**: son duplas $(firmas, clases)$ con paths hacia imágenes y sus clases (autores) respectivamente. Los elementos de *dev* se utilizan para entrenar.
- **test_ds**: consiste en 3-uplas $(idx_1, idx_2, bool)$ donde el booleano determina si $test[idx_1]$ y $test[idx_2]$ constituyen un matcheo o no. Es importante usar las mismas 3-uplas para evaluar cada solución de modo que la comparación sea correcta.

4.3.2.1. Desarrollo: entrenamiento y validación

Del conjunto de firmas dedicadas al desarrollo (*dev*) se obtienen tanto los subconjuntos de entrenamiento (para encontrar los parámetros del modelo) como de validación (para encontrar los hiperparámetros).

La cantidad de clases (autores) distintas en el conjunto de desarrollo puede ser escasa. En dichos casos las métricas en validación pueden variar demasiado dependiendo de la elección de partición entrenamiento/validación, lo cual hace sensible a la elección de hiperparámetros. Es por eso que utilizamos un **5-fold cross validation** sobre el conjunto de desarrollo para generar 5 particiones, de modo que la elección de hiperparámetros sea más robusta.

Es importante, si vamos a comparar el score de validación de varios modelos, fijar también los emparejamientos de elementos (recordemos que nuestra solución toma pares) para los subconjuntos de validación.

4.3.2.2. Construcción del conjunto de pares

Al armar las parejas en los conjuntos de test y validación se construyen siempre conjuntos balanceados: hay tantas coincidencias como no coincidencias. Si se generaran todas las parejas posibles el conjunto contendría muchas más no coincidencias que coincidencias.

Para generar coincidencias, para cada clase se arman parejas sin repetirlas (sin considerar orden). Un parámetro *cls_sampling_limit* limita la cantidad de veces que un elemento aparece en las parejas auténticas. Para generar pares no coincidentes se toma para cada elemento otros elementos no coincidentes (tantos como coincidentes) dejando el conjunto balanceado.

4.3.3. Elección de la métrica

El objetivo de los modelos es realizar una predicción lo más acertada posible. En la práctica, las predicciones se usan para tomar decisiones. Es así que los diferentes tipos de error no son equiparables. Como se vio en 3.1.8.3 en el caso de la clasificación binaria tenemos *falsos negativos* y *falsos positivos*.

En nuestro caso, al aceptar pares de firmas que no coinciden (falsos negativos), el automatismo no cumplirá su objetivo de disminuir las retransmisiones y el trabajo humano innecesario. En caso de falsos positivos, las alarmas emitidas por el asistente serán incorrectas corriendo el riesgo de que el usuario desestime su opinión y que lejos de aumentar la eficiencia del proceso entorpezca la labor.

Es por esto que es importante no basarse en la tasa de aciertos (*accuracy*), sino medir ambos errores de forma separada. Esto también es necesario cuando se trabaja con conjuntos de datos desbalanceados.

Para nuestro trabajo tomaremos como métrica a maximizar: "la mayor *precision* para la cuál el *recall* sea mayor que 0.9". Así, nuestra métrica tendrá un significado práctico: queremos que nuestro sistema capture el 90 % de los errores de firmas no coincidentes (reduciendo al 10 % el número de retransmisiones por esta causa) y que sea lo más fiable posible (en el sentido de que no genere falsas alarmas).

Habiendo calculado esta métrica, podría ser útil dar a conocer al usuario el umbral de *precision*. Esto de alguna manera le indicará el "grado de garantía" que poseen las alarmas. Por ejemplo: si la *precision* obtenida es del 0.8, nos dice que tan sólo alrededor de 1 de cada 5 alarmas son falsas.

4.4. Conjuntos de datos utilizados

4.4.1. Experimento 1

Para este experimento se utilizaron los datasets NISDCC y NFI (usados en SigComp 2009). NFI consta de 80 autores y NISDCC de 12 autores. Presumiblemente las diferencias entre ambos datasets son mayores que las diferencias internas de cada uno (en cuanto al tipo de trazo, calidad de las firmas, metodología de construcción).

Para el conjunto de test se deseó tener un número razonable de autores (al menos 30) y a la vez utilizar uno de los datasets exclusivamente para test:

- 60 autores de NFI como conjunto de desarrollo.
- NISDCC + los 20 autores restantes de NFI como conjunto de test.

4.4.2. Experimento 2

Para este experimento se agregó a los datasets NISDCC y NFI el dataset de firmas CEDAR, aumentando significativamente el número de autores y firmas disponibles para desarrollo y evaluación.

Capítulo 5

Trabajos relacionados y estado del arte

Existen en la literatura científica diversos problemas relacionados al análisis de firmas manuscritas, entre los cuales se encuentran:

- Detección de firmas (*Signature Detection*): consiste en detectar las firmas presentes en una imágen junto a su posición. [N Sharma et al 2018]
- Segmentación de firmas (*Signature Segmentation*): consiste en determinar cuáles píxeles de una imágen pertenecen a una firma. [DNP98]
- Recuperación de firmas (*Signature Retrival*): consiste encontrar una firma similar a una dada en una base de datos de firmas. [S Shirdhonkar 2010] [Lin et al 2015]
- Verificación Automática de Frimas Manuscritas (*VAFM*): consiste en determinar si una firma pertenece o no a un autor (del cuál se conocen ciertas firmas). [NR77] [IP08] [IPP12] [Parodi 2014]
- Similitud de firmas (*Signature Similarity*): dadas dos firmas determinar su grado de similitud para establecer si *"son la misma firma"*. O sea, si fueron realizadas por la misma persona.

El problema que nos atañe es el de *Signature Similarity*: tenemos dos firmas y tenemos que determinar si son de misma autoría o no.

En las secciones siguientes relevaremos literatura que concierne a otros problemas que se ocupan del análisis de firmas. Hacemos foco en *VAFM*, problema del cuál hemos extraído varias técnicas, ya que ha sido muy estudiado por sus implicancias en el proceso de autenticación[1] mediante la firma. Además, es esperable que las características que capturen la "autenticidad" (o autoría) de una firma sirvan para comparar la autoría de dos firmas. En este análisis nuestro trabajo se basa fuertemente en el trabajo de [Parodi 2014].

Como nuestro problema es, de algún modo, un caso particula de la problemática de la similitud de imágenes, exploraremos soluciones que atañen a dicha área. Por ejemplo, mencionaremos soluciones al problema de Reconocimiento Facial (*Face Recognition*), siendo los rostros también una característica biómetrica utilizada para la identificación o verificación de identidad.

5.1. Verificación Automática de Firmas Manuscritas

Las firmas manuscritas son una característica biométrica no invasiva establecida desde hace mucho tiempo alrededor de todo el mundo como método de autenticación y con amplio reconocimiento legal

[1] El problema de *autenticación* consiste en verificar que un sujeto es quién declara ser.

por instituciones administrativas y financieras. Es fácil de recolectar, difícil de olvidar y falsificar, y casi imposible de adivinar[2].

A diferencia de otras características biómetricas (como la huella dactilar) presentan alta variabilidad de una instancia a otra. Esto se debe principalmente a que son características biométricas de comportamiento: dependen del estado físico, psicológico y emocional del autor. O sea son dependientes de la circunstancia. Por lo cual la variabilidad intrapersonal es alta, tornando el problema de verificación mucho más arduo que con otras características biométricas.

El problema de *Offline VAFM*[3], donde se trabaja con imágenes de las firmas.

En el contexto de *Online VAFM*, exploramos el trabajo de Graves [Graves 2013], que demuestra cómo las redes LSTM (*Long Short-Term Memory*) pueden modelar secuencias temporales de escritura manuscrita. El enfoque utiliza *Mixture Density Networks* para predecir las coordenadas del trazo punto a punto, capturando características estilísticas como inclinación, tamaño y cursividad. Aunque estas técnicas son prometedoras para verificación de firmas online (donde se dispone de la trayectoria temporal del trazo), decidimos no profundizar en esta línea dado que nuestros datos consisten en imágenes estáticas de telegramas escaneados (*offline*). Sin embargo, las ideas de codificación de estilo mediante estados ocultos de LSTM podrían aplicarse si se extrajera una pseudo-secuencia temporal mediante esqueletonización de la firma.

5.1.1. Categorías de firmas

En [Parodi 2014] se trabaja con:

- **Firmas auténticas:** firmas del autor que declara ser.
- **Falsificaciones simuladas:** firmas realizadas por un falsificador que posee acceso a una firma del autor que pretende suplantar y puede practicarla.
- **Falsificaciones ficticia (o aleatorias):** el falsificador no tiene acceso a una instancia de la firma auténtica que se trata de imitar, trata de inventar una.

La terminología para referirse a los distintos tipos de firmas auténticas y falsificaciones se puede encontrar con más detalle en [Parodi 2014], que a su vez se basa en [ML12], discusiones de la comunidad de *FHE* (*Forensic Handwriting Experts*) y *PR* (*Pattern Recognition*).

Nosotros no ahondaremos en dicha categorización ya que trabajaremos sólamente con firmas auténticas y dejaremos de lado las falsificaciones ya que en nuestro caso queremos detectar errores en la confección de los telegramas, no suplantaciones.

5.1.2. Proceso *clásico* de VAFM

En [Parodi 2014] se describe el esquema general que cualquier sistema de *VAFM clásico* suele poseer, con tres fases bien diferenciadas (ver figura 5.1):

- **Pre-procesamiento:** tiene como fin mejorar la calidad de los datos. Suele ser bastante estandarizado. Tipicamente, se aplican umbrales de decisión (binarización), filtros gaussianos y técnicas estándar de procesamiento de señales. A veces también se normaliza respecto a tamaño y orientación. [GW08] [GWE09]

  En [Parodi 2014] se binariza la imagen y normaliza el ancho de la misma, manteniendo la relación entre el ancho y la altura. En la figura 5.2 se muestran las distintas técnicas de preprocesamiento disponibles.

- **Extracción de características:** tiene como objetivo encontrar una descripción adecuada para la firma. La idea es extraer características que permitan distinguir una firma de otras del mismo autor y de falsificaciones. Con ellas se construye un vector característico que representará a la firma.

[2]Por adivinar nos referimos al hecho de predecir la firma de un autor desconociendo un ejemplo de la misma.

[3]En contraposición, existe el problema de *Online VAFM*, donde la dinámica del proceso de firmado se encuentra disponible: se cuenta con las coordenadas espaciales en función del tiempo y muchas veces también con otros datos como el ángulo y la presión de la lapicera.

- **Clasificación:** el vector de características alimenta a un clasificador que distingue si una firma es auténtica para una proclamada identidad.

  Para la clasificación [Parodi 2014] utiliza SVM siguiendo la línea de [JBS05], [OSK05] y [FSV06]. Al momento de dicho trabajo, era muy popular utilizar clasificadores basados en ANN [OSK05], [FSV06] y HMM [JYBS00] [JS01], [FAT05], [JBS05]. También existían antecedentes de uso de *Random Forest* [ASVE13], *AdaBoost* [HMM05], [HC13].

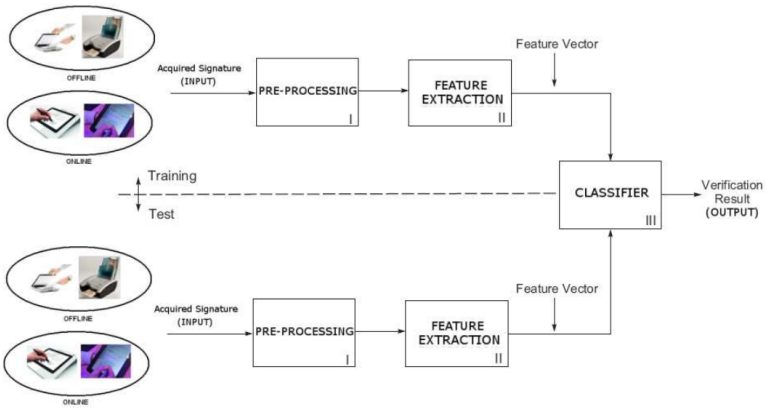


Figura 5.1: Esquema de un sistema de VAFM. Fuente: [Parodi 2014].



Figura 5.2: Distintas técnicas de preprocesamiento aplicadas a imágenes de firmas: binarización, filtros y normalización.

Notar que se ha hecho hincapié en que este es el esquema *clásico* de VAFM, entendiéndose por clásico como "previo al Aprendizaje Profundo". Como se explicó en la subsección 3.3.12, el Aprendizaje Profundo revolucionó el paradigma del Aprendizaje Automatizado: ahora se suelen suministrar los datos crudos y es el algoritmo mismo quién aprende las características necesarias para el proceso de clasificación final.

La parte más crítica (con más impacto en los resultados) y que concentra la mayoría del conocimiento específico del dominio es la extracción de características. Es por ello que el trabajo de [Parodi 2014] gira

fundamentalmente en torno a esta fase. En [IP08] e [IPP12] se propone gran variedad de características. La autora de [Parodi 2014] ha propuesto contribuciones concernientes a la *VAFM offline* en [PG10a], [PG10b] y [PGB11].

5.1.3. Extracción de características

El objetivo de esta fase, que como se dijo suele ser la más importante, es encontrar una descripción adecuada de la firma.

Generalmente las características extraídas se basan en la geometría, tales como área, altura, ancho [AYF90] [SPB94], contorno [NP94], orientación [SPB94], dirección [SD92], [DSG94] [ZNA05], la pendiente [AYF90] y la curvatura [JGC02] [KSX04].

Las características descriptivas se suelen clasificar en [Parodi 2014]:

- **Globales:** consideran la firma como un todo. Son menos sensibles al ruido y a las variaciones de la firma [HYW04], lo cual lo hace una buen opción cuando se trabaja solamente con firmas auténticas [Parodi 2014].
- **Locales:** consideran partes especificas de la firma. Implica recurrir a técnicas de extracción más complejas [MLD11], como por ejemplo utilizar grillas de segmentacion.

A partir de [NR77] se han utilizado grillas para, a partir de ellas, extraer características locales, técnica también utilizada en [SP86], [JS01], [JYBS00], [SJBS04], [JBS05], [OJFS05], [BZZE13].

El trabajo de [Parodi 2014] sigue esta línea proponiendo un método de grilla circular para la extracción de características locales. La idea de utilizar una grilla circular proviene de que resultan en menos sectores vacíos, además de poseer una estructura más apropiada en vistas de conseguir robustez respecto de la rotación de las firmas.

Las características propuestas en [Parodi 2014] están basadas en los antecedentes de: [JYBS00] y [JS01] que utilizaron la densidad de píxeles (número de píxeles dentro de una celda), [SPB94] donde se calcula la distribución de píxles y [JBS05] que propone utilizar el centro de gravedad (distancia del centro de gravedad en cada celda al centro de gravedad de toda la firma).

Escencialmente, [Parodi 2014] ubica un esquema circular alrededor de la firma y la divide en N sectores iguales, haciendo coincidir el centro de la grilla con el centro de masa de la firma (con esto se pretende la invariancia a la traslación). Luego extrae las siguientes características (ver figura 5.3):[4]

- x_{PD}: Densidad de distribución de píxeles (en inglés, *pixel density distributon*)

$$x_{PD_i} = \frac{\text{Número de píxeles negros de la sección } i}{\text{Número total de píxeles de la sección } i}$$

- x_{DGC}: Distancia al centro de masa (en inglés, *gravity center distance*).

$$x_{DGC_i} = \frac{d_{GC_i}}{R}$$

  Dividir por R otorga cierta invarianza al escalado.

- x_{AGC}: Ángulo al centro de masa (en inglés, *gravity center angle*).

$$x_{AGC_i} = \frac{\alpha_{GC_i}}{\frac{2\pi}{N}}$$

  Dividir por $\frac{2\pi}{N}$ no es estrictamente necesario, sino para lograr escalar adecuadamente α_{GC} para diferentes N (aunque nunca se trabajará con diferentes N simultáneamente).

El vector característico es simplemente la concatenación de todas las características extraídas.

[4] Notaremos N el número de secciones circulares e i a la i-ésima sección circular, con $i = 0, ..., N-1$. También referiremos con R al radio de la grilla (distancia entre los dos puntos extremos).

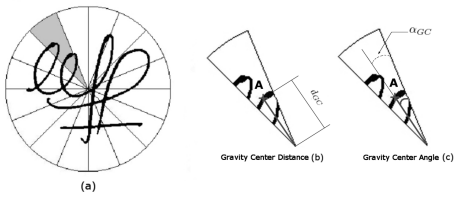


Figura 5.3: Esquema de grilla circular propuesto en [Parodi 2014]. (a) Se ha sombreado el sector analizado; (b) Distancia al centro de masa; (c) Ángulo del centro de masa. Fuente [Parodi 2014].

5.1.3.1. Invarianza rotacional mediante DFT

Para lograr robustez frente a rotaciones, en [Parodi 2014] se propone tomar los módulos de la Transformada Discreta de Fourier (DFT) de los vectores de características correspondientes a cada anillo de la grilla circular. Esta transformación produce características que son invariantes ante desplazamientos cíclicos de la secuencia original, lo cual equivale a rotaciones de la firma cuando el ángulo rotado es múltiplo de $\frac{2\pi}{N}$.

Sin embargo, cabe señalar una limitación de este enfoque: la invarianza obtenida es discreta, ya que solo se preserva ante rotaciones que correspondan exactamente al tamaño angular de una sección. Para valores pequeños de N (es decir, pocas divisiones y por ende secciones angularmente grandes), las rotaciones típicas que pueden aparecer entre distintas instancias de una misma firma —producto de variaciones en la inclinación al momento de firmar— difícilmente coincidirán con múltiplos exactos del tamaño de sección. Esto sugiere que, en la práctica, el beneficio de esta técnica puede verse reducido cuando N es pequeño.

Consideramos que el desarrollo de métodos que provean una invarianza rotacional más flexible podría resultar valioso, no solo para capturar la variabilidad intrapersonal en el ángulo de firmado, sino también porque habilitaría el uso de *data augmentation* basada en rotaciones, permitiendo generar conjuntos de entrenamiento significativamente más grandes.

5.1.4. Reutilizando las técnicas en nuestro problema

5.1.4.1. Entrenamiento de clasificadores y evaluación

Recordar que en [Parodi 2014] el problema a tratar consistía en verificar la identidad proclamada por el autor de cierta firma. Para ello, usa sobre dicha instancia un clasificador binario previamente entrenado para el autor que está siendo verificado, que determinará si la firma pertenece a la clase auténtica o falsa para dicha autoría. Cada clasificador (uno para cada autor) entrena la clase positiva utilizando firmas auténticas de dicho autor, y la clase negativa utilizando falsificaciones aleatorias (simuladas tomando firmas de otros autores). Esto último es muy importante debido a que, en un caso de uso real no se cuenta con falsificaciones al momento del entrenamiento. Se puede leer en detalle el protocolo de evaluación en la sección 3.5 de [Parodi 2014].

Como se dijo en la sección 4.3, nosotros queremos entrenar un clasificador para determinar si dos instancias de firmas son *"la misma firma"* (fueron realizadas por el mismo autor). Notar que, potencialmente, los autores de las mencionadas firmas nunca han sido vistos con anterioridad por nuestro modelo. Es por ello que si bien reutilizaremos el proceso de extracción de catacterísticas con grilla circular de [Parodi 2014], no podemos entrenar un clasificador por autor.

Lo que haremos será entrenar un clasificador que reciba una combinación de las características de las imágenes de las dos firmas a analizar y determine si son o no la "misma firma". Para combinar los vectores característicos probaremos diferentes técnicas posibles tales como la diferencia vectorial o la concatenación de los vectores respectivos. Nuestra metodología se explica con detalle en el capítulo siguiente, donde ahondaremos en el desarrollo de nuestros experimentos.

Nuestro problema es conceptualmente más complicado que el enfoque de [Parodi 2014], donde cada clasificador captura la varianza intrapersonal de un autor específico. La varianza intrapersonal puede diferir significativamente entre autores, por lo que construir un modelo que la capture en términos

generales resulta mucho más difícil.

Más aún, la diferencia fundamental radica en que nuestra función de similaridad debe **extrapolar a autores no vistos durante el entrenamiento**. Mientras que en *VAFM* se entrena un clasificador específico para cada autor (disponiendo de ejemplos de dicho autor), en nuestro caso el modelo debe aprender una noción de "misma autoría" lo suficientemente general como para aplicarse a firmas de personas que nunca formaron parte del conjunto de entrenamiento. Esta capacidad de generalización a nuevas identidades es lo que hace al problema intrínsecamente más difícil y lo emparenta con los problemas de *One-Shot Learning* discutidos en la sección 5.3.2.

5.1.4.2. Métricas y convenciones

Notar que tal como se ha explicado en la subsección 3.1.8.4, los conceptos FAR y FRR se encuentran invertidos en nuestro trabajo respecto a [Parodi 2014]. Esto se debe a que nuestra clase positiva corresponde a cuando las firmas son *no coincidentes* (al contrario de lo que ocurre con [Parodi 2014], donde la clase positiva concierne a que se verifica la identidad proclamada). Debemos tener en cuenta esto cuando analicemos lo resultados reportados en el mencionado trabajo para comprarlos con alguno de nuestros resultados.

Si nos fijamos en el resultado de FAR de [Parodi 2014], es importante siempre considerar el respectivo a firmas ficticias, ya que como se explicó en 5.1.1 en nuestro trabajo no utilizaremos falsificaciones simuladas.

Cabe aclarar, también, que se han utilizado conjunto de datos distintos (en el capítulo siguiente se ahondará sobre la elección y confección de los conjuntos de datos utilizados en nuestros experimentos). Y además, recalcar nuevamente que la metodología de evaluación y entrenamiento es diferente (evaluamos en firmas de autores no vistos en el entrenamiento), por lo cual el problema que nos proponemos resolver es conceptualmente mucho más difícil al de *VAFM*.

5.1.5. Resultados de [Parodi 2014]

Los resultados de [Parodi 2014] correboran que la elección de grilla circular para el problema de *VAFM* es ventajosa sobre la grilla rectangular. La grilla rectangular tiene buenos resultados de FRR para número de subdivisiones grande (N igual a 32, 64 o 128) pero peores resultados de FAR sobre falsificaciones simples y entrenadas en dichos caso (es mejor par N igual a 8). En cambio la grilla circular funciona bien en ambos casos para N medianos (N igual a 16), y luego empeora los resultados obtenidos al aumentar el número de subdivisiones. En *tesis-parodi* se baraja la posibilidad de que esto se deba a que al aumentar las subdivisiones en la grilla circular el área de los sectores se reduce no uniformemente (al contrario de lo que sucede en la grilla rectangular). Como en nuestro trabajo no trabajaremos con falsificaciones simples (sólo con falsificaciones aleatorias como se dijo), consideraremos el grillado rectangular con N grande (N igual a 128), ya que en dicho casos muestra buenos resultados de FAR en [Parodi 2014].

Finalmente los mejores resultados reportados en [Parodi 2014] corresponden a las siguientes configuraciones:

	Grilla circular N=16, Kernal polinómico	Grilla rectangular N=128, Kernel RBF
FRR	18.75 %	27.875 %
FAR (firmas ficticias)	0.0727 %	0.0106 %
FAR (fals. simples y entrenadas)	2.125 %	14.9167 %

Nuestros objetivos, mencionados en 3.1.8.5,

Notar que ninguno de los resultados exhibidos satisface (si bien está cerca) nuestra meta de lograr un *recall* de al menos 0.8 y *precision* de 0.95 mencionada en . De todos modo, cabe recordar las limitaciones expuestas en secciones anteriores sobre extrapolar los citados resultados a nuestro trabajo.

5.1.5.1. Críticas metodológicas

En los siguientes apartados expresamos algunas críticas al trabajo analizado. Entenderlas y analizarlas nos es útil para interpretar los resultados correctamente. Y también nos ha ayudado en el diseño correcto de nuestro protocolo de evaluación (capítulo siguiente).

5.1.5.1.1. Desbalance en el conjunto de datos de entrenamiento

La clase verdadera de cada autor es entrenada con 14 firmas auténticas, mientras que la clase falsa es entrenada con firmas auténticas de otros autores.

El desbalance de clases es muy significativo y esto puede generar (depende de la configuración de entrenamiento de las SVM) un sesgo a favor de reducir el FAR sobre el FRR. Lamentablemente no tenemos acceso al código de los experimentos para constatar si se ha tomado alguna medida para mitigar el desbalance, como ponderar las clases a la hora de entrenar las SVM.

Este problem fue anoticiado por la autora en las conclusiones de la sección 3.9 [Parodi 2014].

5.1.5.1.2. Error metodológico en la evaluación de firmas ficticias

El trabajo reporta resultados de FAR para firmas ficticias. Para calcularlo utiliza firmas auténticas de los otros autores. Estas firmas provienen del conjunto de evaluación, o sea que no han sido utilizadas durante el entrenamiento.

Pese a que a primeras vistas puede parecer correcto tiene un sutil pero severo error: los autores de estas firmas sí han sido potencialmente usados para entrenar la clase falsa del clasificador que está siendo evaluado. Por lo qué el clasificador conoce firmas parecidas para la clase falsa.

Naturalmente, esta correlación oculta podría no repetirse en un caso real. Ya que las falsificaciones simuladas no necesariamente se parecerán a ninguna de las firmas falsas que ha visto el clasificador durante el entrenamiento.

No podemos decir con certeza que no se hayan tomado recaudos frente a esta situación, como evaluar el FAR de firmas ficticias para cada clasificador en autores que no se hayan utilizado para entrenar, porque no tenemos acceso al código de los experimentos. No obstante, como el protocolo de evaluación (sección 3.5 de [Parodi 2014]) no hace salvedad alguna, pensamos que es probable que no se haya anoticiado de este error.

Es por este error que podemos sospechar que el FAR de firmas ficticias es demasiado optimista.

5.1.5.1.3. Desbalance en el conjunto de datos de evaluación

En la sección 3.5 de [Parodi 2014] (*"Protocolo de evaluación"*), se explica que para la fase de test se utilizan

5.2. Conjuntos de datos

5.2.1. GPDS300

Observaciones a su proceso de evaluación y conclusiones En [TESIS-MARIANELA] se utilizó la base de datos GPDS300 Signature CORPUS [VFTA07]. Por regulaciones de la UE ya no se encuentra disponible para su uso. La base contaba con 160 escritore, 24 firmas auténticas y 30 falsificaciones por autor.

Fue el utilizado en [Parodi 2014].

En [TESIS-MARIANELA] se utilizó la base de datos GPDS300 Signature CORPUS [VFTA07]. Por regulaciones de la UE ya no se encuentra disponible para su uso. En la página de GPDS (grupo de procesamiento de señales) existe actualmente un dataset sintético. En la Conferencia Internacional sobre Análisis y Reconocimiento de Documentos (ICDAR por sus siglas en ingles) se han realizado competencias de vital importancia al área brindando a los investigadores la posibilidad de comparar el rendimiento de sus sistemas y proponer desafíos interesantes en sus datasets.

La organización de estas competencias es de vital importancia para el área ya que no sólo brinda a los investigadores la posibilidad de comparar el rendimiento de sus sistemas, sino también de testear sus algoritmos con bases de datos estándar y al alcance de toda la comunidad. Además, las bases de datos presentadas en el marco de las competencias suelen proponer desafíos interesantes. Por ejemplo, para las competencias llevadas a cabo durante ICDAR 2009 y ICDAR 2011, se construyó una base de datos conteniendo firmas de origen chino, mientras que para la competencia llevada a cabo durante ICDAR 2013 se introdujo una base de datos conteniendo firmas de origen japonés. Además de las competencias, en los últimos años se organizaron distintos talleres y tutoriales en el marco de las mencionadas conferencias. Estas reuniones periódicas permiten discutir los as- pectos emergentes del área, brindándoles a los investigadores la oportunidad de intercambiar ideas y discutir su trabajo actual en un contexto informal y disten- didp.

Los primeros dos son de las competencias organizadas por ICDAR (International Conference on Document Analysis and Recognition). La ventaja es que se consigue fácil tanto el dataset como el papper con los resultados. Ademas hay dando vueltas implementaciones (que no probé) de gente que se tiró a resolver esos dataset. SigComp2011 tiene firmas occidentales (alemanas) y orientales (chinas) yo usaria solo las occidentales. SigComp2009 tiene 1800 firmas (entre autenticas y falsificaciones) SigComp2011 tiene 400 firmas occidentales (entre autenticas y falsificaciones). Yo me tiraría a usar la 2009 porque tiene más. Aunque la otra es un poco más nueva.

La que usa Marianela en su tesis es GPDS960 pero no está disponible por una resolución de privacidad de la UE (ver la fuente). En internet tampoco la encontré (no busqué mucho tampoco). La pagina sugiere usar las bases de datos sintéticas que listé abajo. A mí no me gustó el hecho de que sea sintética, aunque deben estar bien generadas, hay un paper y todo. Otro contra es que hay que llenar un formulario y mandarlo, no es tan simple como poner descargar. Lo bueno de esa opción es que son muchísimas como 4000. Tampoco pude encontrar gratis el papper de este último dataset .ºff-line handwritten SignatureGPDS-960 Corpus."F. Vargas, M.A. Ferrer, C.M. Travieso, and J.B. Alonso.

Los conjuntos de datos utilizados Sigcomp 2009 [SigComp 2009] y Sigcomp 2011 [SigComp 2009], provienen de las competencias llevadas a cabo durante las ediciones 2009 y 2011 respectivamente de la International Conference on Document Analysis and Recognition (ICDAR).

Las competencias presentan base de datos interesantes y permiten a los investigadores del área la posibilidad de comparar su rendimiento de manera estándar [Parodi 2014].

5.3. Problemas de similitud

5.3.1. Problemas clásicos similares: *Face Recognition* y *Face Verification*

Los rostros, al igual que las firmas, son una característica biométrica que puede ser utilizada para la autenticación. No es de sorprender, entonces, que los mismos problemas que atañen a firmas tengan su paralelismo con el procesamiento de imágenes de rostros. En ambos casos resulta fundamental extraer características discriminativas que permitan clasificar o comparar instancias provenientes de distintas personas.

El problema de ***Face Verification*** consiste en verificar si un cierto rostro pertenece a una determinada persona (de la cual se conocen rostros). La entrada es una imagen del rostro junto con la identidad declarada, y la salida del modelo es un booleano indicando si la imagen corresponde efectivamente a dicha persona:

$$Face\ Verification\ System(face_image,\ person_id) : Bool$$

En contraste, en el problema de ***Face Recognition*** sólo se posee una imagen y se debe detectar si el rostro corresponde a alguna persona de una base de datos de rostros conocidos. Es decir, la entrada es únicamente la imagen del rostro y la salida es la identidad correspondiente (o "no reconocido" si no coincide con ninguno de la base de datos):

$$Face\ Recognition\ System(face_image) : Person\ ID \cup \{Not\ Recognized\}$$

Notar que si bien el problema de *Face Recognition* parece más difícil que el de *Face Verification*, una solución a este último con suficiente precisión, se puede trasladar al segundo iterando para identidad posible de la base de datos.

5.3.2. *One-Shot Learning*

La mayor dificultad de ambos problemas es que generalmente consisten en problemas de ***One-Shot learning*** (o *Few-Shot learning*):

- El conjunto de entrenamiento cuenta con un (o unos pocos) ejemplos para cada categoría (persona o identidad en el caso de rostros).
- Existen a su vez muchas etiquetas (muchas personas distintas). Y además pueden agregarse nuevas e idealmente no queremos re-entrenar todo el modelo cada vez que esto ocurra.

De estas restricciones se desprende que no será útil atacar el problema utilizando un clasificador donde haya una clase por identidad, ya que al poseer pocos ejemplos de cada clase, el entrenamiento lograría una muy pobre precisión. Además, este enfoque obligaría a re-entrenar el modelo cada vez que se quiera agregar una nueva persona.
En vez de esto, una solución posible es aprender una función de similaridad d:

$$d(img_1, img_2) = \text{grado de diferencia entre ambas personas}$$

Luego, podemos resolver *Face Verification* comparando la salida de d respecto de un umbral τ:

$$d(img_1, img_2) \begin{cases} \leq \tau & si \quad id(img_1) = id(img_2) \\ > \tau & si \quad id(img_1) \neq id(img_2) \end{cases}$$

De este modo resolvemos un problema "más fácil" que el de reconocer la identidad de cada sujeto (el modelo tiene disponible en su valuación la imágen contra la cuál está comparando).

Además, con este enfoque de aprender una función de similaridad de rostros d generalizable a identidades no vistas sobre el entrenamiento, no necesitaremos re-entrenar el modelo de agregarse nuevas personas a la base de datos.

5.3.3. *Redes Siamesas*

La arquitectura que se emplea para esta clase de problemas es conocida como ***Redes Siamesas*** [Face Verification LeCun 2005]. Consiste en utilizar una red convolucional clásica (una sucesión de convoluciones y capas de *pooling*) y valerse de las activaciones de entrada de la última capa (la predicción) como vector de características (también llamado ***embedding*** o ***encoding*** de la imagen). Para finalmente calcular la similitud como la distancia entre ambos *embeddings*. Ver figura 5.4. En la figura 5.5 se muestra un ejemplo de cómo se estructura un dataset pareado para entrenar estas redes.

Notar que en el cálculo de los *embeddings* de ambas imágenes se utilizan los mismos pesos (notado como W en la figura 5.4). Esta técnica dónde los mismos parámetros se reutilizan, en dos o más cómputos diferentes, en cada evaluación del modelo, se suele denominar *Tied Weights* en la jerga del *aprendizaje profundo*.

En línea con la figura 5.4, notaremos con G_W la función definida al aplicar nuestro modelo (también llamado *encoder*). Siendo entonces $G_W(img)$ el *encoding* o *embedding* de la imagen *img* utilizada.

Para comparar los *embeddings* pueden utilizarse diferentes funciones distancia (la distancia coseno, la euclídea y $\mathcal{X}^2$ están entre las más escogidas).

El entrenamiento debiera optimizar el *encoder* G_W, hallando valores de W óptimos, para extraer características útiles a la hora de comparar dos imágenes (rostros en el caso del problema de *Face Verification*). Notar que este es un claro ejemplo de lo discutido en 3.3.12 donde se utiliza redes neuronales para aprender las características útiles a extraer, en vez de codificarlas a mano (las cuáles probablemente

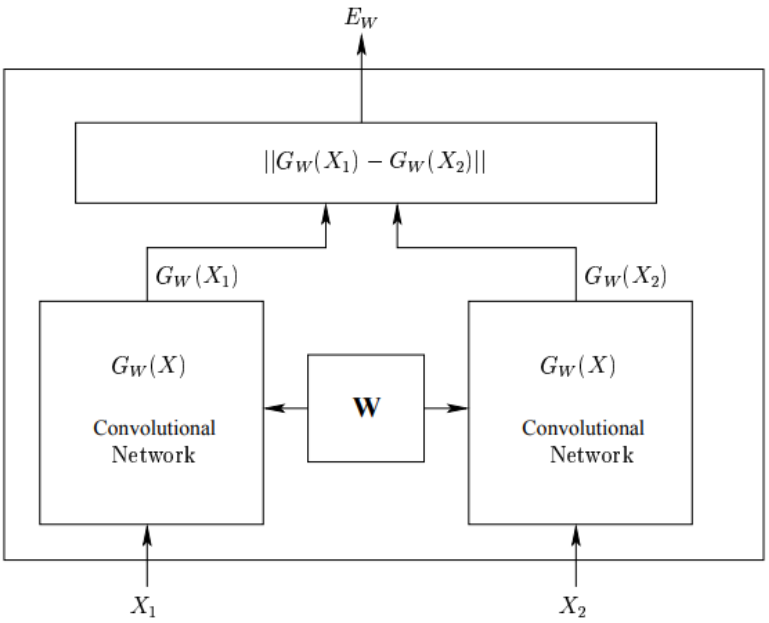


Figura 5.4: Arquitectura general de una *Red Siamesa*. Fuente: [Face Verification LeCun 2005]

son dependiente del dominio de las imágenes a comparar). Más formalmente, el entrenamiento deberá aprender parámetros del *encoder* tal que:

$$|| \ G_W(img_1) - G_W(img_2) \ ||_2 < \tau \iff \text{id}(img_1) = \text{id}(img_2)$$

Notar la importancia para el desarrollo del *Aprendizaje Profundo* de la funcionalidad de diferenciación automática (también llamada ***Autograd***) de los *frameworks* como *Pytorch* o *TensorFlow*: cuestiones como *Tied Weights* en la formulación del costo hacen que las expresiones de los gradientes sean complicadas de tener que calculárselas manualmente. Afortunadamente, las librerías encapsulan dicho problema y los mismos son computados automáticamente. Lo que da pie a que se experimente con formulaciones más complejas y da lugar a entrenamientos más sofisticados.

5.3.4.

Resta definir la función de costo

5.3.5. *Image Similarity*

5.3.6. *Omniglot Dataset*



Figura 5.5: Ejemplo de un dataset pareado de firmas: cada instancia consiste en un par de firmas junto con un veredicto indicando si corresponden o no al mismo autor.

Capítulo 6

Experimentos

6.1. Formulación del problema en la práctica

En este capítulo describimos cómo se lleva a la práctica la formulación presentada en el capítulo 4: el problema a resolver es determinar si dos firmas son de la misma autoría, con el fin de detectar errores de confección en los telegramas electorales que tienen varias páginas y donde las firmas deben coincidir entre página y página. Un ejemplo de telegrama real puede verse en `https://drive.google.com/file/d/1X36KFIH7QEQzfrL6or0WEYuK1FaxuMsS`.

De acuerdo con los objetivos definidos en la sección 4.3, buscamos maximizar la *precision* sujeta a obtener un *recall* no menor que 0,9. Para ello nos basamos en trabajos anteriores de problemas similares y, en particular, utilizamos la tesis de Parodi [Parodi 2014] como principal punto de comparación. La tesis de Parodi resuelve el problema de *VAFM* (*Verificación Automática de Firmas Manuscritas*), donde se dispone de K autores y, dada una firma que declara ser de un autor i, se debe verificar su autenticidad entrenando un clasificador por autor. En nuestro caso el enfoque es distinto: queremos detectar errores de confección y no falsificaciones, por lo que no consideramos firmas falsificadas (hechas por un autor pretendiendo ser otro), sino únicamente firmas auténticas y sus combinaciones.

El problema es conceptualmente cercano al reconocimiento facial (*Face Recognition*/*Face Verification*), donde también se comparan instancias biométricas de distintas personas. En los experimentos aprovechamos ideas de la literatura de reconocimiento facial y de redes siamesas desarrolladas por grandes actores de la industria (por ejemplo, trabajos recientes sobre reconocimiento de caras basados en redes convolucionales profundas).

6.2. Conjuntos de datos

6.2.1. Conjuntos de datos curados

En una primera etapa no contamos con un *dataset* de pares de firmas listo para usar, sino con una colección de telegramas. Para acotar el problema de similaridad de firmas y poder comparar los resultados con soluciones existentes, trabajamos inicialmente con *datasets* curados de firmas manuscritas y recién luego nos movemos al conjunto de datos generado por nosotros.

Uno de los conjuntos de datos considerados es **CEDAR Signature**[1], que contiene 1 320 firmas de 55 autores (24 auténticas por autor) y 1 320 falsificaciones. Siguiendo los criterios de nuestro problema, no utilizamos las falsificaciones directamente, sino que generamos pares negativos (*falsos matcheos*) combinando firmas auténticas de distintos autores.

Otro conjunto de datos utilizado es **SigComp 2009**, que fue desarrollado para la competencia de verificación de firmas del ICDAR 2009. Este conjunto se divide en dos partes: un subconjunto de entrenamiento y uno de evaluación, provistos por instituciones distintas.

[1] Disponible, por ejemplo, en repositorios como `https://paperswithcode.com/dataset/cedar-signature`.

6.2.1.0.1. SigComp 2009 Train (NISDICC). El subconjunto de entrenamiento fue proporcionado por el *National Institute of Standards and Technology* (NIST) y el *Document Image and Character Collection* (DICC). Contiene firmas de 12 autores auténticos (identificados como 001–012) y 31 falsificadores (021–051). Cada autor auténtico firmó 5 originales, y en general cada falsificador realizó 5 falsificaciones por autor auténtico; sin embargo, hay falsificaciones faltantes (1 898 archivos en lugar de 1 920). Las imágenes tienen diferentes tamaños y están en escala de grises.

6.2.1.0.2. SigComp 2009 Test (NFI). El subconjunto de evaluación fue proporcionado por el *Netherlands Forensic Institute* (NFI). Contiene firmas de autores identificados del 001 al 100, aunque muchos identificadores no están presentes. No todos los autores son falsificadores (la cantidad de falsificadores es menor que la cantidad de autores). En general, cada autor auténtico firmó 12 originales (algunos firmaron 11 y uno firmó 6), y cada falsificador realizó 6 falsificaciones para 2–4 autores auténticos. También hay falsificaciones faltantes (1 564 archivos en lugar de 1 953). Las imágenes tienen diferentes tamaños y están en escala de grises.

6.2.2. Confección del *dataset* propio (cropeo de firmas)

TODO: Mencionar que detectar firmas en ubicaciones arbitrarias o segmentar qué píxeles pertenecen a cada firma es un problema en sí mismo, que no abordamos aquí gracias a que la estructura del telegrama es fija. Citar [Handwritten Text Segmentation 2019] como referencia al problema de segmentación de texto manuscrito.

Para evaluar los modelos en un escenario más cercano a la aplicación real, construimos un conjunto de datos propio a partir de telegramas electorales. Inicialmente no contamos con un *dataset* propiamente dicho (de pares de firmas, ni siquiera de firmas aisladas), sino con imágenes completas de telegramas. Aprovechamos que la localización de las firmas en el formulario es fija para extraerlas mediante recortes sobre el telegrama. Luego, para detectar si hay o no hay firma en dicha posición, probamos primero una heurística sencilla basada en el tamaño de las componentes conexas (alrededor de 0.90 de *accuracy*); tras una revisión manual rápida de la clasificación obtenida y algunas correcciones, entrenamos un clasificador de redes neuronales que alcanzó aproximadamente 0.99 de *accuracy*. Esto ilustra cómo un modelo puede servir como herramienta para un paso intermedio de limpieza y construcción del *dataset*.

6.3. Resultados

En todas las gráficas de métricas (por ejemplo, curvas ROC y Precision–Recall) fijamos la escala de los ejes x e y en el intervalo $[0, 1]$ para que las comparaciones entre modelos sean coherentes y visualmente consistentes. Además, para comparar de forma resumida el desempeño entre distintas particiones de validación cruzada reportamos también la métrica F_1, definida como la media armónica entre *precision* y *recall*.

6.3.1. Resultados utilizando la tesis de Marianela

En primer lugar reproducimos, en la medida de lo posible, el esquema basado en extracción de *features* manuales propuesto en [Parodi 2014], utilizando los descriptores construidos sobre grillas y entrenando clasificadores como *Random Forest* o SVM sobre los vectores característicos. Sobre el conjunto de datos de referencia SigComp2009 se obtiene, por ejemplo, una *performance* del orden de 0,85 para la métrica de interés combinando extracción de características manual (Parodi) con *Random Forest*.

6.3.2. Resultados utilizando redes siamesas

Como siguiente paso entrenamos redes siamesas convolucionales que reciben pares de recortes de firmas y producen un puntaje de similitud a partir de la distancia entre sus *embeddings*, tal como se describe en la subsección 3.3.14. Aquí se incluirán los resultados cuantitativos comparables con el enfoque basado en [Parodi 2014] (falta completar esta parte con los valores finales de *precision*, *recall* y curvas ROC / Precision–Recall).

6.3.3. Resultados con diferentes arquitecturas de redes siamesas

Exploramos diferentes arquitecturas convolucionales para el *encoder* G_W (variando profundidad, número de filtros y uso de bloques residuales tipo ResNet) y analizamos su impacto sobre las métricas de similitud. Esta sección detalla comparaciones entre arquitecturas más pequeñas y más profundas, así como el efecto de utilizar modelos preentrenados y *fine-tuning* (resta completar con los experimentos definitivos).

6.3.4. Online triplet loss y offline triplet loss

Finalmente entrenamos modelos siameses utilizando *triplet loss* (ver 3.3.14.1), comparando estrategias de generación de tripletas *offline* (precomputadas) y *online* (minado de tripletas dentro de cada *mini-batch*). El conjunto de datos se genera particionando autores entre entrenamiento y test (no se mezclan firmas de un mismo autor entre ambos, y se utilizan únicamente firmas originales, no falsificaciones). En esta sección se reportan las métricas alcanzadas por cada estrategia y se discuten sus ventajas prácticas en términos de estabilidad de entrenamiento y capacidad de generalización (pendiente de completar con los números finales).

Capítulo 7

Conclusiones

En esta tesina se planteó el problema de detectar automáticamente inconsistencias en las firmas manuscritas de los telegramas electorales utilizados en el escrutinio provisorio con asistencia digital en la Provincia de Santa Fe. A diferencia del enfoque clásico de VAFM, el objetivo no fue identificar falsificaciones intencionales, sino errores de confección (ausencia de firmas o firmas no coincidentes entre páginas) que derivan en retransmisiones y trabajo adicional en el centro de cómputo. El problema se formalizó como una tarea de clasificación binaria sobre pares de imágenes de firmas, definiendo cuidadosamente las métricas de evaluación (*precision*, *recall*, curvas ROC / Precision–Recall) y el rol asimétrico de los errores según el uso práctico del sistema.

Desde el punto de vista metodológico, se construyó primero un conjunto de datos propio a partir de telegramas reales, aprovechando que el diseño de los formularios es conocido de antemano. Se diseñó un pipeline donde, a partir de fotografías de telegramas, se recortan automáticamente las zonas de firmas y se entrena un modelo intermedio para detectar presencia/ausencia de firma, que permitió limpiar y estructurar el conjunto de datos definitivo. Sobre este *dataset* se compararon dos familias de enfoques: (i) métodos "clásicos" basados en extracción manual de características (grillas y descriptores inspirados en [Parodi 2014]) combinados con clasificadores como Random Forest y SVM; y (ii) modelos de Aprendizaje Profundo basados en redes convolucionales y arquitecturas siamesas para aprendizaje de representaciones y métricas de similitud.

Los experimentos muestran que las redes siamesas convolucionales, entrenadas con funciones de costo específicas para aprendizaje de métricas (como *triplet loss* en variantes *online* y *offline*), logran resultados claramente superiores a los enfoques basados en características manuales, tanto en conjuntos de datos estándar de firmas como en el escenario particular de los telegramas. Se alcanzan configuraciones de operación donde el modelo mantiene un *recall* en torno a 0.8–0.9 con alta *precision*, lo que en términos prácticos se traduce en una reducción significativa del número de retransmisiones por errores de firmas, manteniendo a la vez un volumen limitado de falsas alarmas aceptable para el flujo de trabajo del centro de cómputo.

Como principales limitaciones se destaca la dependencia del modelo respecto del formato de los telegramas (posición fija de las firmas y calidad de las imágenes), la ausencia de falsificaciones intencionales en el conjunto de datos (el foco está puesto en errores de confección) y la necesidad de contar con volúmenes suficientes de datos etiquetados para explorar arquitecturas más grandes. En el capítulo de Trabajo futuro se discuten posibles extensiones, entre ellas: el uso de representaciones aprendidas mediante modelos generativos (autoencoders o GANs), la exploración de arquitecturas siamesas y de *metric learning* más modernas, y la adaptación del pipeline a otros problemas de similitud de imágenes, como verificación de identidad en documentos o análisis de otras marcas manuscritas. Estas líneas abren el camino para seguir mejorando la robustez y capacidad de generalización del sistema propuesto.

Apéndice A

Construcción del conjunto de datos

Resultados al problema de presencia de firmas

Resultados al problema de similaridad entre firmas - Conjunto de datos - Sigcomp 2009 (citar y explicar brevemente) - CEDAR (citar y explicar brevemente) - Conjunto de datos propios a partir de telegramas (como lo construí) - recortar - filtrar los lugares donde no aparecen firmas usando la solucion anterior - matchear los cuadraditos entre diferentes paginas - Soluciones planteadas - Approach I (nuestra implementacion a lo de marianela) - Approach II (deep learning) - Experimentos y resultados - Experimentos planteados (1,2,3,4) - Resultados respectivos

Se obtiene el conjunto de datos crudo sobre el cual se desea aprender. Los datos son la base del proceso y condicionarán todo el trabajo posterior. No es trivial controlar de qué manera son recolectados. En la academia se suelen utilizar conjuntos de estándares para los cuales por lo general existen publicaciones que detallan sus características. De este modo se garantiza que se trabaja con datos que han sido rigurosamente extraídos (por ejemplo si se trata de texto manuscrito se detalla el espesor del bolígrafo, el tipo de superficie sobre la cual se escribe, la posición respecto a los renglones, la distribución en género/edad de los voluntarios a escribir, etc). Generalmente se trata de que posean características tales que los resultados de las investigación sobre ellos puedan extrapolarse a conjuntos de datos del mismo tipo. Además se cuenta con de conjuntos de datos estándares que permite a los científicos comparar sus resultados. En trabajos de aplicación (como en la industria), es un paso muy importante que muchas veces se descuida. Una extracción incorrecta de los datos podría introducir errores en el entrenamiento difíciles de detectar. Es importante que se realice de forma iterativa y siempre visualizando los conjuntos de datos generados para garantizar una relativa sanidad de los mismos. Un ejemplo frecuente cuando se enseña *Machine Learning* es el del entrenamiento de un detector de tanques de guerra camuflados que a primeras vistas poseía una asombrosa precisión pero luego resultó estar detectando presencia de nubes en el cielo: las fotos del escenario con los tanques habían sido tomadas un día soleado y las fotos sin los tanques un día nublado. Aparentemente esta situación anecdótica constituye un mito urbano del *Machine Learning*, ver [CITA GWERN https://www.gwern.net/Tanks#should-we-tell-stories-we-know-arent-true]. Aunque en la misma fuente se pueden encontrar ejemplos documentados de errores de generalización en aplicaciones de *Machine Learning* producto de haber entrenado con *datasets* sesgados.

En nuestra implementación que sigue un enfoque clásico basándonos en [Parodi 2014] utilizaremos para la fase de clasificación *SVM* para la extracción de características el método de grilla circular que explicaremos en las subsecciones siguientes *SVM* cuando tratemos con el enfoque clásico y usemos

Del mismo modo es muy importante definir bien el formato de los datos (ver si se trata de un problema de regresión una clasificación y en último caso definir bien las clases)

Apéndice B

Trabajo futuro

detectar suplantaciones

Ordenar alfabeticamente la bibliografia

Apéndice C

Bibliografía

[Hao Li et al. 2018] Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, Tom Goldstein. (2018). *Visualizing the Loss Landscape of Neural Nets*. University of Maryland, College Park United States Naval Academy Cornell University.

[Dauphin et al. 2014] Yann N. Dauphin et al. (2014). *Identifying and attacking the saddle point problem in high-dimensional non-convex optimization*. Advances in Neural Information Processing Systems, pp. 2933-2941.

[Nielsen 2015] Nielsen, M. A. (2015). *Neural Networks and Deep Learning*. Determination Press.

[Ruder 2016] Sebastian Ruder. (2016). *An overview of gradient descent optimization algorithms*. arXiv: 1609.04747 [cs.LG].

[Xu et al. 2015] Xu, B., Wang, N., Chen, T., and Li, M. (2015). *Empirical evaluation of rectified activations in convolutional network*. CoRR, abs/1505.00853.

[Maas et al. 2013] Maas, A. L., Hannun, A. Y., and Ng, A. Y. (2013). *Rectifier nonlinearities improve neural network acoustic models*. In ICML Workshop on Deep Learning for Audio, Speech and Language Processing.

[Nair and Hinton 2010] Nair, V. and Hinton, G. E. (2010). *Rectified linear units improve restricted boltzmann machines*. In Proceedings of the 27th International Conference on Machine Learning (ICML-10), pp. 807–814.

[Deng et al. 2009] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. (2009). *ImageNet: A large-scale hierarchical image database*. 2009 IEEE Conference on Computer Vision and Pattern Recognition.

[Glorot et al. 2011] Glorot, X., Bordes, A., and Bengio, Y. (2011). *Deep sparse rectifier neural networks*. In Proceedings of the 14th International Conference on Artificial Intelligence and Statistics, pp. 315-323.

[Krizhevsky et al. 2012] Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). *Imagenet classification with deep convolutional neural networks*. In Advances in Neural Information Processing Systems, pp. 1097-1105.

[Wolpert 1996] Wolpert, D. H. (1996). *The lack of a priori distinctions between learning algorithms*. Neural Computation, 8(7), pp. 1341-1390.

[Goodfellow et al. 2016] Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.

[LeCun et al. 2015] LeCun, Y., Bengio, Y., and Hinton, G. (2015). *Deep learning*. Nature, 521(7553), p. 436.

[Rumelhart et al. 1986] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). *Learning representations by backpropagating errors*. Nature, 323(9).

[Hinton 2014] Hinton, G. (2014). *Neural Networks for Machine Learning*. Coursera course. `http://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf`

[Kingma and Ba 2014] Kingma, D. P. and Ba, J. (2014). *Adam: A method for stochastic optimization*. CoRR, abs/1412.6980.

[He et al. 2015] He, K., Zhang, X., Ren, S., and Sun, J. (2015). *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*. `https://arxiv.org/abs/1502.01852`

[Glorot and Bengio 2010] Glorot, X. and Bengio, Y. (2010). *Understanding the difficulty of training deep feedforward neural networks*. DIRO, Université de Montréal, Montréal, Québec, Canada. `http://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf`

[Ioffe and Szegedy 2015] Ioffe, S. and Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. `https://arxiv.org/abs/1502.03167`

[Srivastava et al. 2014] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*. Journal of Machine Learning Research, 15, pp. 1929-1958. `https://arxiv.org/pdf/1207.0580.pdf`

[Loshchilov and Hutter 2019] Loshchilov, I. and Hutter, F. (2019). *Decoupled Weight Decay Regularization*. In International Conference on Learning Representations (ICLR). `https://arxiv.org/pdf/1711.05101.pdf`

[Krogh and Hertz 1991] Krogh, A. and Hertz, J. (1991). *A Simple Weight Decay Can Improve Generalization*. Advances in Neural Information Processing Systems 4 (NIPS 1991).

[LeCun 1989] LeCun, Y. (1989). *Generalization and Network Design Strategies*. Department of Computer Science, University of Toronto.

[LeCun et al. 1998] LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). *Gradient-Based Learning Applied to Document Recognition*. Proceedings of the IEEE, 86(11), pp. 2278-2324.

[Cybenko 1989] Cybenko, G. (1989). *Approximation by superpositions of a sigmoidal function*. Mathematics of Control, Signals, and Systems, 2(4), pp. 303–314.

[Rumelhart et al. 1986a] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986a). *Back-Propagation and Other Differentiation Algorithms*. In Parallel Distributed Processing, Vol. 1, Chapter 6.5, pp. 200–220.

[Rumelhart et al. 1986b] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986b). *Learning Internal Representations by Error Propagation*. In Rumelhart, D. E. and McClelland, J. L. (eds.), Parallel Distributed Processing: Explorations in the Microstructure of Cognition, Vol. 1: Foundations. Cambridge: MIT Press. ISBN 0-262-18120-7.

[Rumelhart et al. 1986c] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986c). *Learning representations by back-propagating errors*. Nature, 323(6088), pp. 533–536. doi:10.1038/323533a0.

[Rumelhart et al. 1986d] Rumelhart, D., McClelland, J., and the PDP Research Group. (1986). *Parallel Distributed Processing (PDP)*. MIT Press.

[Mitchell 1997] Mitchell, T. M. (1997). *Machine Learning*. WCB/McGraw-Hill.

[Rosenblatt 1956-1958] Rosenblatt, F. (1956-1958). *Frank Rosenblatt publications*. Cornell Aeronautical Laboratory. `https://www.worldcat.org/title/frank-rosenblatt-publications/oclc/64057171`

[McCulloch and Pitts 1943] McCulloch, W. S. and Pitts, W. (1943). *A logical calculus of the ideas immanent in nervous activity*. Bulletin of Mathematical Biophysics, 5(4), pp. 115-133. https://link.springer.com/article/10.1007/BF02478259

[Chollet 2017] Chollet, F. (2017). *Deep Learning with Python, 1st edition*. Manning Publications Co., Greenwich, CT, USA.

[Samuel 1962] Samuel, A. L. (1962). *Artificial Intelligence: A Frontier of Automation*. The Annals of the American Academy of Political and Social Science. https://www.jstor.org/stable/1033694

[Howard and Gugger 2020] Howard, J. and Gugger, S. (2020). *Deep Learning for Coders with Fastai and Pytorch: AI Applications Without a PhD*. O'Reilly Media, Incorporated. https://books.google.no/books?id=xd6LxgEACAAJ

[Mitchell 1997] Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill, New York.

[Russell and Norvig 2009] Russell, S. and Norvig, P. (2009). *Artificial Intelligence: A Modern Approach, 3rd edition*. Prentice Hall Press, USA.

[Hastie et al. 2001] Hastie, T., Tibshirani, R., and Friedman, J. (2001). *The Elements of Statistical Learning*. Springer Series in Statistics. Springer New York Inc., New York, NY, USA.

[Thomas 2017] Thomas, R. (2017). *How (and why) to create a good validation set*. FastAI Blog, 13 Nov 2017.

[Berrar 2018] Berrar, D. (2018). *Cross-Validation*. In Encyclopedia of Bioinformatics and Computational Biology.

[Mockus 2012] Mockus, J. (2012). *Bayesian Approach to Global Optimization: Theory and Applications*. Springer. https://books.google.com.ar/books?id=VuKoCAAAQBAJ

[Joan.domenech91] Joan.domenech91. (2016). *Metodo de retención.jpg*. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:Metodo_de_retenci%C3%B3n.jpg

[Minsky and Papert 1969] Minsky, M. and Papert, S. (1969). *Perceptrons: An Introduction to Computational Geometry*. M.I.T. Press, Cambridge, Mass. Reviewed by Allen Newell in Science, 165(3895), pp. 780-782. http://science.sciencemag.org/content/165/3895/780

[Parodi 2014] Parodi, M. (2014). *Verificación Automática de Firmas Manuscritas*. Tesis Doctoral, Universidad Nacional de Rosario, Facultad de Ciencias Exactas, Ingeniería y Agrimensura, Laboratorio de Sistemas Dinámicos y Procesamiento de la Información, CIFASIS, CONICET.

[Sammut et al. 2012] Sammut, C., Webb, G., and Ting, K. M. (2012). *Encyclopedia of Machine Learning*. Springer. doi:10.1007/978-0-387-30164-8. ISBN 978-0-387-30164-8.

[Fawcett 2006] Fawcett, T. (2006). *An Introduction to ROC Analysis*. Pattern Recognition Letters, 27(8), pp. 861–874. doi:10.1016/j.patrec.2005.10.010.

[Martin et al. 1997] Martin, A., Doddington, G., Kamm, T., Ordowski, M., and Przybocki, M. (1997). *The DET Curve in Assessment of Detection Task Performance*. Proc. Eurospeech '97, Rhodes, Greece, Vol. 4, pp. 1895-1898.

[Akbani et al. 2004] Akbani, R., Kwek, S., and Japkowicz, N. (2004). *Applying support vector machines to imbalanced data sets*. Lecture Notes in Computer Science, vol. 3201, pp. 39–50.

[Japkowicz 2000] Japkowicz, N. (2000). *The class imbalance problem: Significance and strategies*. Proceedings of the 2000 International Conference on Artificial Intelligence (ICAI).

[He and Garcia 2009] He, H. and Garcia, E. (2009). *Learning from imbalanced data*. IEEE Transactions on Knowledge and Data Engineering, 21, pp. 1263–1284.

[Cortes and Vapnik 1995] Cortes, C. and Vapnik, V. (1995). *Support-vector networks*. Machine Learning, 20(3), pp. 273–297.

[Boser et al. 1992] Boser, B., Guyon, I., and Vapnik, V. (1992). *A training algorithm for optimal margin classifier*. Proceedings of the Fifth Annual ACM Workshop on Computational Learning Theory, 5.

[Vapnik and Chervonenkis 1971] Vapnik, V. N. and Chervonenkis, A. Y. (1971). *On the uniform convergence of relative frequencies of events to their probabilities*. Theory of Probability and its Applications, 16(2), pp. 264–280.

[Vapnik and Chervonenkis 1974] Vapnik, V. and Chervonenkis, A. (1974). *Theory of Pattern Recognition*. Nauka, Moscow. (German Translation: W. Wapnik & A. Tscherwonenkis, Theorie der Zeichenerkennung, Akademie–Verlag, Berlin, 1979).

[Fernandez-Lozano et al.] Fernandez-Lozano, C. et al. *Hybrid Model Based on Genetic Algorithms and SVM Applied to Variable Selection within Fruit Juice Classification*.

[Quinlan 1986] Quinlan, J. R. (1986). *Induction of decision trees*. Machine Learning, 1(1), pp. 81–106.

[Quinlan 1993] Quinlan, J. R. (1993). *C4.5: Programs for Machine Learning*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.

[Kuhn and Johnson 2013] Kuhn, M. and Johnson, K. (2013). *Applied Predictive Modeling*. Springer.

[Raileanu and Stoffel 2004] Raileanu, L. and Stoffel, K. (2004). *Theoretical comparison between the gini index and information gain criteria*. Annals of Mathematics and Artificial Intelligence, 41, pp. 77–93.

[Breiman 2001] Breiman, L. (2001). *Random forests*. Machine Learning, 45, pp. 5–32.

[Breiman 1996] Breiman, L. (1996). *Bagging predictors*. Machine Learning, 24(2), pp. 123–140.

[Han et al. 2012] Han, J., Kamber, M., and Pei, J. (2012). *Data Mining: Concepts and Techniques, third edition*. Morgan Kaufmann Publishers, Waltham, Mass.

[Zhang et al. 2017] Zhang, H., Cisse, M., Dauphin, Y. N., and Lopez-Paz, D. (2017). *Mixup: Beyond Empirical Risk Minimization*. In International Conference on Learning Representations (ICLR). `https://arxiv.org/abs/1710.09412`

[Pividori 2019] Pividori, M. (2019). *Invertibilidad de un Generador Entrenado Adversariamente*. Tesina de Grado, Universidad Nacional de Rosario, Departamento de Ciencias de la Computación, Facultad de Ciencias Exactas, Ingeniería y Agrimensura.

[Zanetti 2019] Zanetti, A. (2019). *Codificación y Reconstrucción de Rostros con Redes Adversarias Generativas*. Tesina de Grado, Universidad Nacional de Rosario, Departamento de Ciencias de la Computación, Facultad de Ciencias Exactas, Ingeniería y Agrimensura.

[Apostol] Apostol, T. *Mathematical Analysis, Second Edition*. Addison-Wesley.

[Deisenroth et al.] Deisenroth, M. P., Faisal, A. A., and Ong, C. S. (2020). *Mathematics For Machine Learning*. Cambridge University Press.

[Smith 2015] Smith, L. N. (2015). *Cyclical Learning Rates for Training Neural Networks*. U.S. Naval Research Laboratory, Code 5514, 4555 Overlook Ave., SW., Washington, D.C. 20375. `https://arxiv.org/abs/1506.01186` doi:10.48550/ARXIV.1506.01186

[Smith and Topin 2017a] Smith, L. N. and Topin, N. (2017). *Exploring loss function topology with cyclical learning rates*. U.S. Naval Research Laboratory, Code 5514, 4555 Overlook Ave., SW., Washington, D.C. 20375. University of Maryland, Baltimore County, Baltimore, MD 21250. `https://arxiv.org/abs/1702.04283` doi:10.48550/ARXIV.1702.04283

[Smith and Topin 2017b] Smith, L. N. and Topin, N. (2017). *Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates*. U.S. Naval Research Laboratory, Code 5514, 4555 Overlook Ave., SW., Washington, D.C. 20375. University of Maryland, Baltimore County, Baltimore, MD 21250. `https://arxiv.org/abs/1708.07120` doi:10.48550/ARXIV.1708.07120

[Smith 2018] Smith, L. N. (2018). *A disciplined approach to neural network hyper-parameters: Part 1 – learning rate, batch size, momentum, and weight decay*. `https://arxiv.org/abs/1803.09820` doi:10.48550/ARXIV.1803.09820

[Aarts and Korst 1988] Aarts, E. and Korst, J. (1988). *Simulated Annealing and Boltzmann Machines: A Stochastic Approach to Combinatorial Optimization and Neural Computing*. Wiley. `https://dl.acm.org/doi/abs/10.5555/61990`

[Dauphin et al. 2015] Dauphin, Y. N., de Vries, H., Chung, J., and Bengio, Y. (2015). *Rmsprop and equilibrated adaptive learning rates for non-convex optimization*. `https://arxiv.org/abs/1502.04390` doi:10.48550/ARXIV.1502.04390

[Stevens and Antiga 2019] Stevens, E. and Antiga, L. (2019). *Deep Learning with PyTorch Essential Excerpts*. Manning Publications. `www.manning.com`

[Zeiler and Fergus 2014] Zeiler, M. D. and Fergus, R. (2014). *Visualizing and Understanding Convolutional Networks*. Dept. of Computer Science, Courant Institute, New York University. `https://arxiv.org/pdf/1311.2901.pdf`

[Hinton et al. 2012] Hinton, G. E., Srivastava, N., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. R. (2012). *Improving neural networks by preventing co-adaptation of feature detectors*. arXiv:1207.0580.

[Dumoulin and Visin 2018] Dumoulin, V. and Visin, F. (2018). *A guide to convolution arithmetic for deep learning*. Université de Montréal, Politecnico di Milano. `https://arxiv.org/pdf/1603.07285.pdf`

[Denker and Schwartz 1987] Denker, J. and Schwartz, D. (1987). *Large automatic learning, rule extraction, and generalization*. Complex Systems, 1(5), pp. 877–922.

[Patarnello and Carnevali 1987] Patarnello, S. and Carnevali, P. (1987). *Learning networks of neurons with boolean logic*. EPL (Europhysics Letters), 4(4), p. 503.

[He et al. 2015] He, K., Zhang, X., Ren, S., and Sun, J. (2015). *Deep Residual Learning for Image Recognition*. `https://arxiv.org/abs/1512.03385`

[Simonyan and Zisserman 2014] Simonyan, K. and Zisserman, A. (2014). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. `https://arxiv.org/abs/1409.1556`

[He et al. 2018] He, T., Zhang, Z., Zhang, H., Zhang, Z., Xie, J., and Li, M. (2018). *Bag of Tricks for Image Classification with Convolutional Neural Networks*. `https://arxiv.org/abs/1812.01187`

[Yosinski et al. 2014] Yosinski, J., Clune, J., Bengio, Y., and Lipson, H. (2014). *How transferable are features in deep neural networks?* `https://arxiv.org/abs/1411.1792`

[SigComp 2009] Muhammad Imran Malik *ICDAR 2009 Signature Verification Competition*
ID:SigComp2009_1 `https://tc11.cvc.uab.es/datasets/SigComp2009_1`

[SigComp 2011] Marcus Liwicki, Michael Blumenstein, Elisa van den Heuvel, Charles E.H. Berger, Reinoud D. Stoel, Bryan Found, Xiaohong Chen, Muhammad Imran Malik. *SigComp11: Signature Verification Competition for On- and Offline Skilled Forgeries*
Proc. 11th Int. Conference on Document Analysis and Recognition, 2011 `http://www.iapr-tc11.org/mediawiki/index.php/ICDAR_2011_Signature_Verification_Competition_(SigComp2011)`

[IPB12] G. Pirlo D. Impedovo y D. Barbuzzi. *Multi-classifier system configuration using genetic algorithms.* In *Proc. of International Conference on Frontiers in Handwriting Recognition (ICFHR).* pages 560–564. 2012. isbn: 38(5):609–635, 2008.

[IPP12] D. Impedovo, G. Pirlo, and R. Plamondon. *Handwritten signature verification: New advancements and open issues.* In *In Proc. of 13th Int. Conf. on Frontiers in Handwriting Recognit.* pages 367–372, Bari, Italy, September 2012.

[IP08] D. Impedovo y G. Pirlo. *Automatic signature verification: The state of the art. IEEE Trans. on Systems, Man, and Cybernetics – Part C: Application and Reviews.* isbn: 38(5):609–635, 2008.

[OSK05] E. Özgündüz, T. Sentürk, and M. E. Karsligil. *Off-line signature verification and recognition by support vector machine.* In *European Signal Processing Conf.* Turkey, 2005.

[FSV06] E. Frias-Martinez, A. Sanchez, and J. Velez. signature recognition. *Support vector machines versus multi-layer perceptrons for efficient off-line Engineering Appl. of Artif. Intell.*, pages 693–704, 2006.

[MLD11] M.I. Malik, M. Liwicki, and A. Dengel. *Evaluation of local and global features for offline signature verification.* In *Proceedings of the 1st International Workshop on Automated Forensic Handwriting Analysis* pages 31–35, Beijing, China, Sept. 2011.

[HYW04] W. Hou, X. Ye, and K. Wang. *A survey of off-line signature verification.* In *Proc. International Conference on Digital Object Identifier, Intelligent Mechatronics and Automation* 2004

[PG10a] M. Parodi and J.C. Gómez. *A circular grid-based feature extraction approach for off-line signatura verification.* In *In Proceedings of the XI Argentine Symposium on Artifi- cial Intelligence (ASAI2010)* pages 163–174, CABA, Argentina, 2010.

[PG10b] M. Parodi and J.C. Gómez. *Off-line signature verification: A circular grid-based feature extraction approach.* In *In Proc. 36th Latinamerican Conf. on Informatics (CLEI 2010)* Asuncion, Paragauy, October 2010.

[PGB11] M. Parodi and J.C. Gómez. *A circular grid-based rotation invariant feature extraction approach for off-line signature verification.* In *Proc. of the Int. Conf.on Doc. Anal. and Recognit.* pages 1289–1293, Beijing. China, Sept. 2011.

[HMM05] Y. Hongo, D. Muramatsu, and T. Matsumoto. *Adaboost-based on-line signature verifier.* In A.K. Jain and N.K. Ratha, editors, *Proc. SPIE, Biometric Technology for Human Identification II* Orlando, USA, April 2005.

[HC13] J. Hu and Y. Chen. *Offline signature verification using real adaboost classifier combination of pseudo-dynamic features.* In A.K. Jain and N.K. Ratha, editors, *Proc. International Conference on Document Analysis and Recognition* 2013.

[DNP98] F. Nouboud S. Djeziri y R. Plamondon. *Extraction of signatures from check background based on a filiformity criterion. IEEE Trans. Image Process.* 1998. isbn: 7(10):1425–1438.

[GW08] R.C. Gonzalez and R.E. Woods. *Digital Image Processing.* Prentice Hall, 3rd edition, 2008.

[GWE09] R.C. Gonzalez, R.E. Woods, and S.L. Eddins. *Digital Image Processing.* Gatesmark Publishing, 2nd edition, 2009.

[FAT05] M.A. Ferrer, J.B. Alonso, and C.M. Travieso. *Offline geometric parameters for automatic signature verification using fixed-point arithmetic.* In *IEEE Trans. on Pattern Anal. and Machine Intell.* 27(5):993–997, 2005.

[FGSD02] M. Fuentes, S. Garcia-Salicetti, and B. Dorizzi. *On-line signature verification: Fusion of a hidden markov model and a neural network via a support vector machine.* In *Proc. 8th Int. Workshop Front. Handwriting Recognit.* pages 253–258, Niagara-on-the-Lake, ON, Canada, Aug. 2002.

[JBS05] E. Justino, F. Bortolozzi, and R. Sabourin. *A comparison of SVM and HMM classifiers in the offline signature verification.* In *Pattern Recognit. Letters* 26(9):1377–1385, July 2005.

[ASVE13] F.A. Alsulaiman, N. Sakr, J.J. Valds, and A. El Saddik. *A comparison of SVM and HMM classifiers in the offline signature verification.* In *Pattern Recognit. Letters* 26(9):1377–1385, July 2005.

[PFV11] J.M. Pascual-Gaspar, M. Faundez-Zanuy, and C. Vivaracho. *Fast on-line signature recognition based on vq with time modeling.* In *Eng. Appl. of Artific. Intell.* 24(2):368–377, 2011.

[AYF90] M. Ammar, Y. Yoshida, and T. Fukumura. *Structural description and classification of signature images.* In *Pattern Recognit.* 23:697–710, 1990.

[SPB94] R. Sabourin, R. Plamondon, and L. Beaumier. *Structural interpretation of handwritten signature images* In *Int. J. Pattern Recognit. Artif. Intell. (IJPRAI),* pages 69–108, 1994.

[SD92] R. Sabourin and J.P. Drouhard. *Offline signature verification using directional PDF and neural networks.* In *Proc. 11th Int. Conf. Pattern Recog., Conf. B: Pattern Recog. Methodol. Syst.* pages 321–325, The Hague, The Netherlands, 1992.

[DSG94] J.P. Drouhard, R. Sabourin, and M. Godbout. *A neural network approach to offline signature verification using directional pdf.* In *Pattern Recognit.* 29(3):415–424, 1994.

[ZNA05] E.N. Zois, A.A. *Nassiopoulos, and V. Anastassopoulos. Signature verification based on line directionality.* In *Proc. 2005 IEEE Workshop on Signal Processing Systems Design and Implementation* pages 343–346, 2005.

[JGC02] A.K. Jain, F.D. Griess, and S.D. Connell. *On-line signature verification.* In *Pattern Recognit.* 35:2963–2972, 2002.

[KSX04] M.K. Kalera, S. Srihari, and A. Xu *Offline signature verification and identification using distance statistics.* In *Int.J. Pattern Recognit. Artif. Intell. (IJPRAI)* 18(6):1339–1360, 2004.

[NR77] R.N. Nagel and A. Rosenfeld. *Computer detection of freehand forgeries.* In *IEEE Trans. Computers* 26(9):895–905, 1977.

[SP86] R. Sabourin and R. Plamondon. *Preprocessing of handwritten signatures from image gradient analysis.* In *Proc. 8th Int. Conf. Pattern Recognition* pages 576–579, 1986.

[JS01] E. Justino and F. Bortolozziand R. Sabourin. *Off-line signature verification using HMM for random, simple and skilled forgeries.* In *Proc. of Int. Conf. on Doc. Anal. and Recognit.*
pages 105–110, Seattle, USA, 2001.

[JYBS00] E. Justino, A. El Yacoubi, F. Bortolozzi, and R. Sabourin. *An off-line signature verification system using HMM and graphometric features.* In *Proc. of 4th IAPR Int. Work-shop on Doc. Anal. Syst.*
pages 211–222, Rio de Janeiro, Brazil, 2000.

[SJBS04] C. Santos, E. Justino, F. Bortolozzi, and R. Sabourin. *An off-line signature verification method based on the questioned document experts approach and a neural network classifier.* In *Proc. of the 9th Int. Workshop on Frontiers in Handwriting Recognition (IWFHR-9)*
2004.

[NP94] F. Nouboud and R. Plamondon. *Global parameters and curves for offline signature verification.* In *Proc. 4th Int. Workshop Front. Handwriting Recognit. (IWFHR-4)*
pages 145–155, Taipei, Taiwan, 1994.

[OJFS05] L.S. Oliveira, E. Justino, C. Freitas, and R. Sabourin. *The graphology applied to signature verification.* In *Proc. of the 12th Conf. of the Int. Graphonomics Soc.*
2005

[BZZE13] K. Barkoula, E.N. Zois, E. Zervas, and G. Economou. *Off-line signature verification based on ordered grid features: An evaluation.* In *Proc. 2nd International Workshop On Automated Forensic Handwritten Analysis (AFHA 2013)*
pages 36–40, 2013.

[UNR GSF] UNR y Gobierno de la Provincia de Santa Fe *Convenio conjunto entre el gobierno de la provincia de Santa Fe y la Universidad Nacional de Rosario. Descripción General del Proceso de Escrutinio Provisorio con Asistencia Digital.*
Reporte técnico, Elecciones Provinciales, Argentina, 2019.

[UNR GSF QRs] UNR y Gobierno de la Provincia de Santa Fe *Convenio conjunto entre el gobierno de la provincia de Santa Fe y la Universidad Nacional de Rosario. Formato de los Códigos de Respuesta Rápida - QR.*
Reporte técnico, Elecciones Provinciales, Argentina, 2019.

[Informe 2019 UNR GSF] Equipo de trabajo UNR dirigido por Dante Zanarini. *Informe sobre el Sistema de Escrutinio Provisorio con Asistencia Digital.*
Elecciones Generales 2019. Reporte técnico, Universidad Nacional de Rosario, Argentina, 2019.

[S Shirdhonkar 2010] M. S. Shirdhonkar, Manesh Kokare *Off-Line Handwritten Signature Retrieval using Curvelet Transforms*
(IJCSIS) International Journal of Computer Science and Information Security, Vol.8, No.8, 2010

[N Sharma et al 2018] N. Sharma, R. Mandal, R. Sharma, U. Pal and M. Blumenstein *"Signature and Logo Detection using Deep CNN for Document Image Retrieval" 2018 16th International Conference on Frontiers in Handwriting Recognition (ICFHR)*
2018, pp. 416-422, doi: 10.1109/ICFHR-2018.2018.00079.

[Lin et al 2015] Lin, K., Yang, H. F., Hsiao, J. H., Chen, C. S. (2015) *Deep Learning of Binary Hash Codes for Fast Image Retrieval*
In Proceedings of the IEEE conference on computer vision and pattern recognition workshops (pp. 27-35).

[ML12] M.I. Malik and M. Liwick. *From terminology to evaluation: Performance assessment of automatic signature verification systems.*
In Proc. of International Conference on Frontiers in Handwriting Recognition, pages 613–618, Bari, Italy, 2012.

[Forgery Detection CNN 2018] S Jerome Gideon, Anurag Kandulna, Aron Abhishek Kujur, A Diana, Kumudha Raimond, *Handwritten Signature Forgery Detection using Convolutional Neural Networks*
In Procedia Computer Science, Volume 143, 2018, Pages 978-987, ISSN 1877-0509, doi: `https://doi.org/10.1016/j.procs.2018.10.336.`

[Signature Verification using NN 2012] Ashwini Pansare , Shalini Bhatia *Handwritten Signature Verification using Neural Network*
In International Journal of Applied Information Systems (IJAIS) – ISSN : 2249-0868 Foundation of Computer Science FCS, New York, USA Volume 1– No.2, January 2012 – `www.ijais.org`

[Image similarity using Deep CNN Amazon 2017] Srikar Appalaraju, Vineet Chaoji *Image similarity using Deep CNN and Curriculum Learning*
Amazon Development Centre (India) Pvt. Ltd.

[Distance Metric Learning Relative Comparisons 2003] Schultz, M., & Joachims, T. (2003). *Learning a distance metric from relative comparisons.*
Advances in neural information processing systems, 16.

[Distance Metric Learning Large Margin 2005] Kilian Q. Weinberger, John Blitzer and Lawrence K. Saul *Distance metric learning for large margin nearest neighbor classification*
In Advances in neural information processing systems, 18 (NEURIPS 2005) `https://proceedings.neurips.cc/paper/2005/file/a7f592cef8b130a6967a90617db5681b-Paper.pdf`

[Signature Verification Siamese NN] Jane Bromley, Isabelle Guyon, Yann LeCun, Eduard Sickinger and Roopak Shah *Signature Verification using a "Siamese" Time Delay Neural Network*
In NEURIPS, 1993 `https://proceedings.neurips.cc/paper/1993/file/288cc0ff022877bd3df94bc9360b9c5d-Paper.pdf`

[Face Verification LeCun 2005] S. Chopra, R. Hadsell and Y. LeCun. *Learning a similarity metric discriminatively, with application to face verification*
2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05), San Diego, CA, USA, 2005, pp. 539-546 vol. 1, doi: 10.1109/CVPR.2005.202.

[Dimensionality Reduction Invariant Mapping Lecun 2005] Raia Hadsell, Sumit Chopra, Yann LeCun *Dimensionality Reduction by Learning an Invariant Mapping*
In CPVR, 2006

[Recognize visual objects classes 2006] Li Fei-Fei *Knowledge transfer in learning to recognize visual objects classes*
In ICDL, 2006

[Signature Verification SOTA Review 2008] Impedovo, D., & Pirlo, G. *Automatic signature verification: The state of the art.*
In IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), 38(5), 609-635.

[Deep Learning Face Representation 2014] Yi Sun, Xiaogang Wang, Xiaoou Tang *Deep Learning Face Representation by Joint Identification-Verification*
In NeurIPS, 2014

[DeepFace 2014 Facebook & Tel Aviv University] Yaniv Taigman, Ming Yang, Marc'Aurelio Ranzato, Lior Wolf *DeepFace: Closing the Gap to Human-Level Performance in Face Verification*
In Conference on Computer Vision and Pattern Recognition (CVPR)

[Human level concept learning 2015] Brenden M. Lake, Ruslan Salakhutdinov, Joshua B. Tenenbaum *Human-level concept learning through probabilistic program induction*
In Science, 2015

[Triplet Network 2015] Elad Hoffer and Nir Ailon *DEEP METRIC LEARNING USING TRIPLET NETWORK*
ICLR 2015

[Siamese NN for One-shot] Gregory Koch, Richard Zemel, Ruslan Salakhutdinov *Siamese Neural Networks for One-shot Image Recognition*
ICML deep learning workshop, 2015 `https://www.cs.cmu.edu/~rsalakhu/papers/oneshot1.pdf`

[FaceNet 2015 Google] Florian Schroff, Dmitry Kalenichenko, James Philbin *FaceNet: A Unified Embedding for Face Recognition and Clustering*
In Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition 2015 `https://arxiv.org/pdf/1503.03832.pdf`

[Person Re-Identification 2015] Ejaz Ahmed, Michael Jones, Tim K. Marks *An Improved Deep Learning Architecture for Person Re-Identification*
In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015, pp. 3908-3916

[Matching Networks for One Shot Learning 2016] Oriol Vinyals, Charles Blundell, Timothy Lillicrap, koray kavukcuoglu, Daan Wierstra *Matching Networks for One Shot Learning*
In Part of Advances in Neural Information Processing Systems 29 (NIPS 2016)

[In Defense of the Triplet Loss 2017] Alexander Hermans, Lucas Beyer, Bastian Leibe *In Defense of the Triplet Loss for Person Re-Identification*
`https://arxiv.org/pdf/1703.07737.pdf`

[Handwritten Text Segmentation 2019] Junho Jo, Hyung Il Koo, Jae Woong Soh, Nam Ik Cho *Handwritten Text Segmentation via End-to-End Learning of Convolutional Neural Network*
arXiv preprint arXiv:1906.05229, 2019 `https://arxiv.org/abs/1906.05229`

[Offline Handwritten Signature Verification Literature Review 2017] Luiz G. Hafemann, Robert Sabourin and Luiz S. Oliveira *Offline Handwritten Signature Verification - Literature Review*
In 2017 seventh international conference on image processing theory, tools and applications (IPTA) (pp. 1-8). IEEE.

[Omniglot review 2019] Brenden M. Lake, Ruslan Salakhutdinov, and Joshua B. Tenenbaum *The Omniglot challenge: a 3-year progress report*
In Current Opinion in Behavioral Sciences, 29, 97-104.

[Zeleni9 s.f.] Zeleni9. (s.f.). *pytorch-wgan*. Repositorio GitHub. `https://github.com/Zeleni9/pytorch-wgan/blob/master/main.py`

[Suran 2020] Suran, A. (2020). *Deep Convolutional vs Wasserstein Generative Adversarial Network*. Towards Data Science. `https://towardsdatascience.com/deep-convolutional-vs-wasserstein-generative-adversarial-network-183fbcfdce1f`

[Graves 2013] Graves, A. (2013). *Generating Sequences With Recurrent Neural Networks*. arXiv:1308.0850 [cs.NE]. `https://arxiv.org/abs/1308.0850`